

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC NÔNG LÂM TP HCM

KHOA CÔNG NGHỆ THÔNG TIN



TIỂU LUẬN TỐT NGHIỆP

**XÂY DỰNG NỀN TẢNG THƯƠNG MẠI ĐIỆN
TỬ THIẾT BỊ ĐIỆN TỬ TÍCH HỢP TRÍ TUỆ
NHÂN TẠO VÀ CHATBOT SỬ DỤNG SPRING
BOOT, REACTJS VỚI HỆ THỐNG GỢI Ý SẢN
 PHẨM CÁ NHÂN HÓA**

Ngành	: Công nghệ thông tin
Niên khoá	: 2022 – 2026
Lớp	: DH22DTB
Sinh viên thực hiện	: Từ Quang Chương

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NÔNG LÂM TP HCM KHOA
CÔNG NGHỆ THÔNG TIN



TIỂU LUẬN TỐT NGHIỆP

XÂY DỰNG NỀN TẢNG THƯƠNG MẠI ĐIỆN
TỬ THIẾT BỊ ĐIỆN TỬ TÍCH HỢP TRÍ TUỆ
NHÂN TẠO VÀ CHATBOT SỬ DỤNG SPRING
BOOT, REACTJS VỚI HỆ THỐNG GỢI Ý SẢN
 PHẨM CÁ NHÂN HÓA

CÁN BỘ HƯỚNG DẪN

TS. Nguyễn Thị Phương Trâm

SINH VIÊN THỰC HIỆN

Từ Quang Chương (MSSV: 22130029)

TP.HỒ CHÍ MINH, tháng 12 năm 2025

DANH SÁCH CHỮ VIẾT TẮT

AI	Artificial Intelligence Trí tuệ nhân tạo
API	Application Programming Interface Giao diện lập trình ứng dụng
TMĐT	Thương Mại Điện Tử
JWT	Json Web Token Token web JSON
REST	Representational State Transfer Chuyển đổi trạng thái biểu diễn
CRUD	Create, Read, Update, Delete Tạo, Đọc, Cập nhật, Xóa

DANH MỤC HÌNH ẢNH

Hình 2.1: Minh họa về kiến trúc Microservices	9
Hình 2.2: So sánh giữa Monolithic và Microservices	11
Hình 2.3: Mô tả về hệ thống API Gateway	15
Hình 2.4: Mô tả cơ bản về cách RESTful API hoạt động	17
Hình 2.5: Mô tả cơ bản về cách RESTful API hoạt động	19
Hình 2.6: Mô tả mô hình Spring Cloud.....	21
Hình 2.7: Logo của Spring Boot.....	22
Hình 2.8: Logo của React Framework	23
Hình 2.9: Logo của MySQL.....	25
Hình 2.10: Logo của Redis.....	26
Hình 2.11: Logo của Gemini	29
Hình 3.1: Sơ đồ kiến trúc hệ thống.....	35
Hình 3.2: Sơ đồ usecase của hệ thống.	40
Hình 3.3: Lược đồ use case tổng quát của hệ thống.....	40
Hình 3.4: Lược đồ use case Xem sản phẩm	41
Hình 3.5: Lược đồ use case quản lý đơn đặt hàng của khách hàng.....	41
Hình 3.6: Lược đồ use case quản lý giỏ hàng	42
Hình 3.7: Lược đồ use case quản lý khuyến mãi của cửa hàng	42
Hình 3.8: Lược đồ use case quản lý hệ thống sàn thương mại điện tử	43
Hình 3.9: Lược đồ activity cho usecase mua hàng.....	56
Hình 3.10: Lược đồ activity cho usecase login	57
Hình 3.11: Lược đồ tuần tự cho usecase mua hàng.....	58
Hình 3.12: Lược đồ tuần tự cho usecase login.	59

Hình 3.13: Lược đồ tuần tự cho usecase bình chọn.	60
Hình 3.14: Giao diện Chatbot tư vấn sản phẩm sử dụng Gemini AI	62
Hình 3.15: Các thư viện Machine Learning được sử dụng trong Python Service	63
Hình 3.16: Logic xử lý gợi ý sản phẩm tại Python Service	64
Hình 3.17: Giao diện hiển thị danh sách sản phẩm gợi ý dựa trên AI	64
Hình 3.18: Màn hình đăng nhập của hệ thống.	65
Hình 3.19: Màn hình đăng ký.....	65
Hình 3.20: Màn hình trang chủ.....	66
Hình 3.21: Màn hình chức năng đổi thông tin cá nhân.	66
Hình 3.22: Màn hình chức năng đổi mật khẩu.	67
Hình 3.23: Màn hình trang chi tiết giỏ hàng.	67
Hình 3.24: Giao diện của trang khi điền thông tin để ra giá tiền giao hàng.....	68
Hình 3.25: Chọn voucher phù hợp với đơn hàng.	69
Hình 3.26: Hình các phương thức thanh toán.	69
Hình 3.27: Khi khách hàng mà chọn vnpay.	70
Hình 3.28: Màn hình trang chi tiết đặt hàng thành công.....	71
Hình 3.29: Hình lịch sử đơn hàng.	72
Hình 3.30: Màn hình admin quản lý các đơn hàng được tạo.	73
Hình 3.31: Màn hình chỉnh sửa trạng thái đơn.....	73
Hình 3.32: Trạng thái đơn hàng đã được update	74
Hình 3.33: Màn hình danh sách người dùng của hệ thống.....	75
Hình 3.34: Màn hình phân quyền.	75
Hình 3.35: Màn hình trang analytics mà admin quản lý	77
Hình 3.36: Màn hình thống kê của admin	77
Hình 3.37: Lược đồ cơ sở dữ liệu của hệ thống products.	78

Hình 3.38: Giao diện phần dữ liệu request test.....	83
Hình 3.39: Giao diện phần cài đặt số lượng để test liên tục	83
Hình 4.40 : Giao diện phần cấu hình.....	84
Hình 4.41: Kết quả add cart.....	85
Hình 5.1: Giao diện trang đăng nhập dành cho khách hàng.....	93
Hình 5.2: Giao diện trang đăng ký cho khách hàng.	93
Hình 5.3: Giao diện quên mật khẩu.....	93
Hình 5.4: Giao diện trang chủ	94
Hình 5.5: Giao diện tìm kiếm sản phẩm.....	95
Hình 5.6: Giao diện gợi ý sản phẩm.....	95
Hình 5.7: Giao diện mục sản phẩm sale	96
Hình 5.8: Giao diện trang chi tiết sản phẩm.....	97
Hình 5.9: Giao diện sản phẩm trong giỏ hàng.....	97
Hình 5.10: Giao diện trang nhập thông tin giao hàng.	98
Hình 5.11: Giao diện áp dụng mã giảm giá.....	98
Hình 5.12: Giao diện tổng khi đang thanh toán.	99
Hình 5.13: Giao diện trang cá nhân người dùng	100
Hình 5.14: Giao diện trang lịch sử đơn hàng.	100
Hình 5.15: Giao diện chatbot.....	101
Hình 5.16: Giao diện trang admin Dashboard và Analytics.....	102
Hình 5.17: Giao diện trang quản lý sản phẩm.....	103
Hình 5.18: Giao diện thêm user và phân quyền.	103
Hình 5.19: Giao diện trang quản lý voucher.	104
Hình 5.20: Giao diện trang quản lý đơn hàng.	104

Hình 5.21: Giao diện trang ghi nhận lại nội dung từ chatbot.	105
Hình 5.22: Giao diện trang đánh giá.	106
Hình 6.1: Trang tải JDK	107
Hình 6.2: Trang tải node.Js.....	108
Hình 6.3: Trang tải python	108
Hình 6.4: Trang tải git	108
Hình 6.5: Trang tải MySQL	109
Hình 6.6: Giao diện phần cấu hình sql	109
Hình 6.7: Cách run lệnh mvn spring-boot:run	110
Hình 6.8: Giao diện run thành công	111
Hình 6.9: Giao diện run frontend	111
Hình 6.10: Giao diện cấu file api key gemini.....	112
Hình 6.11: Giao diện trang lấy api key gemini	112
Hình 6.12: Giao diện trang Spring Eureka	113

DANH MỤC BẢNG

Bảng 2.1: So sánh kiến trúc Monolithic và Microservices.....	11
Bảng 2.2: So sánh giữa SOAP và REST API.....	19
Bảng 3.1: Đặc tả usecase đăng ký.	43
Bảng 3.2: Đặc tả usecase đăng nhập.	44
Bảng 3.3: Đặc tả usecase cập nhật thông tin cá nhân.....	45
Bảng 3.4: Đặc tả usecase đổi mật khẩu	47
Bảng 3.5: Đặc tả usecase mua hàng	47
Bảng 3.6: Đặc tả usecase thanh toán.	52
Bảng 4.1: Kiểm thử Stress test – chức năng test login từ request.....	84
Bảng 4.2: Kiểm thử Load test – chức năng test search products request.....	85
Bảng 4.3: Kiểm thử Load test – chức năng test add cart request	85

TÓM TẮT

Tiểu luận nghiên cứu và xây dựng một nền tảng thương mại điện tử kinh doanh thiết bị điện tử tích hợp trí tuệ nhân tạo và chatbot nhằm nâng cao trải nghiệm người dùng và khả năng cá nhân hóa. Hệ thống được thiết kế theo kiến trúc microservices, trong đó frontend sử dụng ReactJS kết hợp TypeScript và Material-UI, backend phát triển bằng Spring Boot cùng Spring Cloud và cơ sở dữ liệu MySQL. Các dịch vụ được quản lý thông qua Eureka Service Discovery, Spring Cloud Gateway và Redis để tối ưu hiệu năng. Hệ thống tích hợp chatbot trí tuệ nhân tạo sử dụng Gemini API hỗ trợ tìm kiếm, tư vấn và mua sắm 24/7. Chức năng gợi ý sản phẩm cá nhân hóa kết hợp Collaborative Filtering (mô hình SVD) và Content-Based Filtering, triển khai bằng Python FastAPI với Scikit-learn. Ngoài ra, hệ thống áp dụng xác thực JWT kết hợp OAuth2 và cung cấp các chức năng quản lý, thanh toán, vận chuyển và đánh giá sản phẩm.

MỤC LỤC

DANH SÁCH CHỮ VIẾT TẮT	i
DANH MỤC HÌNH ẢNH.....	ii
DANH MỤC BẢNG	vi
TÓM TẮT.....	vii
MỞ ĐẦU	1
1. LÝ DO CHỌN ĐỀ TÀI.....	1
2. MỤC TIÊU VÀ PHẠM VI NGHIÊN CỨU	4
3. Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN	4
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI	6
1. 1. Phân tích, đánh giá các công trình nghiên cứu liên quan	6
1. 2. Những vấn đề còn tồn tại.....	6
1. 3. Nội dung chính của tiểu luận.....	7
CHƯƠNG 2. PHƯƠNG PHÁP VÀ NỘI DUNG NGHIÊN CỨU	8
2.1. Cơ sở lý thuyết	8
2.1.1. Microservices	8
2.1.1.1. Giới thiệu về Microservices	8
2.1.1.2. Những đặc trưng của Microservices.....	9
2.1.1.3. So sánh kiến trúc Microservices và kiến trúc (Monolithic)	11
2.1.1.4. Những khó khăn khi triển khai Microservices	13
2.1.1.5. Kết luận về Microservices	14
2.1.2. Các thành phần giao tiếp và quản lý dịch vụ.....	14
2.1.2.1. API Gateway	14
2.1.2.2. REST API.....	16
2.1.2.3. Spring Cloud Gateway	20

2.2 Các công nghệ hỗ trợ phát triển hệ thống.....	21
2.2.1 Spring Boot.....	21
2.2.1.1 Spring Boot là gì ?.....	21
2.2.1.2. Lý do lựa chọn cho đề tài nghiên cứu	22
2.2.1.3. Ứng dụng vào đồ án	23
2.2.2 React.js và TypeScript.....	23
2.2.2.1. Tổng quan về React.js và TypeScript.....	23
2.2.2.2. Ứng dụng trong xây dựng giao diện người dùng	24
2.2.2.3. Ứng dụng vào đồ án	24
2.2.3 Hệ quản trị cơ sở dữ liệu MySQL	24
2.2.3.1. Tổng quan.....	24
2.2.3.2. Vai trò trong kiến trúc hệ thống	25
2.2.3.3. Ứng dụng vào đồ án	25
2.2.4. Redis	26
2.2.4.1. Tổng quan.....	26
2.2.4.2. Giải pháp tối ưu hiệu năng cho hệ thống.....	26
2.2.4.3. Ứng dụng vào đồ án	26
2.2.5. Eureka Service Discovery	27
2.2.5.1 Khái niệm và vai trò	27
2.2.5.2. Cơ chế hoạt động trong đề tài.....	27
2.2.5.3. Ứng dụng vào đồ án	27
2.2.6. Bảo mật với JWT (JSON Web Token).....	27
2.2.6.1. Tổng quan.....	27
2.2.6.2. Ứng dụng trong xác thực phi trạng thái (Stateless Auth).....	28

2.2.6.3. Ứng dụng vào đồ án	28
2.2.7. Tích hợp OAuth2	28
2.2.7.1. Tổng quan	28
2.2.7.2. Tối ưu trải nghiệm đăng nhập	29
2.2.7.3. Ứng dụng vào đồ án	29
2.2.8. Tích hợp Trí tuệ nhân tạo - Gemini API	29
2.2.8.1. Giới thiệu	29
2.2.8.2. Ứng dụng Chatbot hỗ trợ khách hàng	30
2.2.8.3. Ứng dụng vào đồ án	30
2.2.9. Python FastAPI và Scikit-learn	30
2.2.9.1. Tổng quan	30
2.2.9.2. Ứng dụng trong Hệ thống Gợi ý (Recommender System).....	30
2.2.9.3. Ứng dụng vào đồ án	31
2.2.10. Cổng thanh toán VNPAY	31
2.2.10.1. Giới thiệu	31
2.2.10.2. Ứng dụng thực tiễn.....	31
2.2.10.3. Ứng dụng vào đồ án	31
2.2.11. Dịch vụ vận chuyển GHN (Giao Hàng Nhanh)	32
2.2.11.1. Giới thiệu	32
2.2.11.2. Ứng dụng trong quản lý Logistic	32
2.2.11.3. Ứng dụng vào đồ án	32
2.2.12. Material-UI (MUI).....	32
2.2.12.1. Tổng quan	32
2.2.12.2. Vai trò trong phát triển Frontend.....	33
2.2.12.3. Ứng dụng vào đồ án	33

CHƯƠNG 3. GIẢI PHÁP CHO BÀI TOÁN.....	34
3.1. Đặt vấn đề và phân tích bài toán	34
3.2. Giải pháp công nghệ đề xuất	35
3.2.1. Kiến trúc tổng quan.	35
3.2.2. Sơ đồ kiến trúc hệ thống.....	35
3.2.3. Cơ chế bảo mật.....	37
3.3. Hiện thực giải pháp.....	38
3.3.1. Mô tả giải pháp.....	38
3.3.2. Các chức năng chính đã hiện thực:.....	38
3.3.3. Lược đồ Use case	39
3.3.4. Lược đồ activity.....	56
3.3.5. Lược đồ tuần tự.	58
3.3.6. Các màn hình của hệ thống	60
3.3.6.1 Các chức năng thông minh và tích hợp AI.....	60
3.3.6.2 Các chức năng dành cho khách hàng.	65
3.3.7. Lược đồ cơ sở dữ liệu.....	78
3.3.8. Xây dựng các endpoint API	80
3.3.9. Cơ chế bảo mật JWT	82
3.3.10. Đánh giá hiệu suất hệ thống	82
CHƯƠNG 4. KẾT QUẢ, KẾT LUẬN VÀ KIẾN NGHỊ.....	87
PHỤ LỤC CHO NGƯỜI DÙNG	92
5.1 DÀNH CHO NGƯỜI DÙNG MUA HÀNG.	92
5.1.1 Đăng ký và Đăng nhập	92
5.1.2 Tìm kiếm và Khám phá sản phẩm.....	94

5.1.3 Mua hàng và Thanh toán	96
5.1.4 Quản lý Cá nhân & Đơn hàng	99
5.1.5 Sử dụng Chatbot AI Tư vấn	101
5.2 DÀNH CHO QUẢN TRỊ VIÊN (ADMIN)	101
5.2.1 Tổng quan (Dashboard).....	101
5.2.2 Quản lý Sản phẩm & Người dùng	102
5.2.3 Quản lý Đơn hàng.....	104
5.2.4 Quản lý chatLogs và review sản phẩm.....	105
HƯỚNG DẪN CÀI ĐẶT HỆ THỐNG	107
6.1 TỔNG QUAN HỆ THỐNG.	107
6.2 YÊU CẦU MÔI TRƯỜNG.....	107
6.2.1 Phần mềm cơ bản	107
6.2.2. Cơ sở dữ liệu & Middleware	108
6.3. CẤU HÌNH CƠ SỞ DỮ LIỆU	109
6.4. TRIỂN KHAI BACKEND (MICROSERVICES)	109
6.4.1. Khởi động các dịch vụ lõi (Core Services)	110
6.4.2. Khởi động các dịch vụ nghiệp vụ.....	110
6.5. TRIỂN KHAI AI & MACHINE LEARNING SERVICE	110
6.6 TRIỂN KHAI FRONTEND	111
6.7 CẤU HÌNH TÍCH HỢP AI (GOOGLE GEMINI)	112
6.8 KIỂM TRA TRẠNG THÁI TRIỂN KHAI.....	112

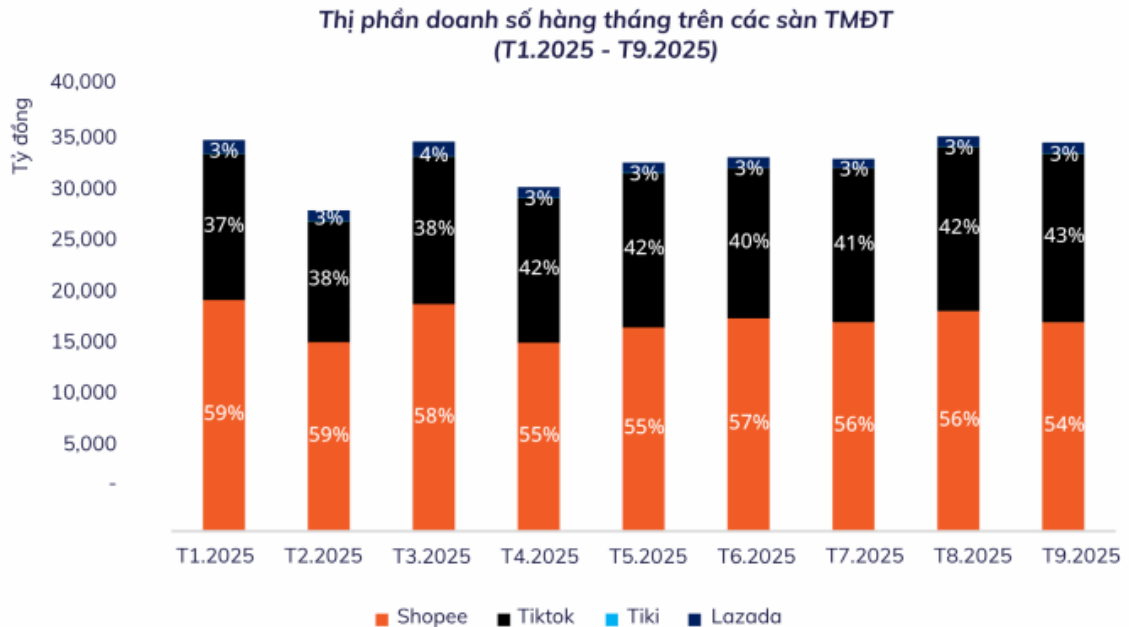
MỞ ĐẦU

1. LÝ DO CHỌN ĐỀ TÀI

Sự phát triển vũ bão của công nghệ thông tin đã tái định hình hoàn toàn phương thức kinh doanh thương mại trên toàn cầu. Từ giai đoạn sơ khởi những năm 2000 với máy tính cá nhân cho đến kỷ nguyên di động và hiện nay là kỷ nguyên của Trí tuệ nhân tạo (AI), thương mại điện tử (E-commerce) đã trở thành một phần không thể thiếu trong đời sống xã hội, thay đổi căn bản thói quen mua sắm của người tiêu dùng từ thụ động sang chủ động tương tác.

Thực tế tại Việt Nam cho thấy một làn sóng dịch chuyển mạnh mẽ từ thương mại truyền thống sang các nền tảng số. Theo "Báo cáo thị trường sàn thương mại điện tử Việt Nam Q3/2025" từ Metric [1], thị trường tiếp tục duy trì đà tăng trưởng kỷ lục bất chấp những biến động kinh tế. Cụ thể, tổng doanh thu của 5 sàn TMĐT hàng đầu (Shopee, Lazada, TikTok Shop, Tiki, Sendo) trong Quý 3/2025 đạt mốc 115.400 tỷ đồng, tăng trưởng 37,2% so với cùng kỳ năm 2024. Đặc biệt, sự thống trị của mô hình Shoppertainment (mua sắm kết hợp giải trí) và xu hướng Livestream bán hàng đã đẩy lưu lượng giao dịch trực tuyến lên mức cao chưa từng có.

Tuy nhiên, sự tăng trưởng "nóng" này đặt ra những thách thức to lớn về mặt công nghệ. Việc hàng triệu người dùng truy cập và giao dịch đồng thời trong các khung giờ "Super Sale" đòi hỏi hệ thống phần mềm phải có khả năng chịu tải cao (High Availability) và hoạt động ổn định 24/7. Các kiến trúc phần mềm Monolithic (nguyên khối) cũ đang dần bộc lộ hạn chế: dễ bị nghẽn cổ chai, khó mở rộng và rủi ro sập toàn bộ hệ thống khi một chức năng gặp lỗi. Bên cạnh đó, khách hàng năm 2025 không chỉ cần mua hàng mà còn cần được thấu hiểu. Các hệ thống cũ thiếu vắng sự hỗ trợ của AI thường thất bại trong việc giữ chân người dùng vì trải nghiệm quá nghèo nàn và thiếu tính cá nhân hóa (Hình 1).[26]



Số liệu thống kê trên 4 sàn Shopee, Lazada, Tiki, TikTok Shop
Dữ liệu trích xuất từ phần mềm số liệu Metric -> [đăng ký sử dụng](#)

Hình 1.1 Tăng trưởng doanh thu theo tháng của các sàn TMĐT tại Việt Nam – Quý 3/2025 so với cùng kỳ

(Nguồn: [Báo cáo thị trường sàn thương mại điện tử Việt Nam Q3/2025](#))

Dù hiện tại, thương mại điện tử vẫn đang phát triển và đã đạt được nhiều bước tiến lớn, vẫn còn tồn đọng rất nhiều vấn đề khi tăng trưởng quá nhanh. Đầu tiên, khi lượng khách hàng tăng cao, vấn đề đảm bảo khả năng sử dụng khi có một lượng lớn khách hàng truy cập vào cùng một thời điểm vẫn còn nhiều hạn chế. Bên cạnh đó, các yếu tố về xây dựng hệ thống dữ liệu, kiến trúc hệ thống làm sao để đảm bảo tốc độ thực thi, sự chính xác của dữ liệu và đưa ra các con số thống kê cũng là một thách thức lớn.

Để giải quyết những vấn đề đó, em đã bắt tay vào nghiên cứu giải pháp để làm sao xây dựng một sàn thương mại điện tử. Với yêu cầu là đảm bảo hệ thống vận hành trơn tru khi lượng truy cập tăng đột biến, mà vẫn mang lại trải nghiệm tuyệt vời dành cho khách hàng và người bán. Ngoài ra, khả năng mở rộng quy mô của sàn thương mại điện tử khi

tăng trưởng và mở rộng cũng là tiêu chí hàng đầu để em tìm hiểu các công nghệ có thể đáp ứng được điều này.

Sau quá trình nghiên cứu, em đã quyết định lựa chọn kiến trúc Microservice để áp dụng vào bài toán xây dựng sàn thương mại điện tử để đáp ứng với các yêu cầu trên. Kiến trúc Microservice có rất nhiều ưu điểm khi sử dụng cho hệ thống thương mại điện tử như:

- Kiểm soát lỗi và khả năng chịu lỗi: Cho phép quản lý các lỗi phát sinh, bao gồm khả năng phát hiện và xử lý lỗi mà không làm ảnh hưởng đến toàn bộ hệ thống. Hệ thống có khả năng cách ly lỗi để duy trì sự ổn định và tiếp tục hoạt động.
- Khả năng mở rộng và thu nhỏ dịch vụ: Hỗ trợ việc mở rộng hoặc giảm quy mô của các dịch vụ mà không làm giảm hiệu suất của hệ thống. Điều này giúp hệ thống linh hoạt hơn trong việc đáp ứng nhu cầu thay đổi về tài nguyên.
- Cập nhật phiên bản của service: Cho phép cập nhật hoặc nâng cấp phiên bản của một dịch vụ để cải thiện hiệu suất hệ thống mà không làm gián đoạn hoạt động của các dịch vụ khác.

Kiến trúc Microservices có rất nhiều cải thiện về hiệu suất so với kiến trúc Monolith truyền thống. Theo bài báo “Designing microservice architectures for scalability and reliability in ecommerce” [27]

- Khả năng xử lý của một trang thương mại điện tử sử dụng Microservice đạt hiệu quả hơn 30% khi load script và 4% khi load phần tử hình ảnh.
- Khả năng chịu tải khi stress test với lần lượt 300, 500 và 1000 lần lặp kiểm tra dữ liệu với tối đa 25000 request đồng thời, tỷ lệ xuất hiện lỗi rất thấp (khoảng 3%)
- Các công nghệ API Gateway, Quản lý và đóng gói ứng dụng đã phát triển rất nhanh nhờ sự phổ biến của Microservices
- Từ đó, em quyết định chọn đề tài ứng dụng kiến trúc Microservices để xây dựng một sàn thương mại điện tử, tự động mở rộng hoặc thu nhỏ services đảm bảo hệ thống luôn luôn hoạt động ổn định.

2. MỤC TIÊU VÀ PHẠM VI NGHIÊN CỨU

Em tập trung vào mục tiêu ứng dụng kiến trúc Microservices và các công nghệ như Spring Boot, API Gateway và ReactJs để xây dựng một sàn thương mại điện tử có khả năng chịu tải cao, đồng thời có thể tự động mở rộng và thu nhỏ các service một cách hiệu quả. Đầu tiên, đề tài sẽ nghiên cứu sâu về kiến trúc Microservices, các khái niệm cơ bản, ưu điểm vượt trội như tính module hóa, khả năng mở rộng linh hoạt, khả năng phục hồi tốt cùng những thách thức đi kèm. Việc nắm vững kiến thức này sẽ tạo nền tảng vững chắc cho quá trình triển khai.

Về phía Front End, đề tài sử dụng ReactJs để hiện thực hóa giao diện và các chức năng giúp cho người dùng cuối như khách hàng, chủ cửa hàng và quản trị viên hệ thống có thể tương tác với hệ thống.

Về phần Back End, đề tài sử dụng Spring Boot, Mysql để xây dựng nên hệ thống RESTful API . Đồng thời để cải thiện tốc độ xử lý, em sử dụng các công nghệ hiện đại như Redis, Cron job và Background Job để tối ưu và tăng hiệu suất của hệ thống.

Front End và Back End giao tiếp với nhau thông qua API Gateway. API Gateway sẽ làm trung gian để trung chuyển các request từ Frontend đến đúng services.

3. Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN

Đề tài "Xây dựng nền tảng thương mại thiết bị điện tử tích hợp trí tuệ nhân tạo và chatbot sử dụng Spring Boot , ReactJS với hệ thống gợi ý sản phẩm cá nhân hóa" mang ý nghĩa khoa học trong việc nghiên cứu kỹ thuật phát triển phần mềm theo hướng microservices và mang ý nghĩa thực tiễn khi áp dụng những nghiên cứu đó thành một sản phẩm hoàn chỉnh.

Việc nghiên cứu và triển khai kiến trúc microservice cho phép phân tách các dịch vụ độc lập, tăng cường khả năng quản lý, bảo trì và mở rộng hệ thống, đồng thời giảm thiểu sự phụ thuộc lẫn nhau giữa các thành phần.

Với việc phát triển một sàn thương mại điện tử thân thiện với người dùng, nhưng lại có khả năng mở rộng và chịu tải tốt, đề tài sẽ cung cấp một ví dụ cụ thể về cách triển

MỞ ĐẦU

khai một hệ thống microservices trên môi trường cloud và ứng dụng để xây dựng một sản phẩm thương mại điện tử.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1. 1. Phân tích, đánh giá các công trình nghiên cứu liên quan

Qua quá trình tìm hiểu tài liệu, em nhận thấy chủ đề Microservices trong thương mại điện tử đã được nhiều tác giả quan tâm nghiên cứu. Cụ thể:

Đối với bài báo “Designing microservice architectures for scalability and reliability in e-commerce” [27], nhóm tác giả Asrowardi đã phân tích bối cảnh bùng nổ của thương mại điện tử (đặc biệt tại Indonesia) và chỉ ra rằng các hệ thống cũ đang gặp khó khăn trong việc mở rộng quy mô. Nghiên cứu này đã đề xuất Microservices như một giải pháp thay thế, giúp hệ thống triển khai nhanh hơn và tái sử dụng dịch vụ tốt hơn. Kết quả thực nghiệm của bài báo bước đầu chứng minh được tính linh hoạt của kiến trúc này so với các mô hình truyền thống.

Bên cạnh đó, nghiên cứu “Microservices: architecture, container, and challenges” [28] đã tiếp cận vấn đề từ góc độ xu hướng công nghệ toàn cầu. Nhóm tác giả liệt kê hàng loạt các ông lớn như Netflix, Amazon, WeChat (với hơn 3000 microservices) để minh chứng rằng kiến trúc Monolithic (nguyên khối) đang trở nên lỗi thời do lãng phí tài nguyên khi cần mở rộng (scale-up). Nghiên cứu khẳng định Microservices là lời giải cho bài toán xử lý hàng tỷ lượt tương tác mỗi ngày mà không gây lãng phí hạ tầng.

1. 2. Những vấn đề còn tồn tại

Mặc dù các công trình trên đã cung cấp nền tảng lý thuyết tốt, tuy nhiên, dưới góc độ của một người đang tìm cách áp dụng kiến trúc này vào thực tế, em nhận thấy còn tồn tại một số hạn chế (gap) sau đây:

- **Thiếu tính minh bạch trong thực nghiệm:** Các bài báo chưa mô tả rõ môi trường, cấu hình phần cứng (server specs) hay công cụ cụ thể khi test. Điều này khiến cho việc sinh viên/người nghiên cứu sau khó có thể tái lập lại kết quả để kiểm chứng.

- **Chưa đề cập sâu đến rào cản triển khai:** Việc chuyển đổi từ Monolithic sang Microservices tốn rất nhiều chi phí và nhân lực, nhưng các tài liệu trên chưa đưa ra

được các chiến lược chuyển đổi (migration strategy) cụ thể hay các bài học kinh nghiệm (case study) về thất bại/rủi ro.

- **Thiếu đánh giá kỹ thuật chuyên sâu:** Các phân tích về hiệu năng (performance) còn khá sơ sài, chưa đi sâu vào các vấn đề cốt lõi của hệ thống phân tán như: độ trễ mạng (latency) khi các service gọi nhau, cơ chế đảm bảo tính nhất quán dữ liệu (data consistency), hay độ phức tạp khi vận hành.

1. 3. Nội dung chính của tiểu luận

Xuất phát từ nhu cầu thực tế và những hạn chế của các nghiên cứu trước, tiểu luận này sẽ tập trung giải quyết bài toán "Xây dựng và đánh giá hiệu năng sản thương mại điện tử dựa trên kiến trúc Microservices".

Nội dung chính sẽ bao gồm:

1. Hệ thống hóa lý thuyết: Làm rõ các thành phần kỹ thuật mà các nghiên cứu trước còn nói chung chung, cụ thể là: API Gateway, Service Discovery, và cơ chế giao tiếp giữa các dịch vụ (Inter-service communication).

2. Lựa chọn công nghệ: Thay vì chỉ nêu lý thuyết, tôi sẽ tiến hành so sánh và lựa chọn bộ công nghệ (Tech Stack) cụ thể (ví dụ: Spring Boot,...) phù hợp cho việc triển khai sản thương mại điện tử.

3. Triển khai ứng dụng thực tế: Xây dựng một hệ thống demo hoàn chỉnh với các chức năng nghiệp vụ cốt lõi (Đặt hàng, Thanh toán, Kho hàng). Hệ thống này sẽ là môi trường để thực hiện các bài đo đạc, đánh giá hiệu năng, khả năng chịu tải và khả năng phục hồi lỗi, từ đó đưa ra cái nhìn thực tế và chi tiết hơn so với các nghiên cứu lý thuyết đơn thuần.

CHƯƠNG 2. PHƯƠNG PHÁP VÀ NỘI DUNG NGHIÊN CỨU

2.1. Cơ sở lý thuyết

2.1.1. Microservices

Trong quy trình phát triển phần mềm hiện đại, việc xác định kiến trúc hệ thống đóng vai trò then chốt, đặc biệt đối với các nền tảng thương mại điện tử (TMĐT). Theo báo cáo thị trường TMĐT gần đây, nhu cầu về tính ổn định và khả năng xử lý lượng truy cập lớn trong các đợt cao điểm là yêu cầu bắt buộc [26]. Một hệ thống sàn TMĐT không chỉ đòi hỏi tính sẵn sàng cao (High Availability) mà còn phải đảm bảo các phân hệ chức năng như xác thực, thanh toán, và gợi ý sản phẩm hoạt động trơn tru mà không ảnh hưởng lẫn nhau [27].

Để đáp ứng các yêu cầu phức tạp về khả năng mở rộng (Scalability) và độ tin cậy (Reliability), kiến trúc Microservices được đánh giá là giải pháp tối ưu cho việc xây dựng các hệ thống TMĐT quy mô lớn [27], [28]. Dựa trên cơ sở đó, em thực hiện quyết định áp dụng kiến trúc Microservices để phát triển hệ thống, cho phép quản lý các dịch vụ một cách biệt lập và hiệu quả.

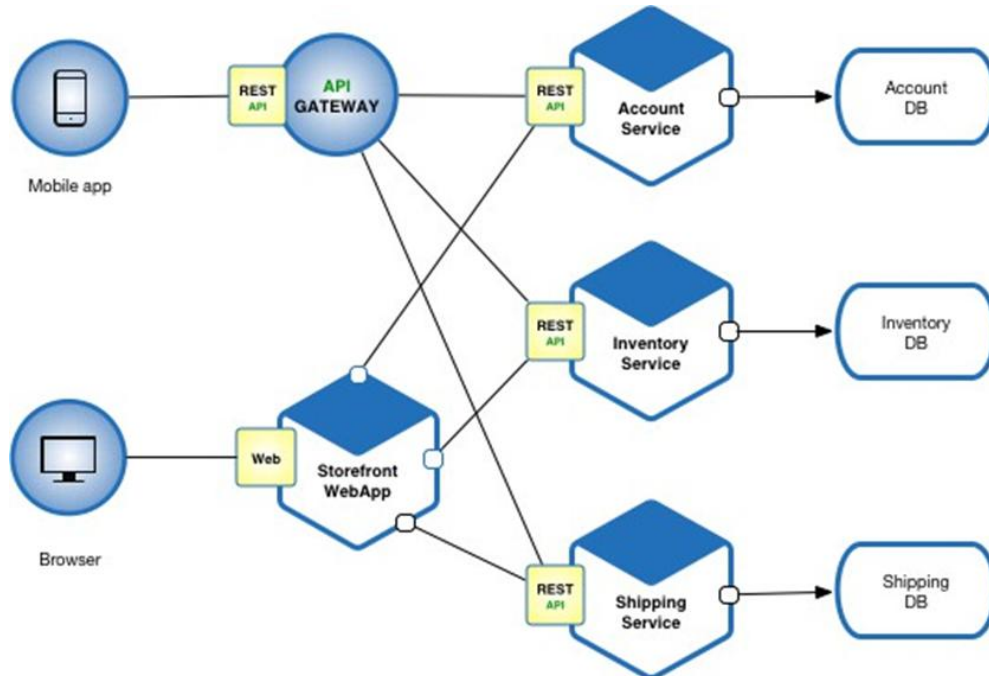
2.1.1.1. Giới thiệu về Microservices

Microservices (Vi mô dịch vụ) là một kỹ thuật phát triển phần mềm, được xem là bước tiến hóa từ kiến trúc hướng dịch vụ (SOA - Service Oriented Architecture) nhằm khắc phục các hạn chế về sự phức tạp và phụ thuộc lẫn nhau của các hệ thống cũ [28].

Về mặt bản chất, Microservices chia nhỏ một ứng dụng lớn thành tập hợp các dịch vụ nhỏ, độc lập (services). Mỗi dịch vụ chạy trên một tiến trình riêng biệt và giao tiếp với nhau thông qua các cơ chế gọn nhẹ, thường là HTTP/REST API [2]. Cách tiếp cận này đã được các tập đoàn công nghệ lớn tiên phong áp dụng từ giữa những năm 2000 để giải quyết bài toán về dữ liệu lớn và quy mô người dùng toàn cầu, dần thay thế cho kiến trúc nguyên khối (Monolithic) truyền thống [28].

Trong kiến trúc này, mỗi microservice tập trung giải quyết một nghiệp vụ cụ thể (ví dụ: Quản lý người dùng, Quản lý đơn hàng) và có thể được phát triển, triển khai, cũng như

mở rộng một cách độc lập. Điều này giúp cải thiện tính module hóa, giúp mã nguồn dễ hiểu hơn và thuận tiện cho công tác kiểm thử, bảo trì [27].



Hình 2.1: Minh họa về kiến trúc Microservices

(Nguồn: https://microservices.io/i/Microservice_Architecture.png)

2.1.1.2. Những đặc trưng của Microservices

Kiến trúc Microservices cung cấp một cách tiếp cận linh hoạt và hiệu quả để quản lý và phát triển các ứng dụng phức tạp, đặc biệt là đối với các nền tảng thương mại điện tử nơi tính mở rộng và linh hoạt là rất quan trọng. Dưới đây là một số lợi ích chính của kiến trúc Microservices đối với thương mại điện tử.

2.1.1.2.1. Tính mở rộng độc lập (Scalability)

Microservices cho phép từng dịch vụ mở rộng độc lập dựa trên nhu cầu. Điều này có nghĩa rằng trong những thời gian cao điểm như siêu sale giảm giá lớn của năm, chỉ các thành phần cần thiết như hệ thống đặt hàng mới cần mở rộng, tối ưu hóa phân bổ tài nguyên và giảm chi phí.

2.1.1.2.2. Tùy chọn công nghệ linh hoạt

Microservices kiến trúc này không ràng buộc hệ thống vào một công nghệ duy nhất. Ví dụ, dịch vụ gợi ý sản phẩm liên quan đến AI có thể dùng Python[16] và các thư viện machine learning, trong khi các dịch vụ xử lý giao dịch tài chính, đơn hàng có thể dùng Java (Spring Boot)[1] để đảm bảo độ ổn định và an toàn. Nhờ đó, hệ thống dễ dàng tích hợp công nghệ mới, nâng cấp từng phần mà không ảnh hưởng toàn bộ kiến trúc.

2.1.1.2.3. Khả năng chịu lỗi và cô lập sự cố (Fault Isolation)

Trong kiến trúc nguyên khối, một lỗi rò rỉ bộ nhớ (memory leak) ở một module có thể làm sập toàn bộ tiến trình ứng dụng. Với Microservices, sự cố được cô lập trong phạm vi của dịch vụ đó, giúp hệ thống tổng thể vẫn duy trì hoạt động (Resilience) [2].

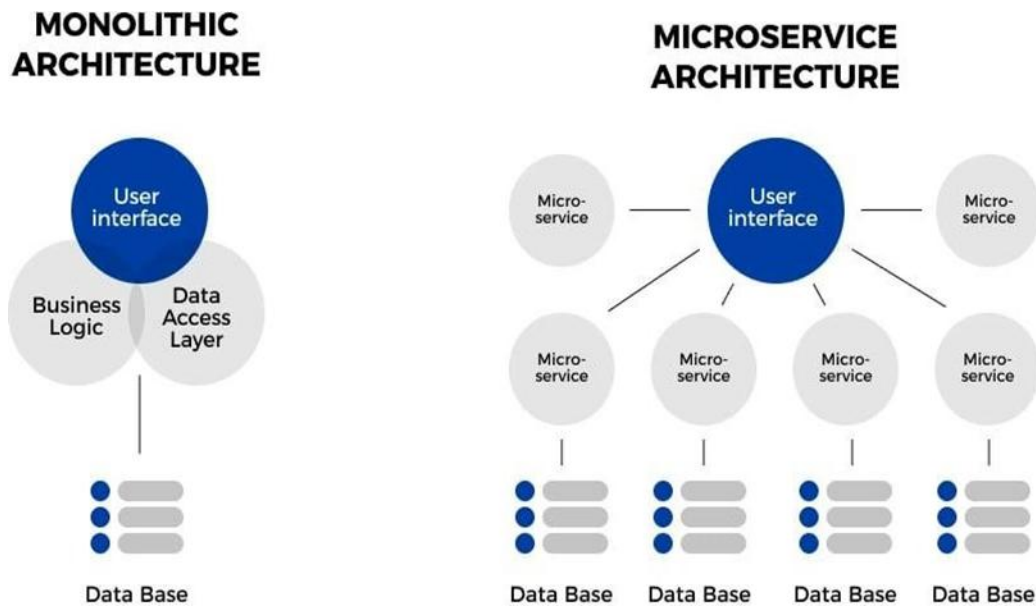
2.1.1.2.4. Tính phân rã và module hóa (Componentization via Services)

Đặc trưng quan trọng nhất là việc phân chia hệ thống thành các dịch vụ nhỏ (components). Thay vì sử dụng các thư viện liên kết trong cùng một tiến trình, Microservices sử dụng các dịch vụ giao tiếp qua mạng. Điều này cho phép thay thế hoặc nâng cấp một dịch vụ cụ thể mà không ảnh hưởng đến toàn bộ hệ thống.

2.1.1.2.5. Phi tập trung hóa trong quản trị dữ liệu (Decentralized Data Management)

Microservices ưu tiên mô hình "Database per Service" (Mỗi dịch vụ một cơ sở dữ liệu). Điều này đảm bảo tính lỏng lẻo (loose coupling) giữa các dịch vụ, tránh việc một thay đổi trong cấu trúc bảng dữ liệu làm lỗi các dịch vụ khác. Tuy nhiên, điều này cũng đặt ra thách thức về tính nhất quán dữ liệu (data consistency) trong hệ thống phân tán [18], [19].

2.1.1.3. So sánh kiến trúc Microservices và kiến trúc (Monolithic)



Hình 2.2: So sánh giữa Monolithic và Microservices

(Nguồn: <https://www.openlegacy.com/hs->

[fs/hubfs/Picture1.webp?width=889&height=478&name=Picture1.webp](https://www.openlegacy.com/hs-fs/hubfs/Picture1.webp?width=889&height=478&name=Picture1.webp))

Chương này sẽ so sánh hai kiến trúc phần mềm phổ biến là **Microservices** và **Monolithic architecture**. Đây là hai mô hình phát triển ứng dụng có ảnh hưởng đáng kể đến cách thiết kế, triển khai, và quản lý các hệ thống phần mềm.

Bảng 2. 1 So sánh kiến trúc Monolithic và Microservices

	Monolithic Architecture	Microservices Architecture
--	-------------------------	----------------------------

Định nghĩa	Trong mô hình này, tất cả các thành phần của ứng dụng (bao gồm cả frontend, backend, và database layers) được phát triển và triển khai như một đơn thể duy nhất. Mô hình này thường đơn giản hơn để phát triển và triển khai	Trong mô hình Microservices, ứng dụng được chia thành nhiều dịch vụ nhỏ, mỗi dịch vụ hoạt động độc lập và thực hiện một chức năng cụ thể của hệ thống. Các Microservices này giao tiếp
	ban đầu, nhưng có thể gặp khó khăn khi quy mô tăng lên hoặc khi cần cập nhật và bảo trì.	với nhau thông qua API (Application Programming Interface) và thường được triển khai một cách độc lập.
Scalability và Maintenance	Việc mở rộng quy mô (scaling) và bảo trì ứng dụng monolithic có thể trở nên phức tạp khi độ lớn của phần mềm tăng lên. Mọi thay đổi, dù nhỏ, đều yêu cầu xây dựng lại và triển khai lại toàn bộ ứng dụng, điều này có thể dẫn đến downtime đáng kể và tăng rủi ro trong quản lý version.	Các dịch vụ trong mô hình Microservices có thể được mở rộng độc lập, cho phép quản lý quy mô hiệu quả hơn. Bảo trì và cập nhật từng Microservice riêng biệt cũng dễ dàng hơn, giúp giảm thiểu rủi ro và downtime. Tuy nhiên, quản lý nhiều dịch vụ đồng thời đòi hỏi các công cụ và kỹ năng quản lý phức tạp.

Độ phức tạp và Cost of Development	Phát triển ban đầu thường nhanh hơn và đơn giản hơn vì tất cả các thành phần của ứng dụng được tích hợp chặt chẽ. Tuy nhiên, chi phí và độ phức tạp có thể tăng lên đáng kể khi ứng dụng phát triển kích thước và tính năng.	Phát triển dựa trên Microservices đòi hỏi kỹ thuật cao hơn và ban đầu có thể tốn kém hơn do cần thiết lập một hạ tầng phức tạp để quản lý các dịch vụ độc lập. Tuy nhiên, trong dài hạn, mô hình này thường mang lại lợi ích về khả năng mở rộng và dễ dàng bảo trì hơn.
Thích hợp với từng loại ứng dụng	Thường phù hợp với các dự án nhỏ đến trung bình, nơi mà độ phức tạp của sản phẩm không cao và không yêu cầu mở rộng quy mô lớn.	Được ưu tiên cho các dự án lớn hơn, có yêu cầu cao về quy mô và tính linh hoạt, đặc biệt là trong môi trường phát triển năng động và phức tạp.

2.1.1.4. Những khó khăn khi triển khai Microservices

2.1.1.4.1. Sự phức tạp trong quản lý và điều hành:

Mặc dù mang lại nhiều lợi ích, việc áp dụng Microservices cũng đặt ra các bài toán kỹ thuật cần giải quyết:

- **Độ phức tạp vận hành:** Quản lý hàng chục dịch vụ nhỏ khó khăn hơn nhiều so với một khối lớn, đòi hỏi phải áp dụng DevOps và tự động hóa.
- **Giao tiếp qua mạng:** Việc gọi hàm nội bộ (in-process calls) được thay thế bằng gọi qua mạng (RPC/REST), làm tăng độ trễ (latency) và nguy cơ lỗi mạng [29].

- **Tính nhất quán dữ liệu:** Việc đảm bảo dữ liệu đồng bộ giữa các database riêng biệt đòi hỏi các mẫu thiết kế phức tạp như Saga Pattern hay Event Sourcing.

2.1.1.5. Kết luận về Microservices

Dựa trên cơ sở lý thuyết đã phân tích, Microservices là kiến trúc phù hợp cho các hệ thống phần mềm có yêu cầu cao về tính sẵn sàng và khả năng mở rộng.

Đối với đề tài nghiên cứu xây dựng hệ thống quản lý bán hàng (cụ thể là sàn thương mại điện tử), em quyết định áp dụng kiến trúc Microservices vì các lý do sau:

- Hệ thống cần tích hợp nhiều phân hệ khác nhau như: Thanh toán (VNPay) [8], Vận chuyển (GHN) [9], và Gọi ý sản phẩm (AI) [10]. Microservices cho phép phát triển các module này bằng các công nghệ chuyên biệt (Spring Boot cho nghiệp vụ chính, Python cho AI).
- Đảm bảo khả năng chịu tải khi số lượng request tăng cao, tránh nghẽn cổ chai tại một điểm duy nhất.
- Tận dụng các công nghệ hiện đại như Spring Cloud [2] để quản lý hệ thống phân tán hiệu quả.

2.1.2. Các thành phần giao tiếp và quản lý dịch vụ

Trong hệ thống phân tán Microservices, việc quản lý giao tiếp giữa các dịch vụ và điều phối yêu cầu từ người dùng là yếu tố then chốt. Đề tài tập trung nghiên cứu ba thành phần cốt lõi: API Gateway, REST API và Spring Cloud Gateway. API Gateway đóng vai trò cổng kết nối duy nhất, REST API chuẩn hóa phương thức giao tiếp, và Spring Cloud Gateway là giải pháp thực thi cụ thể được em lựa chọn [2].

2.1.2.1. API Gateway

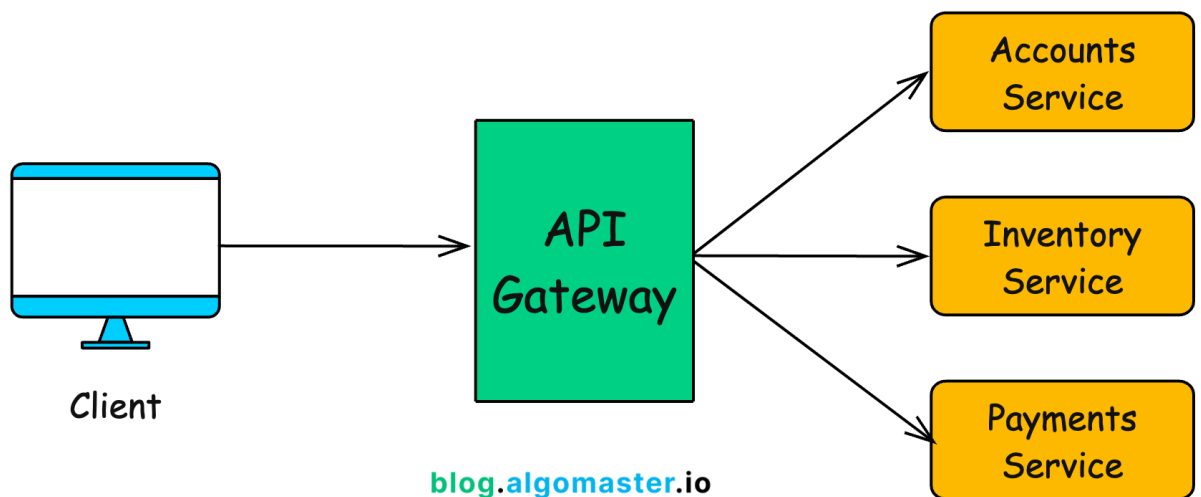
2.1.2.1.1. Khái niệm

Tổng quan API Gateway hoạt động như một lớp trung gian (middleware) hay một "reverse proxy" đứng giữa client (ứng dụng Web/Mobile) và hệ thống các microservices

phía backend. Thay vì client phải gửi yêu cầu trực tiếp đến từng dịch vụ nhỏ lẻ, tất cả các request đều được gửi đến API Gateway. Tại đây, Gateway sẽ chịu trách nhiệm định tuyến (routing) request đến đúng dịch vụ đích và trả kết quả về cho client [2], [28].

Cơ chế hoạt động điển hình trong hệ thống:

- Client gửi request lấy thông tin người dùng.
- API Gateway tiếp nhận, xác thực và chuyển tiếp đến **User Service**.
- Gateway nhận phản hồi từ User Service và trả về cho Client.



Hình 2.3: Mô tả về hệ thống API Gateway (Nguồn:

[https://substackcdn.com/image/fetch/\\$s ! CtQ!f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F6e41325f-2b6b-4f2a-8ca6-d622413ad64c_1692x720.png](https://substackcdn.com/image/fetch/$s ! CtQ!f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F6e41325f-2b6b-4f2a-8ca6-d622413ad64c_1692x720.png))

2.1.2.1.2. Các chức năng trọng yếu

- Định tuyến (Routing): Phân luồng request dựa trên URL hoặc Header. Ví dụ: request `/api/products` được chuyển đến *Product Service*, `/api/orders` chuyển đến *Order Service*.

- **Bảo mật (Security):** Đóng vai trò chốt chặn bảo mật đầu tiên. Gateway tích hợp xác thực (Authentication) và ủy quyền (Authorization) thông qua các tiêu chuẩn như OAuth2 hoặc JWT, đảm bảo chỉ các request hợp lệ mới được đi vào hệ thống [3].
- **Quản lý lưu lượng (Traffic Management):** Bảo vệ hệ thống khỏi quá tải thông qua cơ chế giới hạn tốc độ (Rate Limiting) và ngắt mạch (Circuit Breaker), giúp duy trì độ ổn định cho các dịch vụ backend [2].
- **Giám sát (Monitoring & Logging):** Ghi nhận nhật ký truy cập tập trung, giúp việc theo dõi sức khỏe hệ thống và truy vết lỗi trở nên dễ dàng hơn.

2.1.2.1.3. Thách thức khi triển khai

Bên cạnh lợi ích, việc áp dụng API Gateway cũng tồn tại một số hạn chế cần lưu ý:

- **Độ trễ (Latency):** Việc thêm một lớp trung gian chắc chắn sẽ làm tăng thời gian phản hồi của request do phải xử lý các tác vụ phụ trợ (xác thực, logging).
- **Điểm nghẽn cổ chai (Single Point of Failure):** Nếu API Gateway gặp sự cố, toàn bộ hệ thống sẽ bị ngắt kết nối với bên ngoài. Do đó, yêu cầu thiết kế phải đảm bảo tính sẵn sàng cao (High Availability).
- **Chi phí vận hành:** Đòi hỏi tài nguyên phần cứng và nhân lực để cấu hình, bảo trì hệ thống Gateway riêng biệt.

2.1.2.2. REST API

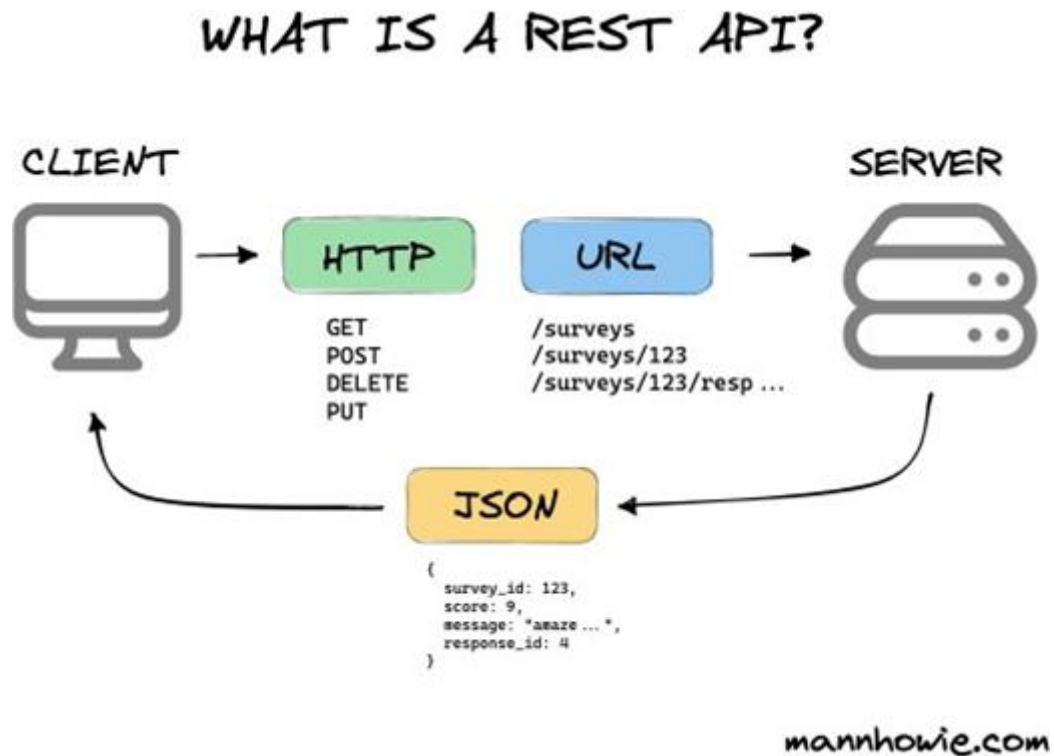
2.1.2.2.1. Khái niệm và nguyên lý

REST (Representational State Transfer) là kiểu kiến trúc phần mềm phổ biến nhất hiện nay cho các dịch vụ Web, được giới thiệu bởi Roy Fielding năm 2000. RESTful API là các API tuân thủ theo nguyên lý REST, sử dụng giao thức HTTP làm phương thức giao tiếp chính để thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên dữ liệu [29].

Các nguyên tắc thiết kế cơ bản của REST bao gồm:

- **Stateless (Phi trạng thái):** Server không lưu giữ trạng thái của Client giữa các request. Mọi thông tin cần thiết phải được gửi kèm trong mỗi request.

- **Uniform Interface (Giao diện thống nhất):** Các tài nguyên được định danh rõ ràng qua URI (ví dụ: /api/products) và thao tác qua các phương thức HTTP chuẩn.



Hình 2.4: Mô tả cơ bản về cách RESTful API hoạt động

(Nguồn:

<https://images.ctfassets.net/vwq10xzb6iz/5sBH4Agl6l4xM7exeLsTo7/9e84dce01735f155911e611c42c9793f/rest-api.png>)

2.1.2.2.2. Các phương thức HTTP và Mã trạng thái

Hệ thống sử dụng các phương thức chuẩn:

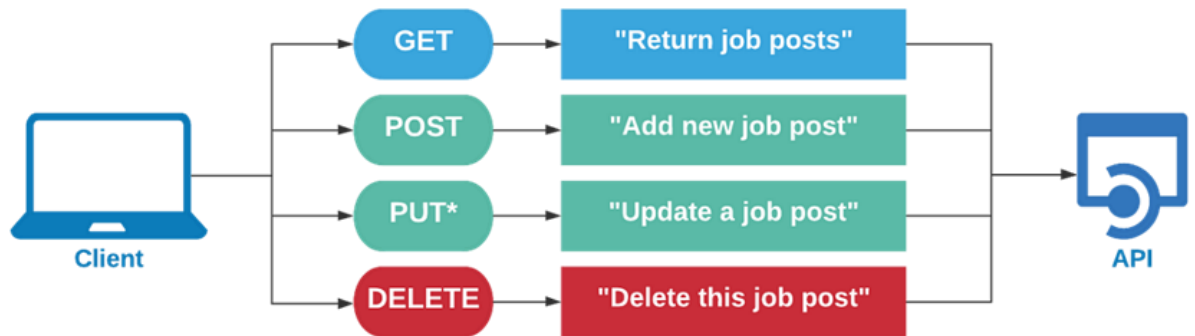
- **GET:** Truy xuất thông tin từ server mà không làm thay đổi trạng thái của tài nguyên. Ví dụ: Yêu cầu GET đến /api/products trả về danh sách tất cả các sản phẩm.

- **POST:** Gửi dữ liệu đến server để tạo mới một tài nguyên. Ví dụ: Yêu cầu POST đến `/api/products` với dữ liệu sản phẩm mới trong phần thân yêu cầu sẽ tạo một sản phẩm mới.
- **PUT:** Cập nhật một tài nguyên hiện có trên server với dữ liệu mới. Ví dụ: Yêu cầu PUT đến `/api/products/1` với dữ liệu cập nhật trong phần thân yêu cầu sẽ cập nhật thông tin của sản phẩm có ID là 1.
- **DELETE:** Xóa một tài nguyên trên server. Ví dụ: Yêu cầu DELETE đến `/api/products/1` sẽ xóa sản phẩm có ID là 1.
- **PATCH:** Cập nhật từng phần của một tài nguyên hiện có. Ví dụ: Yêu cầu PATCH đến `/api/products/1` với dữ liệu cập nhật từng phần trong phần thân yêu cầu sẽ cập nhật một hoặc vài thuộc tính của sản phẩm có ID là 1.

Một số các Status Code (mã trạng thái) nhận được khi gửi một yêu cầu API:[29]

- 200 OK – Trả về thành công cho những phương thức GET, PUT, PATCH hoặc DELETE.
- 201 Created – Trả về khi một Resource vừa được tạo thành công.
- 204 No Content – Trả về khi Resource xóa thành công.
- 304 Not Modified – Client có thể sử dụng dữ liệu cache.
- 400 Bad Request – Request không hợp lệ - 401 Unauthorized – Request cần có auth.
- 403 Forbidden – bị từ chối.
- 404 Not Found – Không tìm thấy resource từ URI
- 405 Method Not Allowed – Phương thức không cho phép với user hiện tại.
- 410 Gone – Resource không còn tồn tại, Version cũ đã không còn hỗ trợ.
- 415 Unsupported Media Type – Không hỗ trợ kiểu Resource này.
- 422 Unprocessable Entity – Dữ liệu không được xác thực.

- 429 Too Many Requests – Request bị từ chối do bị giới hạn.



Hình 2.5: Mô tả cơ bản về cách RESTful API hoạt động (Nguồn:

https://miro.medium.com/v2/resize:fit:1400/1*UXTnlhEg4aua_6eQv3HHSg.png)

Trong tiểu luận này, Restful API được sử dụng để Frontend gửi yêu cầu đến Backend, Backend xử lý yêu cầu và trả về phản hồi cho Frontend.

2.1.2.2.3. So sánh RESTful và SOAP

SOAP (Simple Object Access Protocol) là một giao thức dựa trên XML được sử dụng để trao đổi thông điệp giữa các ứng dụng qua mạng. Dưới đây là bảng so sánh chi tiết giữa REST và SOAP để có thể thấy được sự khác biệt giữa 2 giao thức này

Bảng 2.2: So sánh giữa SOAP và REST API

Tiêu chí	REST	SOAP
Giao thức	HTTP, HTTPS	HTTP, HTTPS, SMTP, TCP
Định dạng dữ liệu	JSON, XML, YAML, v.v.	XML
Khả năng mở rộng	Dễ dàng mở rộng	Khó khăn hơn do tính chặt chẽ của giao thức

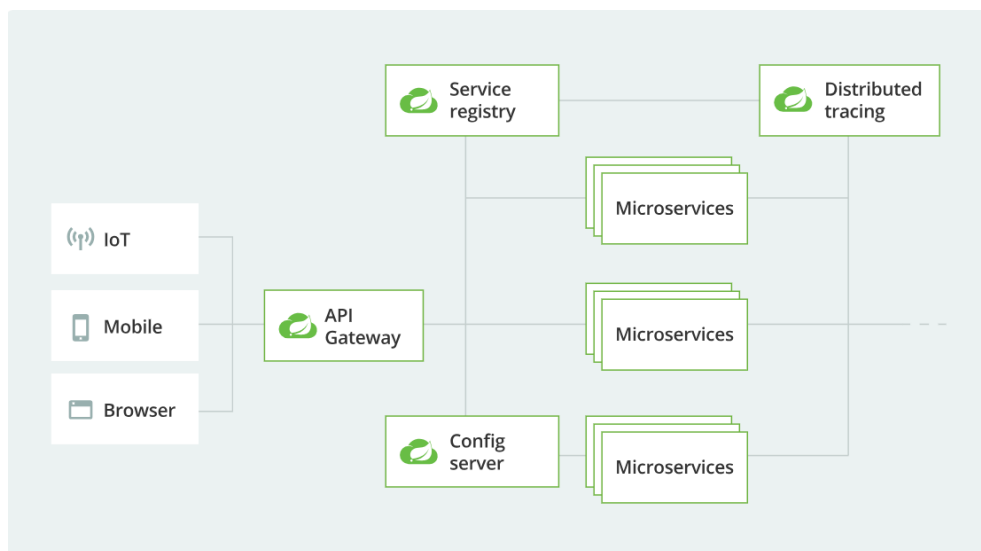
Caching	Tận dụng rất tốt caching	Bỏ qua caching
Bảo mật	Tích hợp bảo mật HTTP, OAuth, JWT	WS-Security, SSL, HTTPS
Khả năng xử lý trạng thái	Stateless	Có thể stateful hoặc stateless
Khả năng tương thích	Tương thích với nhiều ngôn ngữ lập trình	Hạn chế hơn do yêu cầu SOAP libraries
Khả năng giao tiếp	Đơn giản, dễ hiểu	Phức tạp, yêu cầu kiến thức chuyên sâu về XML và SOAP
Tiêu chuẩn	Không tuân theo tiêu chuẩn cụ thể	Tuân theo các tiêu chuẩn WSDL, XSD
Sử dụng phổ biến	Web APIs, Microservices, Mobile applications	Enterprise applications, Financial services

2.1.2.3. Spring Cloud Gateway

Spring Cloud Gateway là một dự án mã nguồn mở thuộc hệ sinh thái Spring Cloud, được xây dựng trên nền tảng Spring Framework 5, Project Reactor và Spring Boot 2. Đây là giải pháp API Gateway hiện đại, hỗ trợ lập trình phản ứng (non-blocking APIs), giúp hệ thống chịu tải tốt hơn so với các giải pháp cũ như Netflix Zuul [2]

Kiến trúc và thành phần chính:

- Route: Thành phần cơ bản nhất, bao gồm ID, URI đích, tập hợp các Predicates và Filters.
- Predicate (Điều kiện): Dùng để khớp request (ví dụ: khớp theo đường dẫn /product/**).
- Filter (Bộ lọc): Cho phép can thiệp vào request trước hoặc sau khi gửi đến service đích (ví dụ: thêm Header xác thực, ghi log).



Hình 2.6: Mô tả mô hình Spring Cloud (Nguồn: <https://spring.io/cloud>)

Trong phạm vi đề tài, Spring Cloud Gateway được sử dụng để định tuyến request từ Frontend đến các microservices (Product, Order, User), đồng thời tích hợp với Eureka để cân bằng tải động (Load Balancing).

2.2 Các công nghệ hỗ trợ phát triển hệ thống

2.2.1 Spring Boot

2.2.1.1 Spring Boot là gì ?

Spring Boot là một framework mã nguồn mở được xây dựng trên nền tảng Spring Framework, được thiết kế để đơn giản hóa việc phát triển các ứng dụng Java enterprise. Spring Boot cung cấp cấu hình tự động (auto-configuration), giảm thiểu việc cấu hình thủ

công và cho phép các nhà phát triển tập trung vào logic nghiệp vụ thay vì cấu hình infrastructure[1].

Spring Boot được phát triển bởi Pivotal Software (hiện tại là VMware) và được xây dựng dựa trên nguyên tắc "convention over configuration", giúp giảm thiểu boilerplate code và tăng tốc độ phát triển ứng dụng. Framework này tích hợp sẵn các web server như Tomcat, Jetty, cho phép chạy ứng dụng như một standalone JAR file mà không cần triển khai trên application server riêng biệt.



Hình 2.7: Logo của Spring Boot

(Nguồn: https://upload.wikimedia.org/wikipedia/commons/7/79/Spring_Boot.svg)

2.2.1.2. Lý do lựa chọn cho đề tài nghiên cứu

Trong bối cảnh xây dựng hệ thống phân tán, em quyết định lựa chọn Spring Boot làm nền tảng phát triển Backend cốt lõi vì các ưu điểm kỹ thuật sau:

- **Tương thích với kiến trúc Microservices:** Khả năng hoạt động độc lập của Spring Boot giúp mỗi dịch vụ (Service) trong hệ thống có thể được phát triển, đóng gói và triển khai riêng biệt, tạo điều kiện thuận lợi cho việc container hóa bằng Docker [28].
- **Hệ sinh thái toàn diện:** Spring Boot tích hợp liền mạch với các dự án con như *Spring Data JPA* (quản lý cơ sở dữ liệu), *Spring Security* (bảo mật) [3] và đặc biệt là *Spring Cloud* [2]. Sự kết hợp này cung cấp một bộ công cụ hoàn chỉnh để giải quyết các bài toán khó của hệ thống phân tán như quản lý cấu hình, cân bằng tải và giám sát sức khỏe dịch vụ (Health Check) thông qua Spring Actuator.

- **Hiệu suất phát triển:** Các gói thư viện "Starter Dependencies" giúp em giảm thiểu thời gian thiết lập dự án ban đầu, tập trung nguồn lực vào việc phát triển các logic nghiệp vụ (Business Logic) đặc thù của sàn thương mại điện tử [1].

2.2.1.3. Ứng dụng vào đồ án

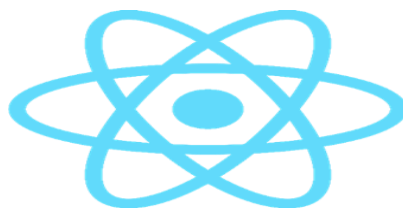
Trong phạm vi đồ án, Spring Boot đóng vai trò là nền tảng cốt lõi (Core Framework) để xây dựng toàn bộ hệ thống Backend theo kiến trúc Microservices. Cụ thể, Spring Boot được sử dụng để phát triển các dịch vụ nghiệp vụ độc lập như Service Sản phẩm, Service Đơn hàng, và Service Người dùng. Nhờ cơ chế Auto-configuration và hệ sinh thái phong phú, Spring Boot giúp rút ngắn thời gian thiết lập dự án, đơn giản hóa việc kết nối cơ sở dữ liệu và cung cấp chuẩn RESTful API để giao tiếp với phía Frontend.

2.2.2 React.js và TypeScript

2.2.2.1. Tổng quan về React.js và TypeScript

React.Js: là thư viện JavaScript mã nguồn mở do Meta (Facebook) khởi xướng, chuyên dùng để xây dựng giao diện người dùng (User Interface). React hoạt động dựa trên kiến trúc hướng thành phần (Component-based Architecture) và sử dụng cơ chế Virtual DOM để tối ưu hóa hiệu năng render, giúp ứng dụng phản hồi mượt mà ngay cả khi xử lý lượng dữ liệu lớn [5].

TypeScript: là một siêu tập hợp (superset) của JavaScript do Microsoft phát triển, bổ sung tính năng định kiểu tĩnh (static typing). Mã nguồn TypeScript sẽ được biên dịch về JavaScript thuần để chạy trên trình duyệt, giúp phát hiện lỗi cú pháp và logic ngay trong giai đoạn viết code [6].



Hình 2.8: Logo của React Framework

(Nguồn: <https://upload.wikimedia.org/wikipedia/commons/a/a7/React-icon.svg>)

2.2.2.2. Ứng dụng trong xây dựng giao diện người dùng

Đối với đề tài này, việc kết hợp React.js và TypeScript mang lại giải pháp toàn diện cho phía Frontend:

- **Kiến trúc Component:** Phù hợp với đặc thù của website thương mại điện tử vốn chứa nhiều thành phần lặp lại (thẻ sản phẩm, giỏ hàng, thanh điều hướng). Việc tái sử dụng component giúp code gọn gàng và dễ bảo trì [5].
- **An toàn dữ liệu (Type Safety):** Khi giao tiếp với nhiều Microservices thông qua REST API, dữ liệu trả về rất phức tạp. TypeScript giúp định nghĩa rõ ràng các Interface (ví dụ: cấu trúc User, Product), giảm thiểu rủi ro lỗi runtime do sai lệch kiểu dữ liệu [6].
- **Tối ưu trải nghiệm:** React hỗ trợ xây dựng ứng dụng đơn trang (SPA), giúp người dùng chuyển đổi giữa các trang danh mục, chi tiết sản phẩm mà không cần tải lại toàn bộ trang web, nâng cao trải nghiệm mua sắm.

2.2.2.3. Ứng dụng vào đồ án

ReactJS được lựa chọn để xây dựng toàn bộ giao diện người dùng (Frontend) cho cả trang mua sắm của khách hàng (Client) và trang quản trị hệ thống (Admin Dashboard).

TypeScript được tích hợp vào dự án ReactJS nhằm tăng cường tính chặt chẽ và an toàn cho mã nguồn Frontend. Trong đồ án này, TypeScript được sử dụng để định nghĩa rõ ràng các kiểu dữ liệu (Interfaces, Types) cho các đối tượng như Sản phẩm (Product), Đơn hàng (Order) hay Dữ liệu trả về từ API.

2.2.3 Hệ quản trị cơ sở dữ liệu MySQL

2.2.3.1. Tổng quan

MySQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở phổ biến nhất thế giới, được bảo trợ bởi Oracle. Hệ thống này sử dụng ngôn ngữ truy vấn có cấu trúc (SQL) và nổi tiếng với độ ổn định cao, tuân thủ chặt chẽ các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability) [18].



Hình 2.9: Logo của MySQL

(Nguồn: <https://www.mysql.com/common/logos/logo-mysql-170x115.png>)

2.2.3.2. Vai trò trong kiến trúc hệ thống

Trong khuôn khổ nghiên cứu, MySQL được chọn làm nơi lưu trữ dữ liệu chính tắc (Source of Truth) cho các dịch vụ quan trọng:

- Mô hình Database-per-service: Để đảm bảo tính độc lập của Microservices [28], em thực hiện thiết kế mỗi dịch vụ (Product Service, Order Service, User Service) sẽ sở hữu một cơ sở dữ liệu MySQL riêng biệt.
- Đảm bảo tính nhất quán: Đối với các nghiệp vụ tài chính như đặt hàng, thanh toán và quản lý kho, tính toàn vẹn dữ liệu là yêu cầu bắt buộc. Cơ chế Transaction của MySQL đảm bảo các giao dịch này được thực hiện chính xác tuyệt đối, tránh tình trạng sai lệch dữ liệu [18].
- Tương thích: MySQL hoạt động hiệu quả với *Spring Data JPA*, giúp đơn giản hóa các thao tác CRUD thông qua cơ chế ánh xạ đối tượng - quan hệ (ORM).

2.2.3.3. Ứng dụng vào đồ án

MySQL đóng vai trò là Hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) chính của hệ thống. Trong đồ án, mỗi Microservice quan trọng (như Product Service, Order Service) sẽ sở hữu một cơ sở dữ liệu MySQL riêng biệt để đảm bảo tính độc lập dữ liệu (Database per Service). MySQL chịu trách nhiệm lưu trữ bền vững các dữ liệu có cấu trúc phức tạp và yêu cầu độ chính xác cao về giao dịch (ACID) như thông tin đơn hàng, danh mục sản phẩm và hồ sơ khách hàng.

2.2.4. Redis

2.2.4.1. Tổng quan

Redis (Remote Dictionary Server) là một kho lưu trữ cấu trúc dữ liệu trong bộ nhớ (In-memory data structure store), được sử dụng làm cơ sở dữ liệu, bộ nhớ đệm (cache) và trình môi giới tin nhắn (message broker). Ưu điểm vượt trội của Redis là tốc độ truy xuất dữ liệu cực nhanh (độ trễ dưới 1ms) [19].



Hình 2.10: Logo của Redis

(Nguồn: <https://logo.svgcdn.com/logos/redis.png>)

2.2.4.2. Giải pháp tối ưu hiệu năng cho hệ thống

Đề tài sử dụng Redis để giải quyết các bài toán về hiệu năng mà MySQL khó đáp ứng tốt:

- Lưu trữ giỏ hàng: Dữ liệu giỏ hàng thay đổi liên tục nhưng không yêu cầu lưu trữ vĩnh viễn lâu dài. Redis với cơ chế TTL (Time-To-Live) là lựa chọn lý tưởng để lưu trạng thái giỏ hàng tạm thời, giúp giảm tải cho database chính.
- Caching: Lưu trữ đệm các thông tin ít thay đổi nhưng được truy vấn nhiều (như danh mục sản phẩm, cấu hình hệ thống), giúp tăng tốc độ phản hồi API.
- Quản lý phiên (Session): Lưu trữ token xác thực trong danh sách đen (blacklist) khi người dùng đăng xuất, hỗ trợ cơ chế bảo mật của hệ thống [19].

2.2.4.3. Ứng dụng vào đồ án

Redis được triển khai như một giải pháp lưu trữ bộ nhớ đệm (Caching Layer) nhằm tối ưu hóa hiệu năng hệ thống. Cụ thể, Redis được dùng để lưu trữ các dữ liệu được truy vấn thường xuyên nhưng ít thay đổi (như danh sách danh mục, chi tiết sản phẩm bán chạy) và quản lý phiên đăng nhập (Session) của người dùng. Việc này giúp giảm tải đáng kể số lượng truy vấn trực tiếp xuống MySQL và tăng tốc độ phản hồi của API.

2.2.5. Eureka Service Discovery

2.2.5.1 Khái niệm và vai trò

Eureka là một giải pháp đăng ký và khám phá dịch vụ (Service Discovery) thuộc bộ công cụ Spring Cloud Netflix. Trong môi trường Microservices năng động, nơi các instance của dịch vụ có thể khởi tạo hoặc tắt đi liên tục và địa chỉ IP thay đổi thường xuyên, việc cấu hình tĩnh (hard-code) địa chỉ IP là bất khả thi [2].

2.2.5.2. Cơ chế hoạt động trong đề tài

Hệ thống sử dụng Eureka Server làm trung tâm đăng ký dịch vụ:

- Khi các Microservice (Product, Order...) khởi động, chúng tự động đăng ký thông tin định danh và địa chỉ mạng với Eureka Server.
- API Gateway hoặc các dịch vụ khác khi cần giao tiếp sẽ truy vấn Eureka để lấy danh sách địa chỉ khả dụng.
- Eureka tích hợp cơ chế nhịp tim (Heartbeat) để liên tục kiểm tra sức khỏe của các dịch vụ, tự động loại bỏ các node bị lỗi khỏi danh sách định tuyến, đảm bảo tính sẵn sàng cao (High Availability) cho hệ thống [2].

2.2.5.3. Ứng dụng vào đề án

Trong kiến trúc phân tán của đề án, Netflix Eureka Server đóng vai trò là "cuốn danh bạ" trung tâm (Service Registry). Tất cả các microservices (Order, Product, User...) khi khởi động sẽ tự động đăng ký địa chỉ IP và công hoạt động của mình lên Eureka.

2.2.6. Bảo mật với JWT (JSON Web Token)

2.2.6.1. Tổng quan

JSON Web Token (JWT) là một chuẩn mở (RFC 7519) được sử dụng để tạo ra và xác thực các token an toàn giữa hai bên bằng cách sử dụng JSON. JWT được sử dụng rộng rãi để xác thực và phân quyền trong các ứng dụng web và API.

JWT bao gồm ba phần chính được ngăn cách bởi dấu chấm (`. `): Header, Payload, và Signature. Header chứa loại token và thuật toán mã hóa được sử dụng. Payload chứa các claims (thông tin về người dùng hoặc các dữ liệu mở rộng khác). Signature được tạo

ra từ Header, Payload và một chìa khóa bí mật, đảm bảo tính toàn vẹn của token và xác thực nguồn gốc của nó [23].

2.2.6.2. Ứng dụng trong xác thực phi trạng thái (Stateless Auth)

Với kiến trúc Microservices, việc sử dụng Session truyền thống (lưu trạng thái trên server) gặp nhiều khó khăn khi mở rộng quy mô. JWT là giải pháp thay thế hoàn hảo được em lựa chọn:

- Cơ chế: Khi người dùng đăng nhập thành công, hệ thống cấp phát một chuỗi JWT chứa thông tin định danh và quyền hạn.
- Hiệu quả: Các request tiếp theo chỉ cần gửi kèm token này. Các Microservice có thể tự xác thực token độc lập mà không cần truy vấn lại cơ sở dữ liệu người dùng, giúp giảm độ trễ và tăng khả năng chịu tải của hệ thống [3], [23].

2.2.6.3. Ứng dụng vào đồ án

JWT được sử dụng làm cơ chế xác thực (Authentication) và phân quyền (Authorization) chính cho toàn bộ hệ thống. Vì kiến trúc Microservices có tính chất phi trạng thái (Stateless), JWT cho phép truyền tải thông tin định danh người dùng và quyền hạn (Role) một cách an toàn giữa Client và Server thông qua HTTP Header. Điều này giúp hệ thống xác minh người dùng ở mọi microservice mà không cần phải truy vấn lại cơ sở dữ liệu người dùng liên tục.

2.2.7. Tích hợp OAuth2

2.2.7.1. Tổng quan

OAuth2 là một framework authorization được sử dụng để cho phép các ứng dụng truy cập tài nguyên của người dùng từ các nhà cung cấp dịch vụ (như Google, Facebook) mà không cần chia sẻ mật khẩu. OAuth2 cung cấp các flow khác nhau để phù hợp với các loại ứng dụng khác nhau.

OAuth2 được phát triển như một chuẩn mở (RFC 6749) để giải quyết vấn đề authorization trong các ứng dụng web và mobile. OAuth2 cho phép người dùng cấp quyền

truy cập tài nguyên của họ cho các ứng dụng bên thứ ba mà không cần chia sẻ credentials[24].

2.2.7.2. Tối ưu trải nghiệm đăng nhập

Đề tài tích hợp OAuth2 nhằm giải quyết bài toán "ngại đăng ký" của người dùng thương mại điện tử:

- Cho phép đăng nhập nhanh thông qua tài khoản Google/Facebook.
- Tăng tỷ lệ chuyển đổi người dùng mới và giảm thiểu rủi ro bảo mật liên quan đến việc lưu trữ mật khẩu người dùng [24]. Hệ thống chỉ lưu trữ thông tin định danh cơ bản và cấp phát JWT sau khi xác thực thành công qua OAuth2 provider.

2.2.7.3. Ứng dụng vào đồ án

OAuth2 được tích hợp để cung cấp tính năng "Đăng nhập bằng Google" hoặc "Đăng nhập bằng Facebook". Trong đồ án, công nghệ này giúp đơn giản hóa quy trình đăng ký/đăng nhập, mang lại sự tiện lợi cho người dùng.

2.2.8. Tích hợp Trí tuệ nhân tạo - Gemini API

2.2.8.1. Giới thiệu

Gemini API là một API trí tuệ nhân tạo được phát triển bởi Google, cung cấp khả năng xử lý ngôn ngữ tự nhiên (NLP), hiểu ngữ cảnh, và tạo ra phản hồi thông minh. Gemini API hỗ trợ nhiều loại tác vụ AI như chat, text generation, và content understanding.

Gemini API sử dụng các mô hình ngôn ngữ lớn (LLM) được train trên lượng dữ liệu khổng lồ, cho phép hiểu và xử lý ngôn ngữ tự nhiên một cách chính xác và tự nhiên. API có thể duy trì ngữ cảnh trong cuộc hội thoại, hiểu các tham chiếu và câu hỏi tiếp theo, và tạo ra phản hồi phù hợp với ngữ cảnh[4].



Hình 2.11: Logo của Gemini

(Nguồn:

https://upload.wikimedia.org/wikipedia/commons/8/8a/Google_Gemini_logo.svg)

2.2.8.2. Ứng dụng Chatbot hỗ trợ khách hàng

Thay vì xây dựng các mô hình AI phức tạp từ đầu (tốn kém tài nguyên tính toán), em tận dụng Gemini API để xây dựng tính năng Chatbot thông minh:

- Tư vấn tự động: Chatbot có thể hiểu ý định tìm kiếm của khách hàng (ví dụ: "Tìm điện thoại dưới 5 triệu chụp ảnh đẹp") và đưa ra gợi ý phù hợp.
- Hỗ trợ 24/7: Giải đáp thắc mắc về chính sách giao hàng, đổi trả dựa trên ngữ cảnh được cung cấp, giúp giảm tải cho bộ phận chăm sóc khách hàng [4].

2.2.8.3. Ứng dụng vào đồ án

Gemini API (của Google) được tích hợp để mang lại các tính năng thông minh cho sàn thương mại điện tử. Cụ thể, API này được sử dụng để xây dựng Chatbot hỗ trợ khách hàng tự động hoặc hỗ trợ người bán tạo nội dung mô tả sản phẩm (Product Description) một cách sáng tạo và nhanh chóng.

2.2.9. Python FastAPI và Scikit-learn

2.2.9.1. Tổng quan

- FastAPI: Framework web hiệu năng cao dành cho Python, được thiết kế để xây dựng API nhanh chóng với khả năng xử lý bất đồng bộ, hỗ trợ tự động tạo tài liệu API (Swagger UI) [16].
- Scikit-learn: Thư viện học máy (Machine Learning) mã nguồn mở mạnh mẽ, cung cấp các công cụ phân tích dữ liệu và thuật toán thống kê [17].

2.2.9.2. Ứng dụng trong Hệ thống Gợi ý (Recommender System)

Mặc dù hệ thống chính viết bằng Java, em quyết định tách phân hệ Gợi ý sản phẩm thành một Microservice riêng biệt sử dụng Python để tận dụng thế mạnh về xử lý dữ liệu:

- Service này sử dụng thư viện Scikit-learn để triển khai thuật toán Lọc cộng tác (Collaborative Filtering) [10], [13], phân tích lịch sử đánh giá của người dùng để dự đoán sở thích.
- FastAPI đóng vai trò lớp giao tiếp, cung cấp các API dự đoán với độ trễ thấp để hệ thống chính (Spring Boot) có thể gọi và hiển thị danh sách sản phẩm gợi ý cho người dùng [16].

2.2.9.3. Ứng dụng vào đồ án

Bộ đôi công nghệ này được sử dụng để xây dựng một Microservice riêng biệt chuyên trách xử lý các tác vụ Trí tuệ nhân tạo (AI Service). Python FastAPI cung cấp môi trường API hiệu năng cao, trong khi thư viện Scikit-learn được dùng để huấn luyện và vận hành mô hình Gợi ý sản phẩm (Recommendation System). Hệ thống này sẽ phân tích lịch sử hành vi của người dùng để đưa ra các đề xuất sản phẩm phù hợp, giúp cá nhân hóa trải nghiệm mua sắm.

2.2.10. Cổng thanh toán VNPay

2.2.10.1. Giới thiệu

VNPay là đơn vị cung cấp giải pháp thanh toán điện tử hàng đầu tại Việt Nam, kết nối với mạng lưới rộng lớn các ngân hàng và ví điện tử trong nước [8].

2.2.10.2. Ứng dụng thực tiễn

Việc tích hợp VNPay vào đề tài giúp mô phỏng sát thực tế quy trình thanh toán của một sàn TMĐT tại Việt Nam:

- Cung cấp phương thức thanh toán an toàn, quen thuộc cho người dùng nội địa (QR Pay, thẻ ATM nội địa).
- Đảm bảo tính bảo mật giao dịch thông qua cơ chế mã hóa và xác thực chữ ký số (Checksum) theo tài liệu tích hợp của VNPay [8]

2.2.10.3. Ứng dụng vào đồ án

VNPay được tích hợp vào quy trình thanh toán (Checkout) của hệ thống như một cổng thanh toán trực tuyến (Payment Gateway). Trong đồ án, VNPay cho phép mô phỏng

và xử lý các giao dịch thanh toán thực tế qua ngân hàng hoặc ví điện tử. Hệ thống sẽ nhận tín hiệu phản hồi từ VNPay (IPN) để tự động cập nhật trạng thái đơn hàng (từ "Chờ thanh toán" sang "Đã thanh toán") một cách chính xác và bảo mật.

2.2.11. Dịch vụ vận chuyển GHN (Giao Hàng Nhanh)

2.2.11.1. Giới thiệu

GHN (Giao Hàng Nhanh) là một dịch vụ vận chuyển hàng hóa tại Việt Nam, cung cấp API để tích hợp tính toán phí vận chuyển và tạo đơn hàng vận chuyển. GHN là một trong những đơn vị vận chuyển lớn nhất tại Việt Nam, phục vụ trên toàn quốc.

GHN API cho phép các nền tảng thương mại điện tử tích hợp tính toán phí vận chuyển, tạo đơn hàng vận chuyển, và theo dõi trạng thái vận chuyển một cách tự động[9].

2.2.11.2. Ứng dụng trong quản lý Logistic

Tiểu luận nghiên cứu tích hợp GHN API để tự động hóa quy trình giao nhận:

- Tính phí tự động: Hệ thống tự động tính toán phí ship dựa trên API của GHN ngay khi khách hàng nhập địa chỉ, đảm bảo tính minh bạch về chi phí.
- Đồng bộ trạng thái: Trạng thái đơn hàng (Đang lấy hàng, Đang giao, Đã giao) được cập nhật tự động về hệ thống thông qua cơ chế Webhook hoặc API Tracking, giúp người dùng dễ dàng theo dõi hành trình đơn hàng [9].

2.2.11.3. Ứng dụng vào đồ án

API của Giao Hàng Nhanh (GHN) được tích hợp để giải quyết bài toán Logistics. Trong đồ án này, hệ thống kết nối với GHN để tự động tính toán phí vận chuyển theo thời gian thực dựa trên địa chỉ nhận hàng và trọng lượng sản phẩm.

2.2.12. Material-UI (MUI)

2.2.12.1. Tổng quan

Material-UI (MUI) là một thư viện React component được xây dựng dựa trên Material Design guidelines của Google. MUI cung cấp một bộ component UI đẹp,

responsive, và có thể tùy biến, giúp developers xây dựng giao diện người dùng một cách nhanh chóng và chuyên nghiệp.

MUI được phát triển bởi cộng đồng và là một trong những thư viện UI component phổ biến nhất cho React, với hàng triệu lượt tải xuống mỗi tuần[7].

2.2.12.2. Vai trò trong phát triển Frontend

Việc sử dụng MUI giúp em tối ưu hóa quy trình phát triển giao diện:

- Tiết kiệm thời gian: Tận dụng kho component dựng sẵn chất lượng cao (Button, DataGrid, Modal, Form), giảm thiểu việc phải viết CSS thủ công.
- Tính nhất quán và Responsive: Đảm bảo giao diện hiển thị đồng bộ, chuyên nghiệp và tự động thích ứng trên nhiều kích thước màn hình (Mobile, Tablet, Desktop) mà không cần xử lý media query phức tạp [7].
- Hỗ trợ quản trị: Component *DataGrid* của MUI hỗ trợ đặc lực cho trang quản trị (Admin Dashboard) với các tính năng phân trang, lọc và sắp xếp dữ liệu mạnh mẽ.

2.2.12.3. Ứng dụng vào đồ án

Việc sử dụng Material-UI giúp giao diện đồ án đạt được tính thẩm mỹ cao, đồng nhất về phong cách thiết kế và đảm bảo khả năng hiển thị tốt trên nhiều loại thiết bị (Responsive) mà không tốn quá nhiều thời gian CSS thủ công.

CHƯƠNG 3. GIẢI PHÁP CHO BÀI TOÁN

3.1. Đặt vấn đề và phân tích bài toán

Qua quá trình khảo sát và nghiên cứu thực trạng các hệ thống thương mại điện tử (TMĐT) hiện hành, có thể nhận thấy nhiều hạn chế kỹ thuật và nghiệp vụ đang tồn tại, gây ảnh hưởng trực tiếp đến trải nghiệm người dùng cũng như khả năng vận hành của doanh nghiệp. Các vấn đề cốt lõi cần giải quyết bao gồm:

- Hạn chế về khả năng mở rộng (Scalability): Đa phần các hệ thống cũ được phát triển trên kiến trúc một khối (Monolithic). Điều này tạo ra rào cản lớn khi hệ thống cần nâng cấp hoặc mở rộng. Khi lượng truy cập tăng đột biến ở một phân hệ (ví dụ: đặt hàng), việc phải nhân bản toàn bộ khối ứng dụng gây lãng phí tài nguyên phần cứng và chi phí vận hành.
- Hiệu năng xử lý (Performance): Với dữ liệu sản phẩm và người dùng ngày càng lớn, các truy vấn trực tiếp vào cơ sở dữ liệu (Database) mà không có cơ chế đệm (Caching) hiệu quả dẫn đến độ trễ cao. Điều này làm giảm tốc độ phản hồi trang, gây ức chế cho người mua hàng.
- Rủi ro an toàn thông tin: Mô hình xác thực truyền thống lưu trữ mật khẩu trực tiếp hoặc quản lý phiên làm việc không chặt chẽ tạo ra lỗ hổng bảo mật. Quy trình đăng ký phức tạp cũng tạo ra "ma sát" (friction) khiến người dùng ngại tiếp cận.
- Thiếu hụt tính năng hỗ trợ thông minh:
 - Gợi ý sản phẩm: Thiếu các thuật toán cá nhân hóa khiến người mua khó tìm thấy sản phẩm ưng ý, làm giảm tỷ lệ chuyển đổi đơn hàng.
 - Chăm sóc khách hàng: Việc phụ thuộc vào nhân sự trực page trong giờ hành chính không đáp ứng được nhu cầu mua sắm 24/7. Thiếu chatbot AI thông minh dẫn đến việc phản hồi khách hàng bị chậm trễ.
- Quy trình vận hành thủ công: Việc tính toán phí vận chuyển, đối soát thanh toán hay quản lý kho vận còn phụ thuộc nhiều vào thao tác thủ công, dễ dẫn đến sai sót số liệu và khó khăn trong việc theo dõi trạng thái đơn hàng theo thời gian thực.

Xuất phát từ những bất cập trên, đề tài tập trung xây dựng một nền tảng TMĐT thể hệ mới dựa trên kiến trúc Microservices, tích hợp trí tuệ nhân tạo (AI) và các công nghệ thanh toán/vận chuyển hiện đại. Mục tiêu là tạo ra hệ sinh thái phục vụ đa dạng đối tượng từ doanh nghiệp bán lẻ, người bán cá nhân đến người tiêu dùng cuối, đảm bảo các tiêu chí: Nhanh chóng – Tiện lợi – An toàn – Thông minh.

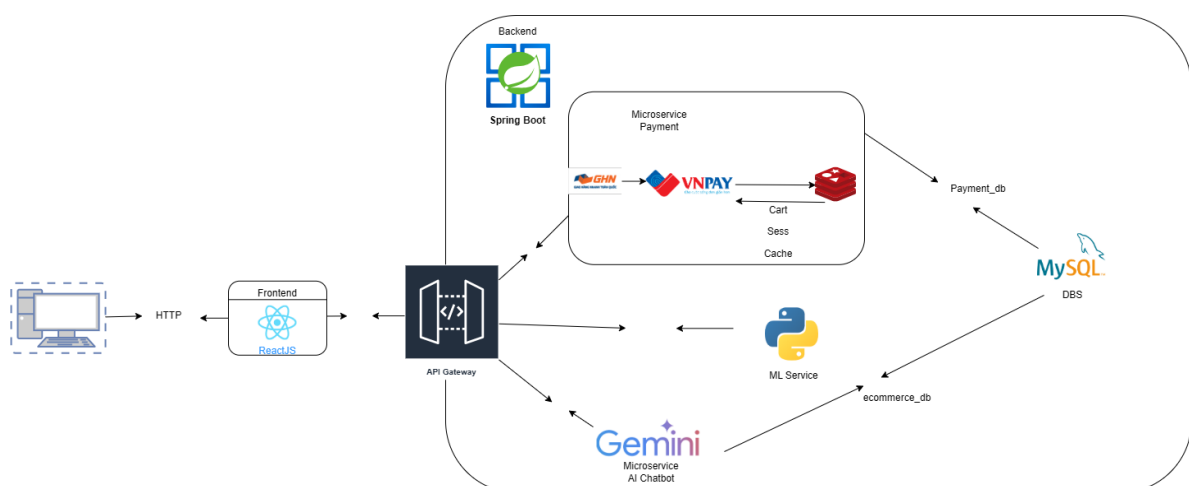
3.2. Giải pháp công nghệ đề xuất

3.2.1. Kiến trúc tổng quan.

Để giải quyết bài toán về hiệu năng và khả năng mở rộng linh hoạt, hệ thống được thiết kế theo mô hình phân tán Client-Server với sự tách biệt rõ ràng giữa Frontend và Backend:

- Client Side (Frontend): Xây dựng dưới dạng Single Page Application (SPA) sử dụng thư viện ReactJS, đảm bảo giao diện phản hồi tức thì và trải nghiệm mượt mà tương tự ứng dụng mobile.
- Server Side (Backend): Áp dụng kiến trúc Microservices với Spring Boot (Java) là nòng cốt, kết hợp với hệ sinh thái Spring Cloud để quản lý giao tiếp giữa các dịch vụ. Dữ liệu được lưu trữ phân tán và quản lý chính bởi hệ quản trị MySQL.

3.2.2. Sơ đồ kiến trúc hệ thống



Hình 3.1: Sơ đồ kiến trúc hệ thống

Chi tiết các thành phần:

1. Frontend (Giao diện người dùng):

Được phát triển bằng ReactJS kết hợp TypeScript nhằm tăng cường tính chặt chẽ của mã nguồn, giao diện sử dụng bộ thư viện Material-UI để đảm bảo tính thẩm mỹ và đồng nhất (Consistency). Frontend đóng vai trò là lớp trình diễn, tiếp nhận tương tác người dùng và gọi dữ liệu thông qua RESTful API, hoàn toàn không chứa logic nghiệp vụ phức tạp để đảm bảo tính nhẹ nhàng (lightweight).

2. API Gateway:

Đây là "cánh cổng" duy nhất tiếp nhận mọi yêu cầu từ phía Client. Thay vì Client phải gọi trực tiếp đến từng service lẻ tẻ (Product, Order, User...), tất cả sẽ đi qua API Gateway. Thành phần này đảm nhiệm các chức năng quan trọng: định tuyến (routing), cân bằng tải (load balancing), xác thực token tập trung và kiểm soát tần suất truy cập (rate limiting) để bảo vệ hệ thống khỏi các cuộc tấn công DDoS.

3. Backend Microservices:

Hệ thống backend được chia nhỏ thành các dịch vụ độc lập, mỗi dịch vụ quản lý một nghiệp vụ riêng biệt (Domain-Driven Design), bao gồm: Auth, User, Product, Order, Cart, Payment, v.v. Các dịch vụ này giao tiếp với nhau qua giao thức HTTP/REST hoặc Message Queue. Cơ chế Service Discovery (Eureka) được áp dụng để các service tự động nhận diện địa chỉ của nhau trong môi trường mạng động.

4. Cơ sở dữ liệu (Database):

Áp dụng mẫu thiết kế Database-per-service. Mỗi microservice sở hữu một cơ sở dữ liệu MySQL riêng biệt. Điều này đảm bảo tính đóng gói (encapsulation), một service bị lỗi database sẽ không kéo sập toàn bộ hệ thống. MySQL được chọn nhờ tính ổn định và tuân thủ các chuẩn ACID, phù hợp cho các giao dịch tài chính, đặt hàng.

5. Caching Layer (Redis):

Sử dụng Redis làm bộ nhớ đệm tốc độ cao (In-memory database).

- Lưu trữ Giỏ hàng (Cart): Giúp thao tác thêm/sửa/xóa sản phẩm diễn ra tức thì, dữ liệu tự động hết hạn sau 24h để tối ưu bộ nhớ.
- Caching: Lưu kết quả các truy vấn phổ biến (như danh sách sản phẩm trang chủ) để giảm tải cho MySQL.

6. AI & Machine Learning Services:

- AI Service (Chatbot): Tích hợp Gemini API để xây dựng trợ lý ảo. Không chỉ trả lời theo kịch bản có sẵn, chatbot có khả năng hiểu ngữ cảnh tự nhiên (NLP) để tư vấn sản phẩm và giải đáp chính sách bảo hành/đổi trả 24/7.

- Recommendation Service: Một service riêng biệt viết bằng Python (FastAPI) sử dụng thư viện Scikit-learn. Hệ thống áp dụng thuật toán Lọc cộng tác (Collaborative Filtering - SVD) để phân tích lịch sử đánh giá và hành vi mua sắm, từ đó đưa ra gợi ý sản phẩm phù hợp nhất cho từng cá nhân.

7. Tích hợp dịch vụ bên thứ 3 (Third-party Integration)

- Vận chuyển (Shipping): Kết nối API của Giao Hàng Nhanh (GHN) để tự động tính phí ship theo địa chỉ thực tế và đồng bộ trạng thái đơn hàng (đang lấy hàng, đang giao, đã giao) về hệ thống.

- Thanh toán (Payment): Tích hợp cổng thanh toán VNPay, cho phép thanh toán qua thẻ ATM/QR Code nội địa, phù hợp với thói quen tiêu dùng tại Việt Nam.

3.2.3. Cơ chế bảo mật

Hệ thống loại bỏ cơ chế Session truyền thống (Stateful) để chuyển sang cơ chế Stateless sử dụng JSON Web Token (JWT):

- Xác thực (Authentication): Hỗ trợ đăng nhập nhanh qua OAuth2 (Google/Facebook). Khi đăng nhập thành công, Server cấp phát chuỗi Access Token (JWT).

- Phân quyền (Authorization): Token chứa thông tin về quyền hạn (Role: Admin/User/Staff). API Gateway sẽ giải mã Token này để quyết định xem người dùng có được phép truy cập vào tài nguyên (ví dụ: trang quản trị) hay không.

- Lợi ích: Cơ chế này giúp giảm tải việc lưu session trên server, dễ dàng mở rộng hệ thống (scale out) mà không lo vấn đề đồng bộ session giữa các server.

3.3. Hiện thực giải pháp

3.3.1. Mô tả giải pháp

Hệ thống được hiện thực hóa là một giải pháp TMĐT hoàn chỉnh, vận hành trên nền tảng Web. Kiến trúc phần mềm tuân thủ nghiêm ngặt các nguyên lý của Microservices nhằm đảm bảo tính linh hoạt trong bảo trì và nâng cấp.

Về mặt công nghệ, dự án là sự kết hợp giữa sức mạnh xử lý của Java Spring Boot ở Backend và tính linh hoạt của ReactJS ở Frontend. Việc container hóa các dịch vụ và sử dụng Docker (nếu có triển khai) giúp môi trường phát triển và sản phẩm (Production) được đồng nhất.

Dòng chảy dữ liệu (Data Flow) được thiết kế tối ưu: Dữ liệu nóng (Hot data) như giỏ hàng được xử lý tại Redis, dữ liệu giao dịch (Transaction) được đảm bảo an toàn tại MySQL, và các tác vụ tính toán nặng (AI/ML) được tách ra các service riêng biệt để không làm chậm luồng xử lý chính.

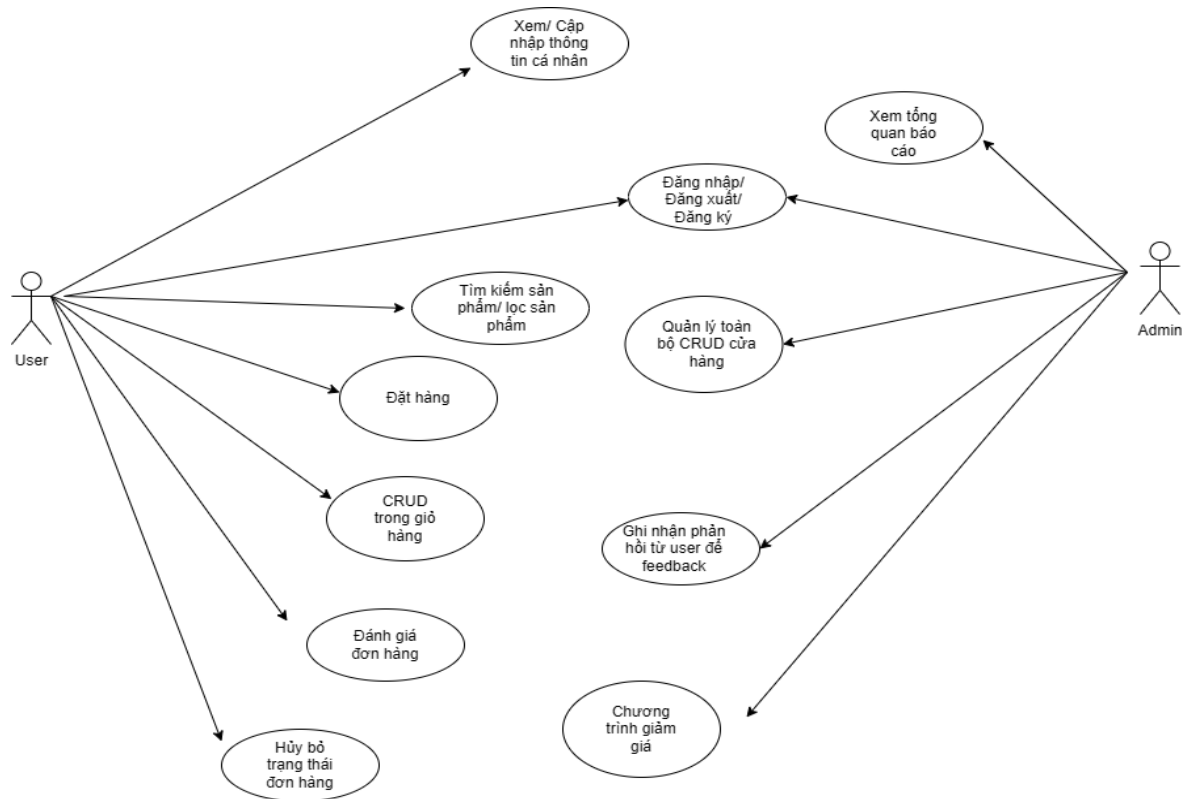
3.3.2. Các chức năng chính đã hiện thực:

Hệ thống cung cấp đầy đủ các phân hệ chức năng phục vụ cho cả người mua (Front-office) và người quản trị (Back-office):

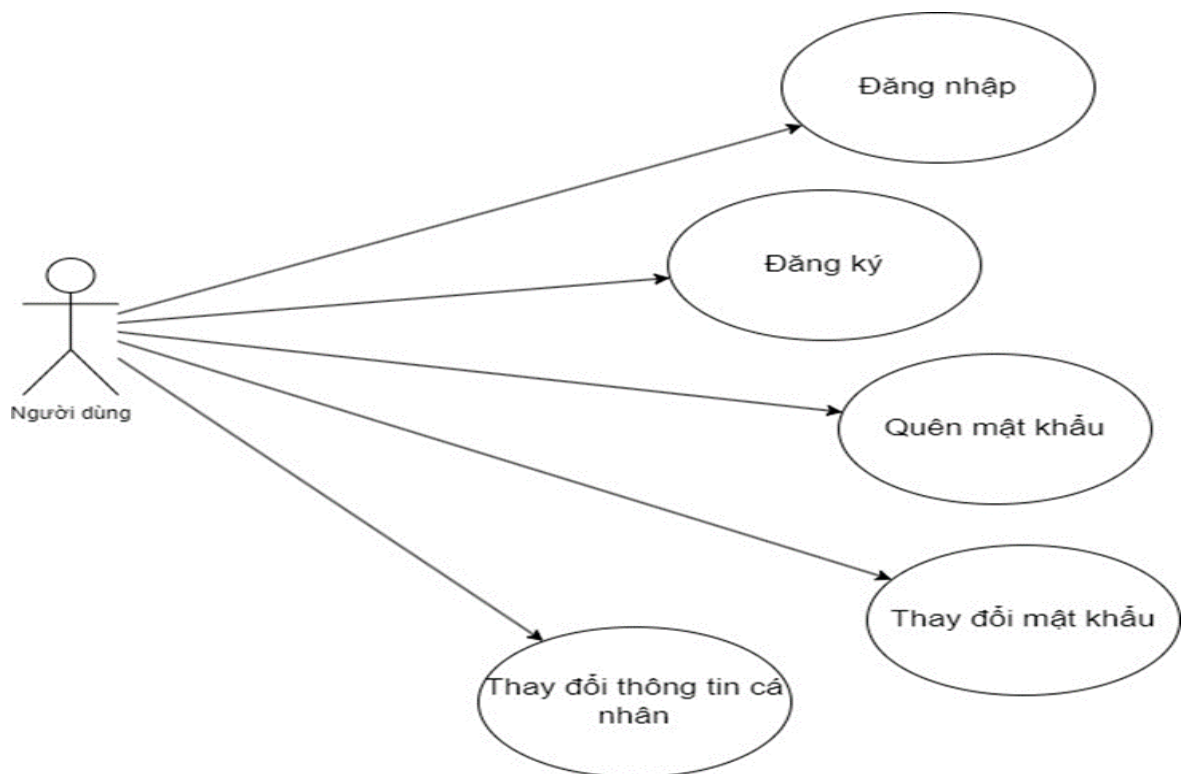
1. Quản lý danh mục và Sản phẩm:
 - Hỗ trợ tìm kiếm, lọc nâng cao theo giá, thương hiệu, danh mục.
 - Hiện thị chi tiết thông tin kỹ thuật, hình ảnh trực quan và tình trạng tồn kho theo thời gian thực.
2. Giỏ hàng thông minh (Smart Cart):
 - Lưu trữ giỏ hàng tạm thời trên Redis, đảm bảo không mất giỏ hàng khi người dùng tải lại trang (F5) hoặc tắt trình duyệt (trong thời gian TTL).
 - Tự động tính tổng tiền và áp dụng mã giảm giá sơ bộ.
3. Quy trình Đặt hàng và Vận chuyển:
 - Tự động kết nối GHN để tính phí ship dựa trên phường/xã người nhận.
 - Quy trình Checkout 3 bước tinh gọn. Người dùng có thể theo dõi mã vận đơn và lộ trình đơn hàng ngay trên Website.

4. Thanh toán điện tử:
 - Xử lý thanh toán an toàn qua cổng VNPay. Hệ thống tự động cập nhật trạng thái đơn hàng thành "Đã thanh toán" ngay khi nhận tín hiệu IPN (Instant Payment Notification) từ cổng thanh toán.
5. Hệ thống Gợi ý (Recommendation System):
 - Hiện thị mục "Có thể bạn cũng thích" tại trang chủ và chi tiết sản phẩm.
 - Sử dụng thuật toán học máy để cá nhân hóa danh sách này dựa trên lịch sử xem và mua hàng của user.
6. Trợ lý ảo AI (Chatbot):
 - Cửa sổ chat luôn hiển thị (Floating button).
 - Hỗ trợ tra cứu đơn hàng, tư vấn cấu hình sản phẩm bằng ngôn ngữ tự nhiên.
7. Quản lý Khuyến mãi (Voucher):
 - Admin có thể tạo các mã giảm giá theo % hoặc số tiền cố định, giới hạn số lượng sử dụng và thời gian áp dụng.
8. Đánh giá và Phản hồi (Review):
 - Cho phép người dùng đã mua hàng đánh giá sao và viết bình luận.
 - Tích hợp bộ lọc từ khóa nhạy cảm cơ bản.
9. Bảo hành và Hậu mãi:
 - Module cho phép người dùng gửi yêu cầu bảo hành online và theo dõi tiến độ xử lý thay vì phải gọi điện thoại.
10. Phân hệ Quản trị (Admin Dashboard):
 - Cung cấp các biểu đồ thống kê doanh thu, số lượng đơn hàng theo ngày/tháng.
 - Quản lý toàn bộ vòng đời của sản phẩm, đơn hàng và tài khoản người dùng trong hệ thống.

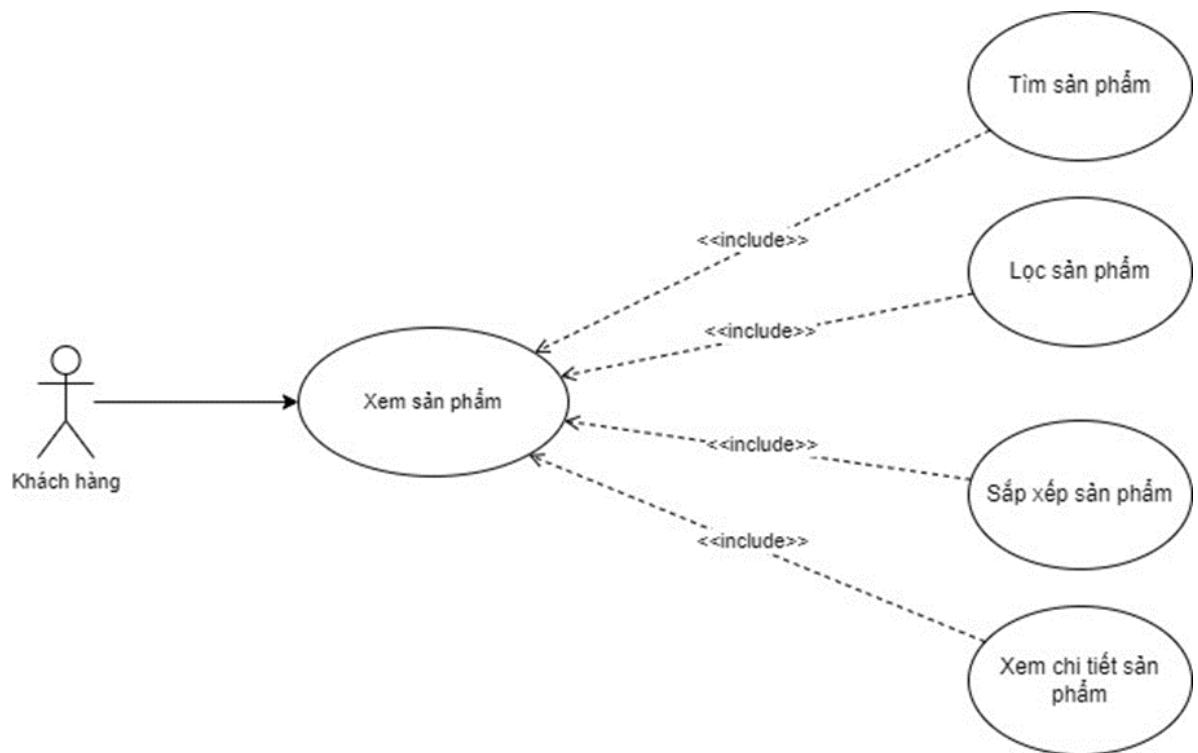
3.3.3. Lược đồ Use case



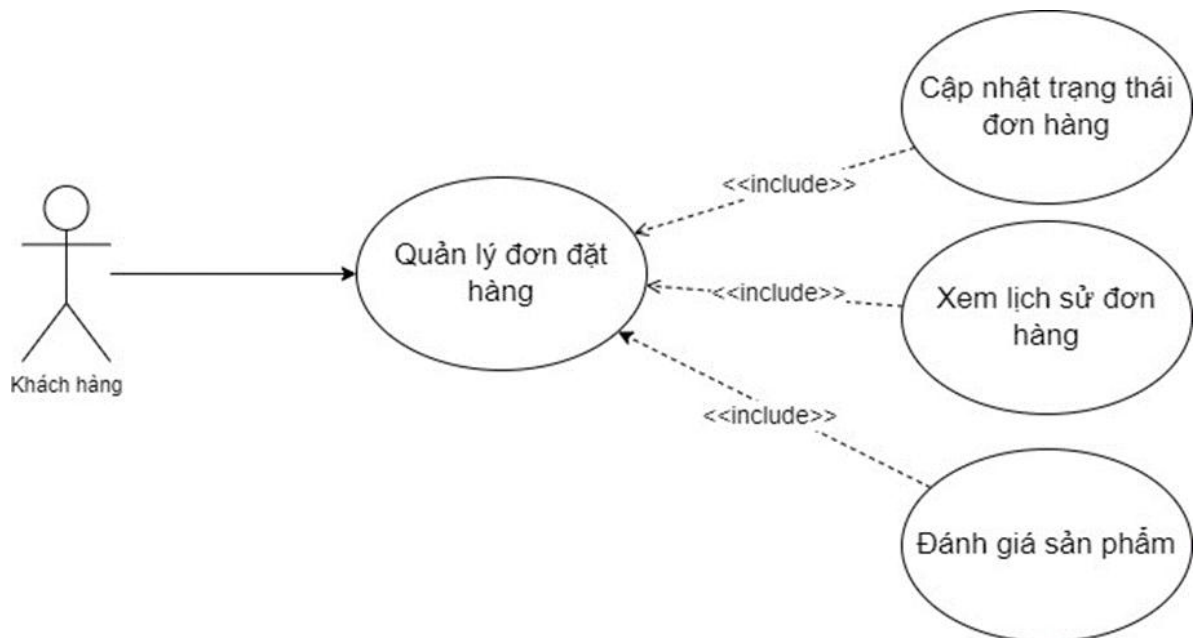
Hình 3.2: Sơ đồ usecase của hệ thống.



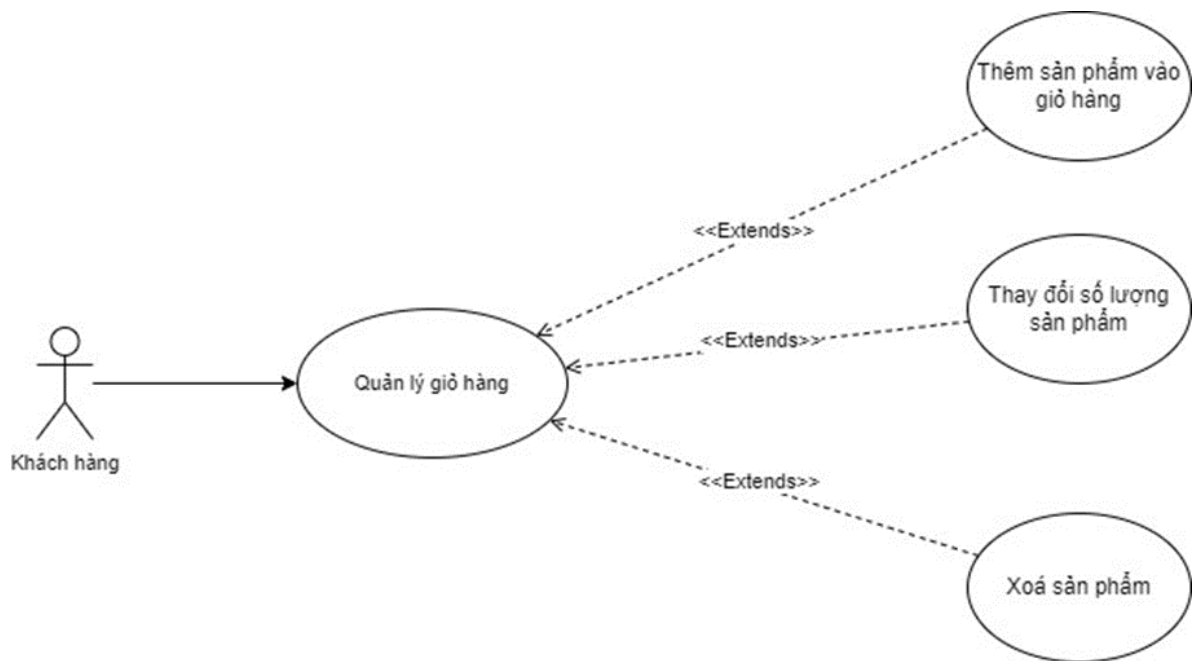
Hình 3.3: Lược đồ use case tổng quát của hệ thống



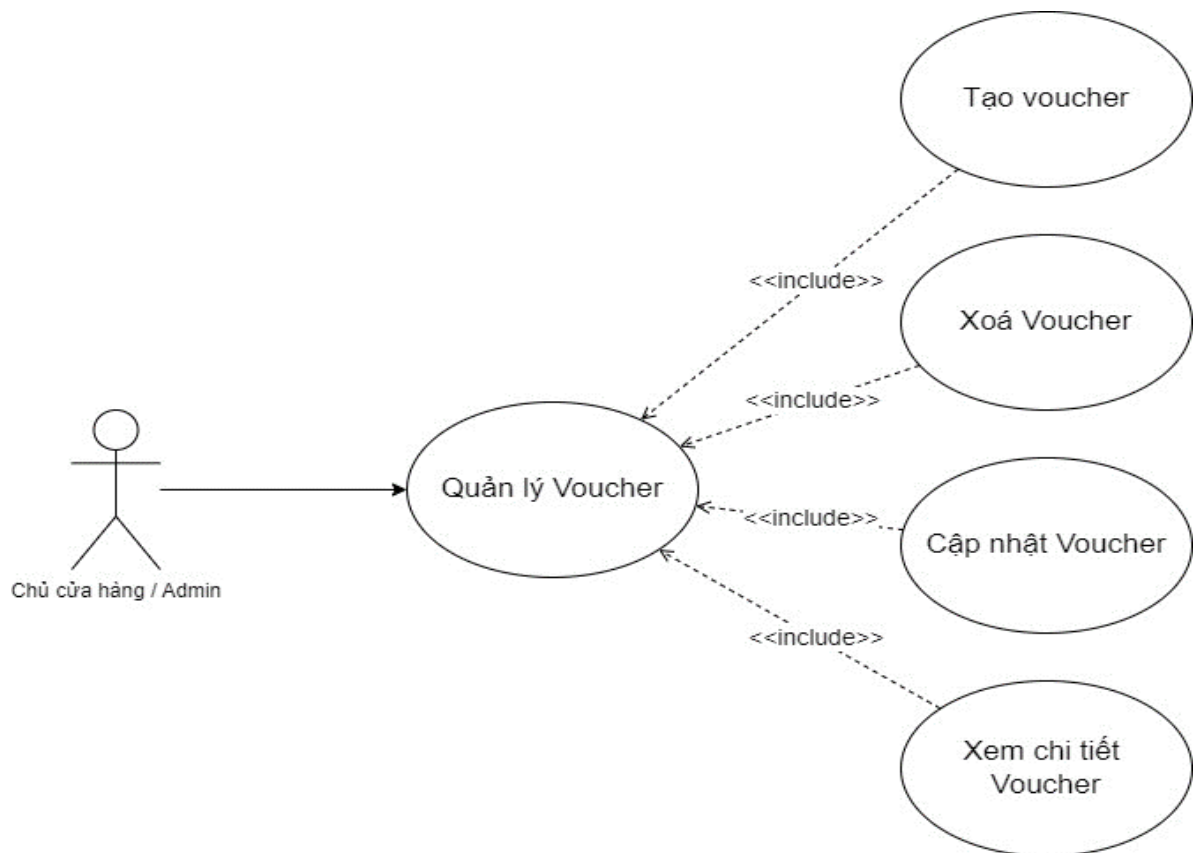
Hình 3.4: Lược đồ use case Xem sản phẩm



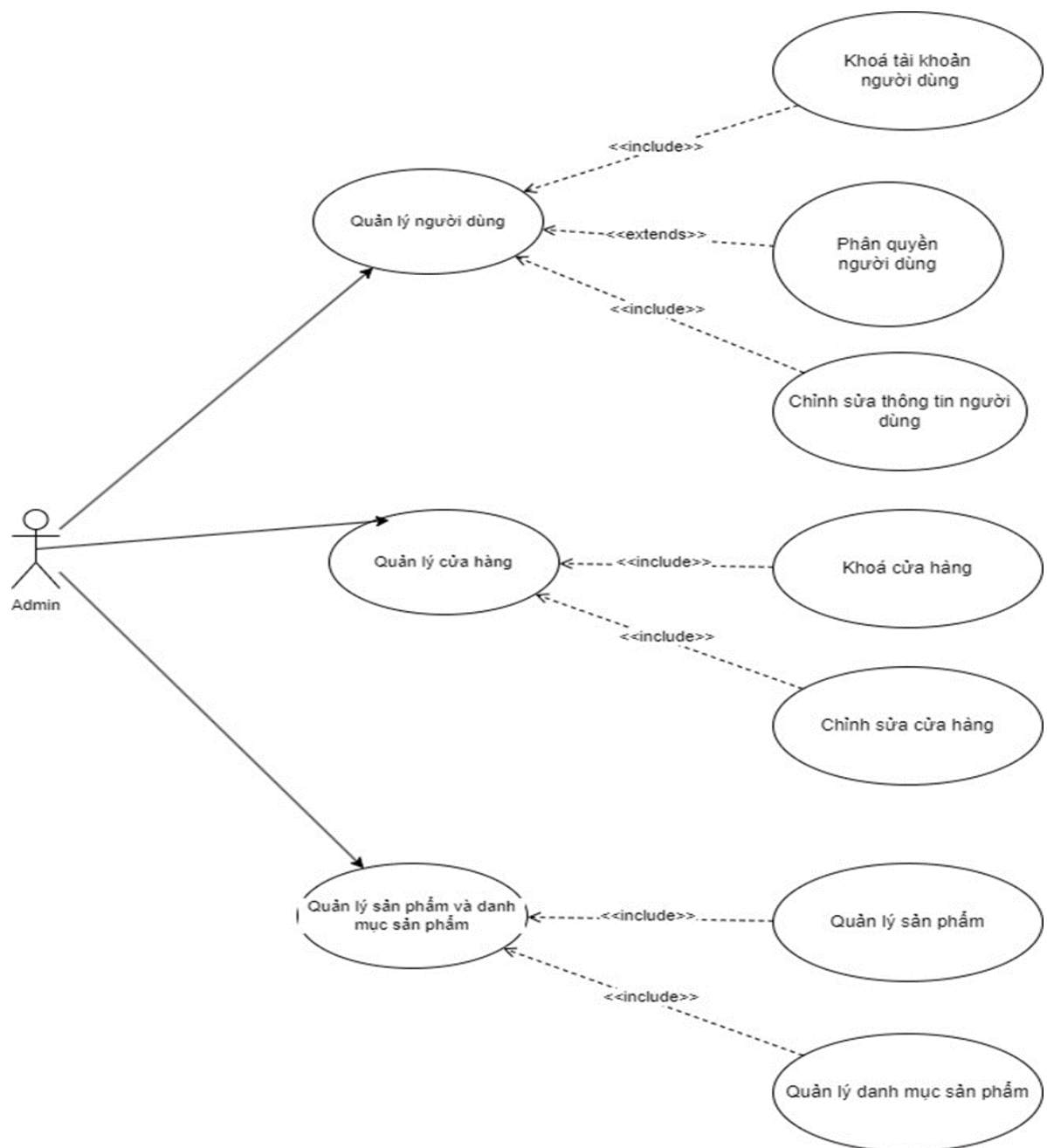
Hình 3.5: Lược đồ use case quản lý đơn đặt hàng của khách hàng



Hình 3.6: Lược đồ use case quản lý giỏ hàng



Hình 3.7: Lược đồ use case quản lý khuyến mãi của cửa hàng



Hình 3.8: Lược đồ use case quản lý hệ thống sàn thương mại điện tử

Bảng 3.1: Đặc tả usecase đăng ký.

Tên usecase	Đăng kí
Tác nhân	Người dùng

Mô tả	Cho phép người dùng tạo một tài khoản với hệ thống.
Tiền điều kiện	Người dùng có email hợp lệ.
Hậu điều kiện	Thông tin của người dùng được lưu vào cơ sở dữ liệu. Mật khẩu của người dùng được mã hóa trong cơ sở dữ liệu. Hệ thống tự động đăng nhập khi đăng kí thành công.
Luồng sự kiện chính	1. Người dùng truy cập vào trang đăng nhập. 2. Người dùng nhập họ tên, email, password. 3. Hệ thống kiểm tra thông tin đăng ký. 4. Nếu hợp lệ. Hệ thống thông báo đăng ký thành công điều hướng sang trang chủ.
Luồng sự kiện thay thế	4a. Nếu không hợp lệ. Hệ thống thông báo và yêu cầu nhập lại

Bảng 3.2: Đặc tả usecase đăng nhập.

Tên usecase	Đăng nhập
Tác nhân	Người dùng, admin
Mô tả	Cho phép người dùng đăng nhập vào hệ thống.
Tiền điều kiện	Người dùng đã đăng ký tài khoản trước đó. Tài khoản không bị chặn.

Luồng sự kiện chính	1. Người dùng truy cập vào trang đăng nhập của hệ thống. 2. Người dùng nhập thông tin đăng nhập gồm có email và password. 3. Hệ thống kiểm tra thông tin đăng nhập. 4. Nếu hợp lệ. Hệ thống điều hướng sang trang chủ.
Luồng sự kiện thay thế	4a. Nếu không hợp lệ. Hệ thống thông báo lỗi và yêu cầu nhập lại thông tin đăng nhập.

Bảng 3.3: Đặc tả usecase cập nhật thông tin cá nhân

Tên usecase	Cập nhật thông tin cá nhân
Tác nhân	Người dùng
Mô tả	Cho phép người dùng xem và cập nhật thông tin như họ và tên email, ảnh đại diện.
Tiền điều kiện	Người dùng đã đăng nhập. Thông tin của người dùng được cập nhật trong cơ sở dữ liệu.

Luồng sự kiện chính	1. Người dùng nhấn nút có biểu tượng “account” ở Header. 2. Hệ thống hiển thị ngăn kéo các hiển thị thông tin cá nhân hiện tại của người dùng. 3. Người dùng nhấn biểu tượng “edit” để cập nhật ảnh đại diện. 4. Hệ thống hiển thị các file trong thiết bị 5. Người dùng chọn file và nhấn OK. 6. Hệ thống hiển thị hình ảnh. 7. Người dùng nhập email và họ và tên và nhấn “Cập nhật”. 8. Hệ thống kiểm tra thông tin cập nhật. 9. Nếu người dùng có thay đổi email. Hệ thống yêu cầu nhập mật khẩu. 10. Nếu đúng mật khẩu, hệ thống thông báo cập nhật thành công.
	9. Nếu hợp lệ. Hệ thống hiển thị thông báo cập nhật thành công và gửi mail cho đến tài khoản email cũ của tài khoản.
Luồng sự kiện thay thế	9a. Nếu người dùng không thay đổi thông tin nào. Hệ thống thông báo “Vui lòng thay đổi thông tin”. 9b. Nếu người dùng không thay đổi email. Hệ thống thông báo cập nhật thông tin thành công. 10a. Nếu người dùng nhập sai mật khẩu. Hệ thống thông báo sai mật khẩu.

Bảng 3.4: Đặc tả usecase đổi mật khẩu

Tên usecase	Đổi mật khẩu
Tác nhân	Người dùng
Mô tả	Cho phép người xem và cập nhật thông như họ và tên email, ảnh đại diện.
Tiền điều kiện	Người dùng đã đăng nhập. Người dùng đã mở “ngăn kéo” cập nhật thông tin cá nhân.
Luồng sự kiện chính	1. Người dùng nhấn “Đổi mật khẩu”. 2. Hệ thống mở “ngăn kéo” đổi mật khẩu. 3. Người dùng nhập mật khẩu cũ và mật khẩu mới và nhấn “Thay đổi”. 4. Hệ thống gửi yêu cầu kiểm tra mật khẩu. 5. Nếu mật khẩu đúng. Hệ thống thông báo thay đổi mật khẩu thành công.
Luồng sự kiện thay thế	5a. Nếu mật khẩu sai, hệ thống thông báo mật khẩu cũ không chính xác. 5b. Nếu người dùng chưa nhập vào mật khẩu cũ hoặc mật khẩu mới, hệ thống thông báo “Mật khẩu cũ và mới không được để trống”.

Bảng 3.5: Đặc tả usecase mua hàng

Tên usecase	Mua hàng
Tác nhân	Người dùng

GIẢI PHÁP CHO BÀI TOÁN

Mô tả	Cho phép người dùng đặt mua sản phẩm trên hệ thống, bao gồm chọn sản phẩm, xác nhận thông tin nhận hàng, phương thức thanh toán và hoàn tất đơn hàng.
Tiền điều kiện	Người dùng đã đăng nhập. Người dùng đã chọn được ít nhất một sản phẩm đưa vào giỏ hàng. Sản phẩm vẫn còn hàng trong kho.
Luồng sự kiện chính	1. Người dùng truy cập vào trang giỏ hàng. 2. Hệ thống hiển thị danh sách sản phẩm trong giỏ (tên, số lượng, đơn giá, thành tiền). 3. Người dùng kiểm tra lại số lượng, thông tin sản phẩm và nhấn “Tiến hành đặt hàng”. 4. Hệ thống hiển thị trang xác nhận đơn hàng gồm: thông tin người nhận (họ tên, số điện thoại, địa chỉ), phương thức thanh toán, phí vận chuyển và tổng tiền. 5. Người dùng nhập/kiểm tra thông tin người nhận, chọn phương thức thanh toán và nhấn “Xác nhận đặt hàng”. 6. Hệ thống tạo yêu cầu đặt hàng. 7. Hệ thống lưu đơn hàng xuống cơ sở dữ liệu, trừ tồn kho tương ứng. 8. Hệ thống hiển thị thông báo “Đặt hàng thành công” và mã đơn hàng.
Luồng sự kiện thay thế	3a. Nếu người dùng muốn thay đổi số lượng hoặc xóa sản phẩm trong giỏ. 3a1. Người dùng chỉnh sửa số lượng hoặc xóa sản phẩm khỏi giỏ.

	3a2. Hệ thống cập nhật lại giỏ hàng và tổng tiền. 3a3. Người dùng nhấn “Tiến hành đặt hàng” để tiếp tục luồng chính tại bước 4. 5a. Nếu người dùng chọn thanh toán khi nhận hàng (COD) 5a1. Người dùng chọn phương thức “Thanh toán khi nhận hàng”. 5a2. Người dùng nhấn “Xác nhận đặt hàng”. 5a3. Hệ thống xử lý như luồng chính từ bước 6 trở đi. 5b. Nếu người dùng chọn thanh toán online (ví điện tử/thẻ ngân hàng) 5b1. Người dùng chọn phương thức thanh toán online. 5b2. Hệ thống chuyển hướng đến cổng thanh toán. 5b3. Người dùng nhập thông tin thanh toán và xác nhận. 5b4. Cổng thanh toán thông báo kết quả cho hệ thống. 5b5. Nếu thanh toán thành công, hệ thống tiếp tục luồng chính từ bước 6. 5b6. Nếu thanh toán thất bại, hệ thống hiển thị thông báo lỗi và cho phép người dùng chọn lại phương thức thanh toán hoặc thử lại. 7a. Nếu trong quá trình lưu đơn hàng phát sinh lỗi hệ thống 7a1. Hệ thống không lưu được đơn hàng. 7a2. Hệ thống hiển thị thông báo “Đặt hàng thất bại, vui lòng thử lại sau”. 7a3. Hệ thống không trừ tồn kho và không tạo mã đơn hàng.
--	--

Bảng 3.6: Đặc tả use case admin duyệt đơn hàng.

Tên usecase	Duyệt đơn hàng
Tác nhân	Admin
Mô tả	Cho phép quản trị viên xem và duyệt các đơn hàng do người dùng tạo, xác nhận các đơn hợp lệ để chuyển sang bước xử lý/đóng gói/giao hàng.
Tiền điều kiện	Admin đã đăng nhập. Tài khoản có quyền quản trị và quyền duyệt đơn hàng.
Luồng sự kiện chính	1. Admin nhấn vào biểu tượng/ mục “Quản lý đơn hàng” trong thanh điều hướng (menu quản trị). 2. Hệ thống hiển thị trang danh sách đơn hàng chờ duyệt (trạng thái “Chờ duyệt”). 3. Admin chọn một đơn hàng bất kỳ trong danh sách. 4. Hệ thống hiển thị trang chi tiết đơn hàng (thông tin khách hàng, địa chỉ, sản phẩm, số lượng, tổng tiền, phương thức thanh toán, trạng thái hiện tại...). 5. Admin kiểm tra thông tin đơn hàng và nhấn nút “Duyệt đơn”. 6. Hệ thống gửi yêu cầu cập nhật trạng thái đơn hàng. 7. Hệ thống cập nhật trạng thái đơn hàng sang “Đã duyệt” (hoặc “Đang xử lý”) xuống cơ sở dữ liệu. 8. Hệ thống hiển thị thông báo “Duyệt đơn hàng thành công” và cập nhật lại trạng thái trên giao diện.

Luồng sự kiện thay thế	2a. Nếu không có đơn hàng chờ duyệt 2a1. Hệ thống hiển thị thông báo “Không có đơn hàng chờ duyệt”. 2a2. Admin có thể chọn bộ lọc khác (đã duyệt, đã hủy, ...) hoặc quay lại trang quản trị. 3a. Admin muốn lọc/ tìm kiếm đơn hàng trước khi chọn 3a1. Admin nhập điều kiện tìm kiếm (mã đơn, tên khách hàng, ngày đặt, trạng thái...) hoặc chọn bộ lọc. 3a2. Hệ thống hiển thị danh sách đơn hàng theo điều kiện tìm kiếm/lọc. 3a3. Admin chọn một đơn hàng trong danh sách để tiếp tục luồng chính tại bước 4. 5a. Admin không duyệt mà từ chối đơn hàng 5a1. Admin nhấn nút “Từ chối đơn hàng” (hoặc “Hủy đơn”). 5a2. Hệ thống hiển thị hộp thoại yêu cầu nhập lý do từ chối. 5a3. Admin nhập lý do và nhấn “Xác nhận”. 5a4. Hệ thống cập nhật trạng thái đơn hàng sang “Bị từ chối” (hoặc “Đã hủy”) và lưu lý do. 5a5. Hệ thống hiển thị thông báo “Từ chối đơn hàng thành công”. 7a. Nếu trong quá trình cập nhật trạng thái phát sinh lỗi hệ thống 7a1. Hệ thống không cập nhật được trạng thái đơn hàng. 7a2. Hệ thống hiển thị thông báo “Duyệt đơn hàng thất bại, vui lòng thử lại sau”.
-------------------------------	---

	7a3. Trạng thái đơn hàng vẫn giữ nguyên như trước khi duyệt. 4a. Nếu đơn hàng có dấu hiệu bất thường (thiếu thông tin, tổng tiền sai, sản phẩm hết hàng, ...) 4a1. Admin phát hiện bất thường khi xem chi tiết đơn hàng. 4a2. Admin không nhấn “Duyệt đơn” mà có thể: Chỉnh sửa một số thông tin cho phép (nếu hệ thống hỗ trợ), hoặc Ghi chú nội bộ, hoặc Liên hệ khách hàng/ bộ phận liên quan để xác minh. 4a3. Admin quyết định: Quay lại bước 5 của luồng chính để duyệt, hoặc Thực hiện luồng 5a để từ chối đơn hàng.
--	---

Bảng 3.7: Đặc tả usecase thanh toán.

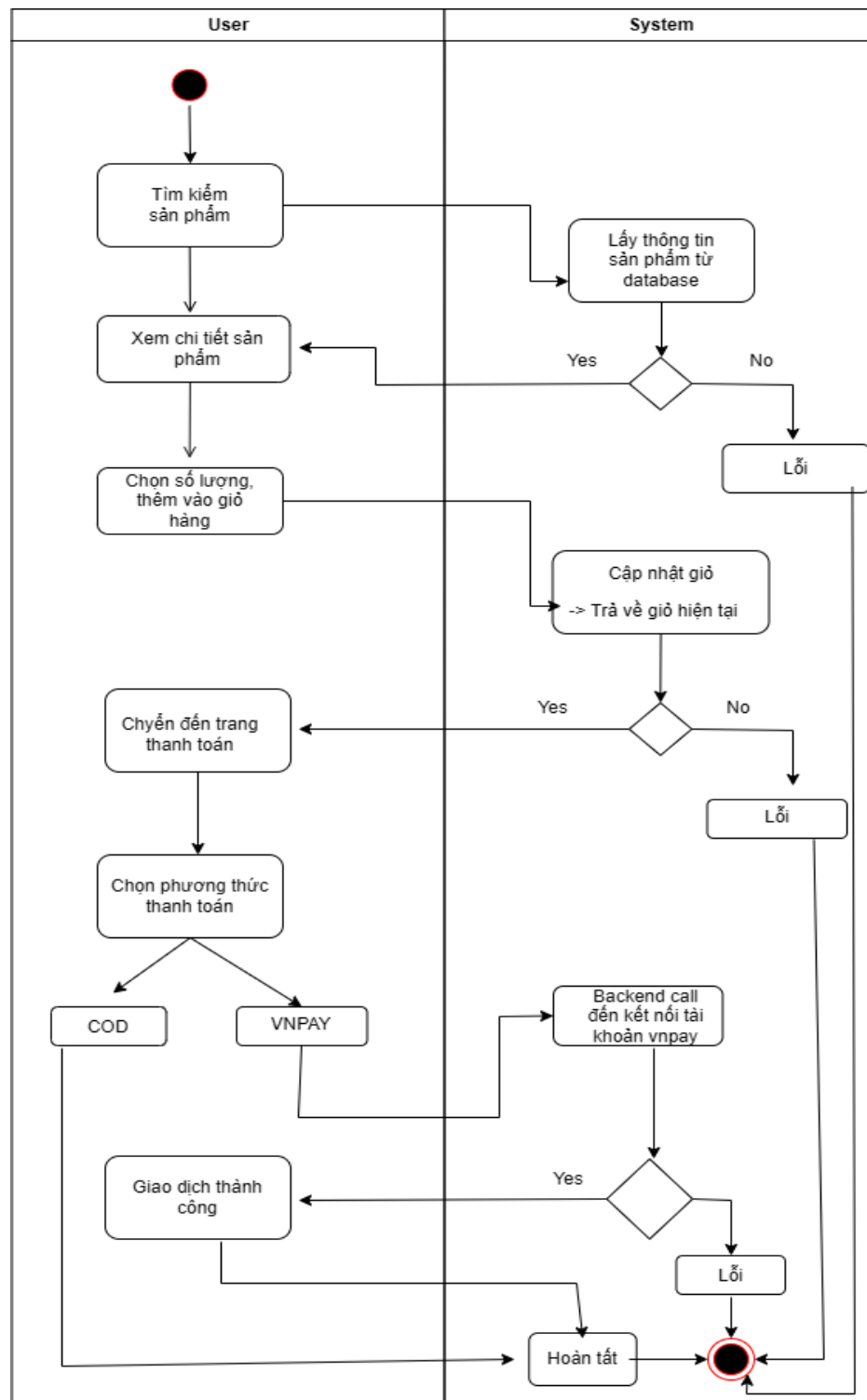
Tên usecase	Thanh toán
Tác nhân	Người dùng
Mô tả	Cho phép người dùng thanh toán cho đơn hàng đã tạo bằng nhiều phương thức khác nhau (COD, ví điện tử, thẻ ngân hàng...), xác nhận giao dịch và hoàn tất đơn hàng.
Tiền điều kiện	Người dùng đã đăng nhập.

	Người dùng đã có đơn hàng cần thanh toán (đã tạo đơn hàng, còn hiệu lực). Tổng tiền đơn hàng và thông tin nhận hàng đã được xác nhận.
Luồng sự kiện chính	1. Người dùng truy cập trang chi tiết đơn hàng hoặc trang thanh toán. 2. Hệ thống hiển thị thông tin đơn hàng (sản phẩm, số lượng, tổng tiền, phí vận chuyển, thông tin người nhận). 3. Người dùng chọn phương thức thanh toán (ví dụ: Thanh toán khi nhận hàng (COD), Thanh toán online qua ví/thẻ). 4. Người dùng nhấn nút “Thanh toán”. 5. Hệ thống kiểm tra tính hợp lệ của đơn hàng (trạng thái, tồn kho, tổng tiền...). 6. Hệ thống gửi yêu cầu xử lý thanh toán tương ứng với phương thức đã chọn. 7. Thanh toán thành công, hệ thống cập nhật trạng thái đơn hàng (ví dụ: “Đã thanh toán”, “Chờ giao hàng”). 8. Hệ thống hiển thị thông báo “Thanh toán thành công” và thông tin tóm tắt đơn hàng.
Luồng sự kiện thay thế	3a. Người dùng chọn thanh toán khi nhận hàng (COD) 3a1. Người dùng chọn phương thức “Thanh toán khi nhận hàng (COD)”. 3a2. Người dùng nhấn “Thanh toán”.

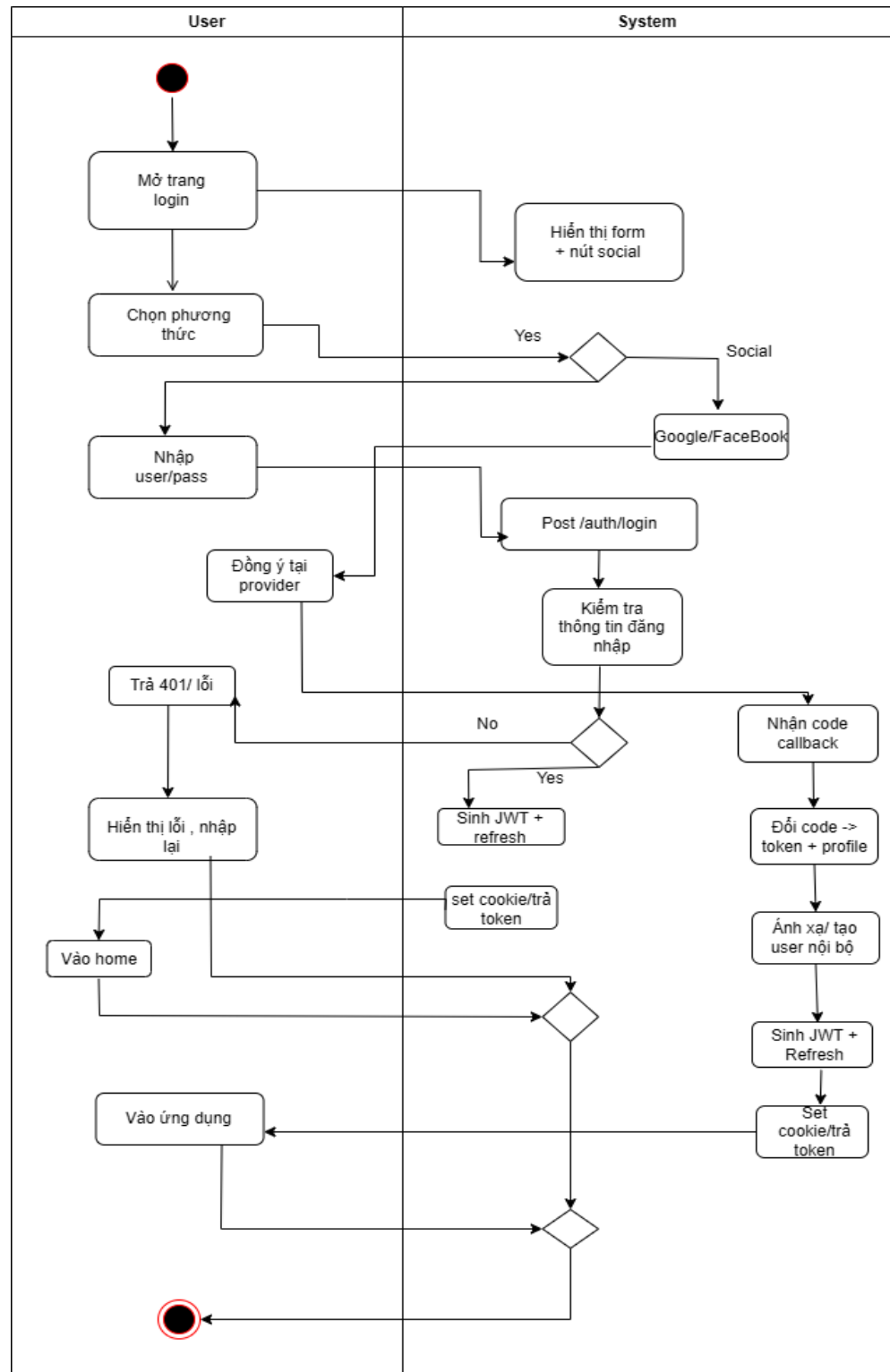
	3a3. Hệ thống bỏ qua bước chuyển hướng cổng thanh toán online, đánh dấu hình thức thanh toán là COD. 3a4. Hệ thống cập nhật trạng thái đơn hàng sang “Chờ giao hàng” (hoặc tương đương) và tiếp tục từ bước 7 luồng chính. 3b. Người dùng chọn thanh toán online (ví/thẻ) 3b1. Người dùng chọn phương thức thanh toán online (ví điện tử, thẻ ngân hàng...). 3b2. Người dùng nhấn “Thanh toán”. 3b3. Hệ thống chuyển hướng đến cổng thanh toán tương ứng. 3b4. Người dùng nhập thông tin thanh toán (nếu cần) và xác nhận trên cổng thanh toán. 3b5. Cổng thanh toán trả kết quả về cho hệ thống. 3b6. Nếu thanh toán thành công, hệ thống tiếp tục luồng chính từ bước 7. 3b7. Nếu thanh toán thất bại, xem luồng thay thế 7a. 5a. Kiểm tra đơn hàng không hợp lệ 5a1. Hệ thống phát hiện đơn hàng không hợp lệ (hết hàng, đơn đã hủy, tổng tiền sai, quá hạn, v.v.). 5a2. Hệ thống không tiếp tục xử lý thanh toán. 5a3. Hệ thống hiển thị thông báo lỗi phù hợp (ví dụ: “Đơn hàng không còn hợp lệ, vui lòng kiểm tra lại”). 7a. Thanh toán online thất bại 7a1. Cổng thanh toán thông báo giao dịch thất bại (thẻ không đủ tiền, nhập sai thông tin, bị từ chối...).
--	--

	7a2. Hệ thống không cập nhật trạng thái đơn hàng sang “Đã thanh toán”. 7a3. Hệ thống hiển thị thông báo “Thanh toán thất bại, vui lòng thử lại hoặc chọn phương thức khác”. 7a4. Người dùng có thể: Thử thanh toán lại cùng phương thức, hoặc Quay lại bước 3 để chọn phương thức thanh toán khác, hoặc Hủy thao tác thanh toán. 7b. Lỗi hệ thống trong quá trình cập nhật trạng thái 7b1. Hệ thống nhận kết quả thanh toán thành công nhưng lỗi khi cập nhật trạng thái đơn hàng. 7b2. Hệ thống ghi log lỗi, không đổi trạng thái đơn hàng hoặc chuyển sang trạng thái “Chờ xác minh”. 7b3. Hệ thống hiển thị thông báo “Có lỗi xảy ra khi xử lý đơn hàng, vui lòng liên hệ hỗ trợ”.
--	---

3.3.4. Lược đồ activity.

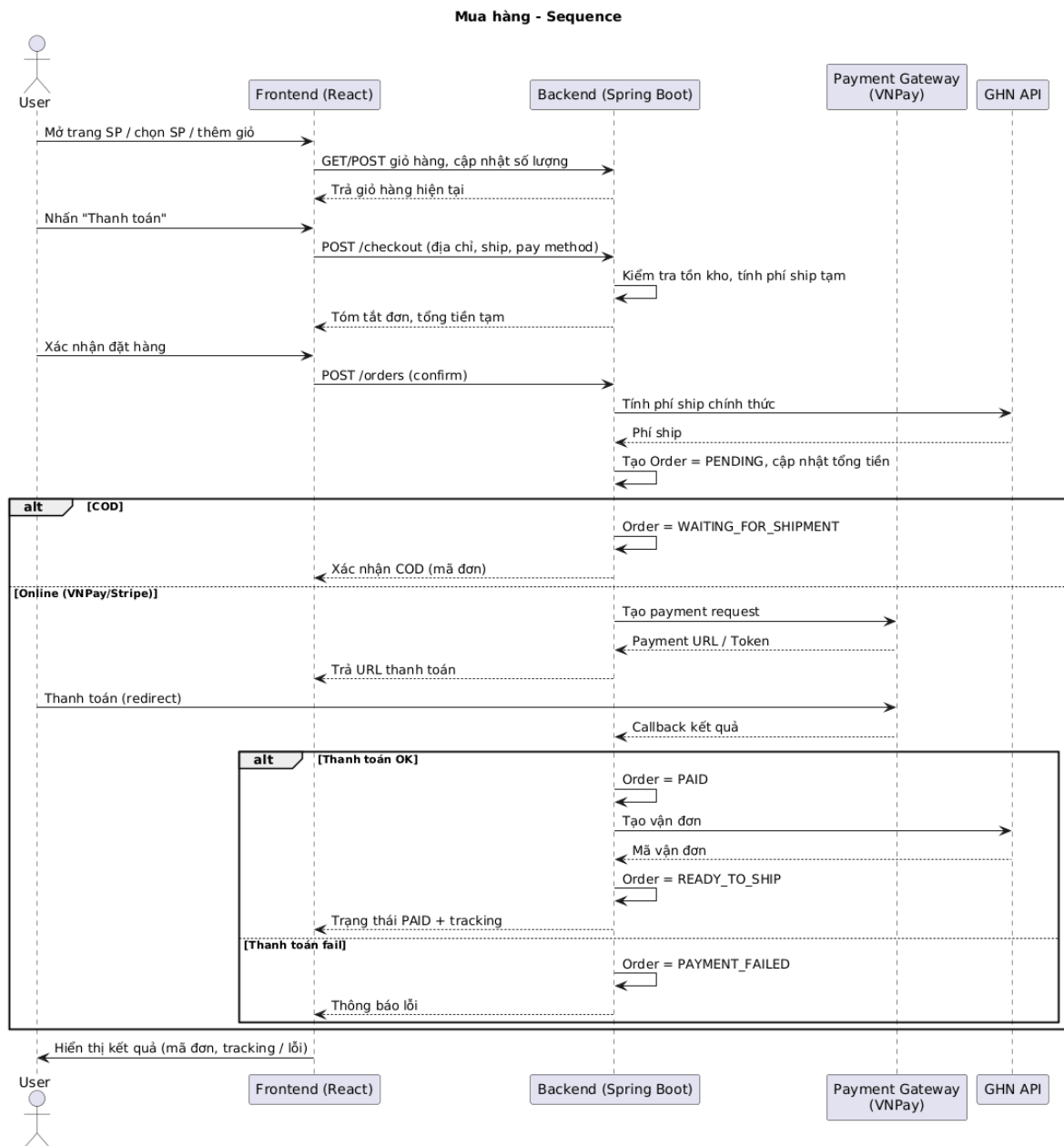


Hình 3.9: Lược đồ activity cho usecase mua hàng



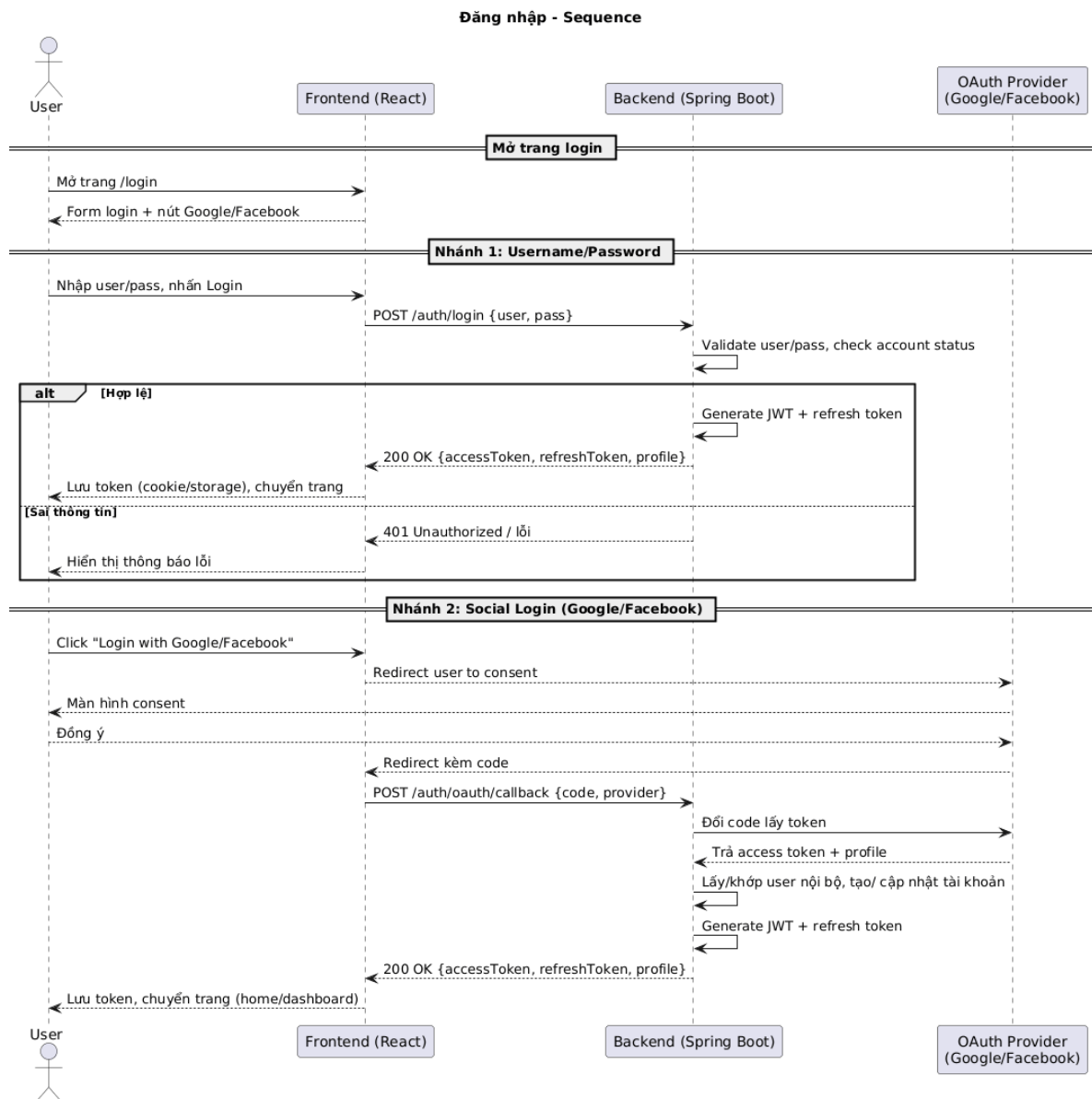
Hình 3.10: Lược đồ activity cho usecase login

3.3.5. Lược đồ tuần tự.



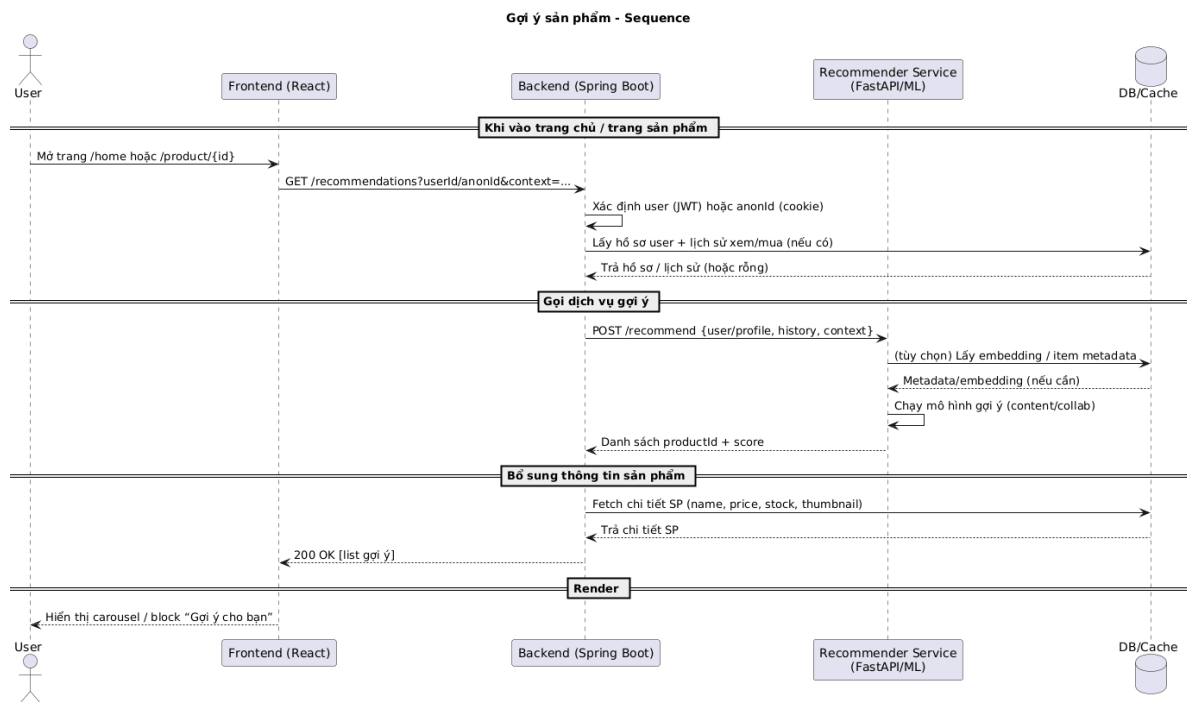
Hình 3.11: Lược đồ tuần tự cho usecase mua hàng.

GIẢI PHÁP CHO BÀI TOÁN



Hình 3.12: Lược đồ tuần tự cho usecase login.

GIẢI PHÁP CHO BÀI TOÁN



Hình 3.13: Lược đồ tuần tự cho usecase bình chọn.

3.3.6. Các màn hình của hệ thống

Hệ thống được triển khai với Frontend React + TypeScript (MUI), Backend Spring Boot (Java 17) dùng MySQL làm cơ sở dữ liệu và Redis cho cache/session, tích hợp VNPAY/Stripe cho thanh toán và GHN cho vận chuyển. Xác thực người dùng dựa trên JWT, hỗ trợ đăng nhập Google/Facebook. Tiểu luận đã hoàn thiện nghiên cứu và xây dựng luồng mua hàng trực tuyến: duyệt sản phẩm, giỏ hàng, đặt hàng, thanh toán (COD hoặc online), tạo vận đơn, cùng các tính năng gợi ý sản phẩm và chatbot hỗ trợ, đảm bảo khả năng mở rộng và tích hợp với hạ tầng Spring Cloud (Gateway, Eureka, Config).

Tiểu luận đã hoàn thành việc nghiên cứu và xây dựng một hệ thống mua sắm, tích hợp các chức năng chính sau:

3.3.6.1 Các chức năng thông minh và tích hợp AI

3.3.6.1.1 Tích hợp Generative AI (Chatbot tư vấn thông minh)

1. Công nghệ và Thư viện sử dụng

Hệ thống sử dụng Google Gemini API làm lõi xử lý ngôn ngữ tự nhiên, được tích hợp vào Backend thông qua Spring Boot.

- Mô hình: gemini-2.5-pro (cho việc sinh văn bản/câu trả lời) và text-embedding-004 (cho chức năng tìm kiếm ngữ nghĩa).
- Cơ chế giao tiếp: Sử dụng Spring WebFlux (Reactive Programming) để gọi API bất đồng bộ, đảm bảo hiệu năng cao khi xử lý nhiều request chat cùng lúc.

2. Logic xử lý (Code Implementation)

Logic chính được xử lý tại lớp GeminiService.java (thuộc ai-service). Quy trình xử lý bao gồm:

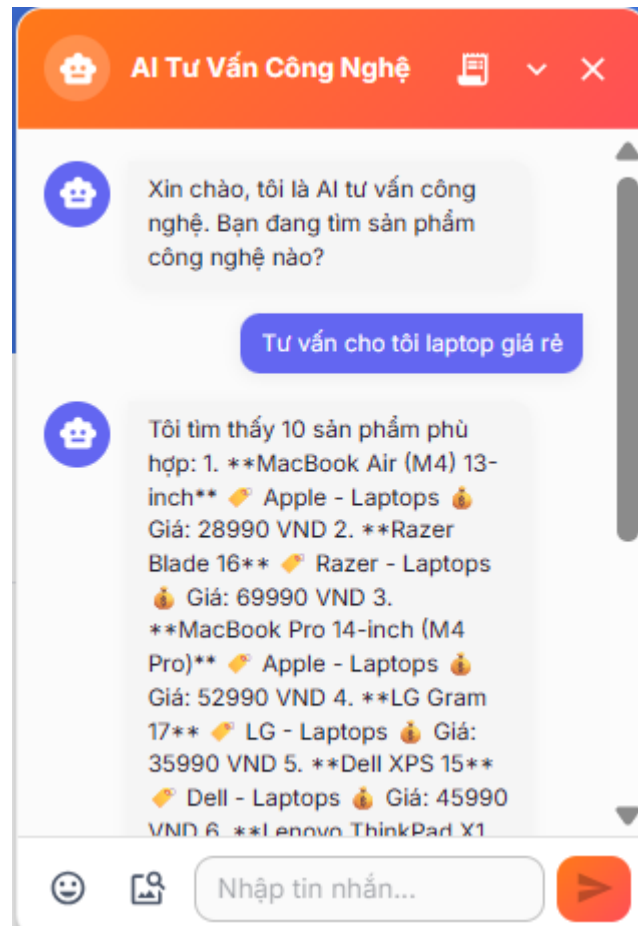
- Nhận câu hỏi từ người dùng qua REST API.
- Xây dựng ngữ cảnh (Prompt Engineering) bao gồm thông tin sản phẩm hoặc lịch sử chat để gửi kèm request.
- Gọi đến Google Gemini API thông qua WebClient.
- Nhận phản hồi JSON, trích xuất nội dung văn bản trả lời và gửi về cho Client.

```
371  /**
372   * Generate chatbot response
373   */
374  public String generateChatbotResponse(String userMessage, String context) {
375      String prompt = String.format(
376          "Bạn là trợ lý bán hàng thông minh của cửa hàng điện tử. " +
377          "Trả lời câu hỏi của khách hàng một cách thân thiện và hữu ích. " +
378          "Context: %s. " +
379          "Câu hỏi: %s",
380          context, userMessage
381      );
382
383      return generateText(prompt);
384  }
385
```

Hình 3.14: Đoạn mã xử lý tại lớp GeminiService.java gọi Google Gemini API

3. Giao diện minh chứng (UI)

Trên giao diện người dùng, tính năng này được hiện thực hóa qua cửa sổ Chatbot (Floating Bubble). Chatbot có khả năng trả lời các câu hỏi về thông số kỹ thuật, so sánh sản phẩm dựa trên dữ liệu đã được học.



Hình 3.15: Giao diện Chatbot tư vấn sản phẩm sử dụng Gemini AI

3.3.6.1.2 Hệ thống gợi ý sản phẩm (Recommendation System)

1. Công nghệ và Thư viện sử dụng

Phân hệ gợi ý sản phẩm được xây dựng tách biệt dưới dạng một Microservice sử dụng Python (FastAPI), tận dụng sức mạnh của các thư viện khoa học dữ liệu:

- **scikit-learn & joblib:** Sử dụng để tải (load) các mô hình đã huấn luyện (.pkl, .joblib) và mã hóa dữ liệu (Label Encoding) cho UserID và ProductID.
- **numpy:** Thực hiện các tính toán ma trận số học hiệu năng cao trong quá trình suy luận (inference).

- **FastAPI & Uvicorn:** Cung cấp RESTful API để Java Backend có thể giao tiếp và lấy danh sách ID sản phẩm gợi ý.

```
1  # Python dependencies for Recommendation Service
2  fastapi==0.115.0
3  uvicorn[standard]==0.32.0
4  pydantic==2.9.0
5  joblib==1.4.2
6  requests==2.32.3
7  # ML dependencies for model inference
8  # Model uses implicit library (AlternatingLeastSquares)
9  implicit>=0.7.0
10 scikit-learn>=1.0.0,<2.0.0
11 numpy<2.0.0
12 python-multipart==0.0.9
13
14 | Ctrl+L to chat, Ctrl+K to generate
```

Hình 3.16: Các thư viện Machine Learning được sử dụng trong Python Service

2. Logic xử lý (Code Implementation)

Quy trình xử lý gợi ý sản phẩm diễn ra như sau:

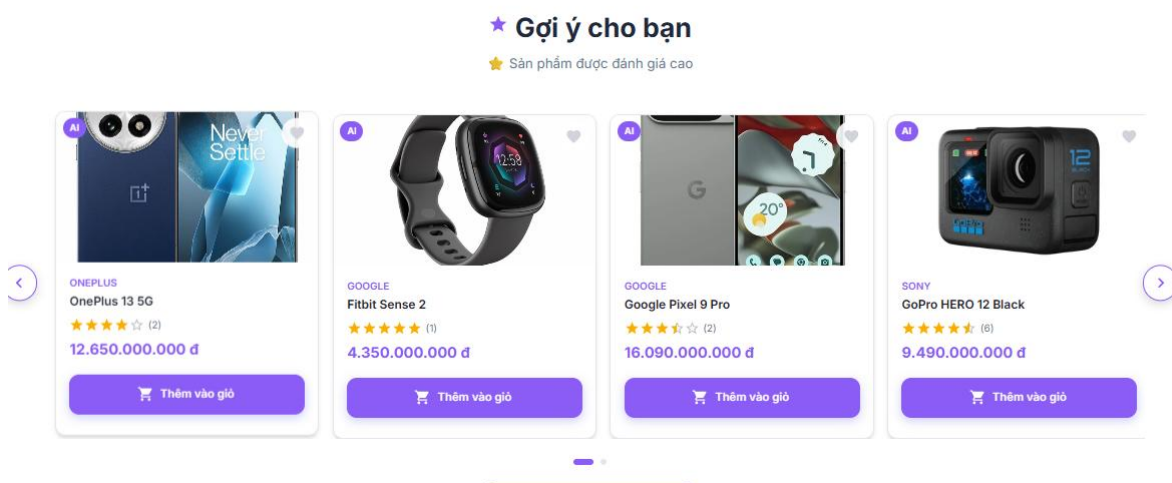
- Nhận Request: FastAPI nhận user_id từ Java Backend.
- Chuẩn bị dữ liệu: Sử dụng LabelEncoder (đã load từ file joblib) để chuyển đổi user_id thực tế sang ID nội bộ mà mô hình hiểu được.
- Suy luận (Inference): Mô hình (Collaborative Filtering) tính toán và dự đoán điểm đánh giá (rating) của người dùng này cho các sản phẩm chưa mua.
- Sắp xếp & Trả kết quả: Lọc ra Top N sản phẩm có điểm dự đoán cao nhất và trả về danh sách Product IDs.

```
elif hasattr(model, "predict"):\n    try:\n        if encoders_loaded and user_encoder is not None and item_encoder is not None:\n            try:\n                # COLLABORATIVE FILTERING: Convert string IDs to internal IDs using encoders\n                user_internal = user_encoder.transform([user_id])[0]\n                item_internal = item_encoder.transform([product_id])[0]\n\n                # AI Prediction: SVD model phân tích patterns và predict rating\n                pred = model.predict(user_internal, item_internal)\n                score = float(getattr(pred, "est", 0.0))\n\n                # Ensure score is in valid range (1-5)\n                score = max(1.0, min(5.0, score))\n\n            if score > 0:\n                return score, True\n        except (ValueError, KeyError, IndexError):\n            pass\n        except Exception as e:\n            import logging\n            logging.getLogger(__name__).debug(f"Surprise prediction error: {e}")\n    except Exception:\n        pass\nexcept Exception:\n    pass
```

Hình 3.17: Logic xử lý gợi ý sản phẩm tại Python Service

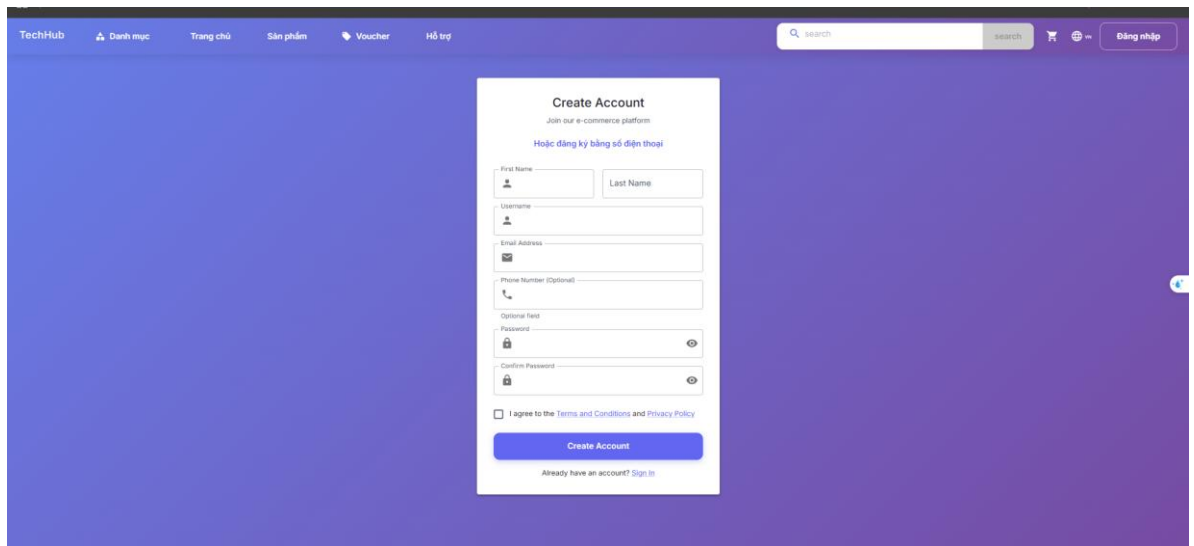
3. Giao diện minh chứng (UI)

Kết quả của thuật toán được hiển thị tại trang chủ hoặc trang chi tiết sản phẩm dưới mục "Có thể bạn sẽ thích" hoặc "Sản phẩm gợi ý".



Hình 3.18: Giao diện hiển thị danh sách sản phẩm gợi ý dựa trên AI

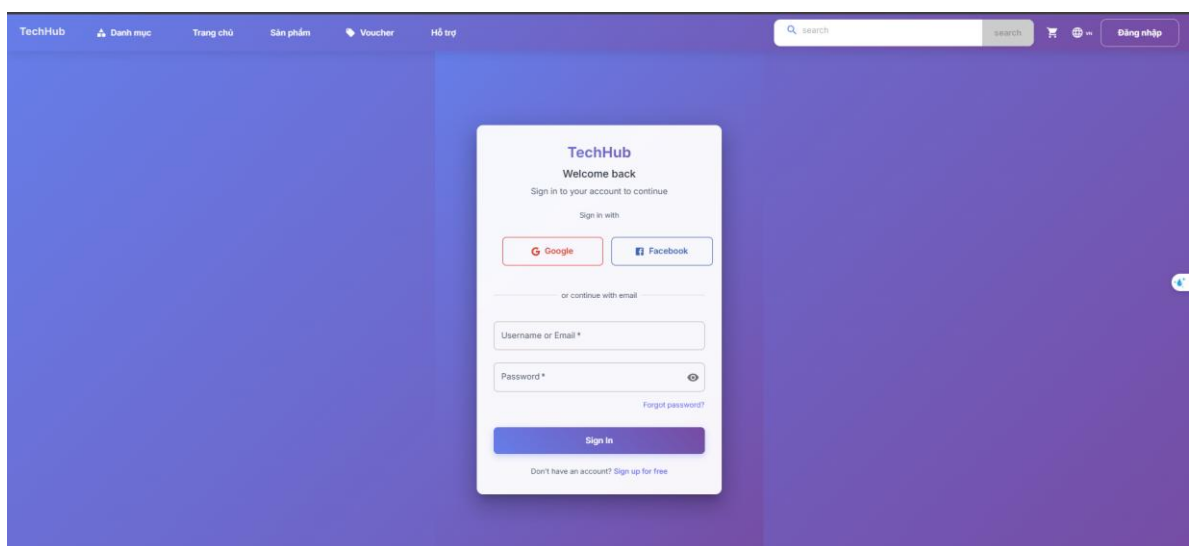
3.3.6.2 Các chức năng dành cho khách hàng.



Hình 3.19. Màn hình đăng nhập của hệ thống.

- Người dùng nhập thông tin như : Họ và Tên, Email và Password để đăng kí tài khoản.
- Sau khi đăng ký xong hệ thống tự động đăng nhập vào hệ thống.

Màn hình đăng nhập:

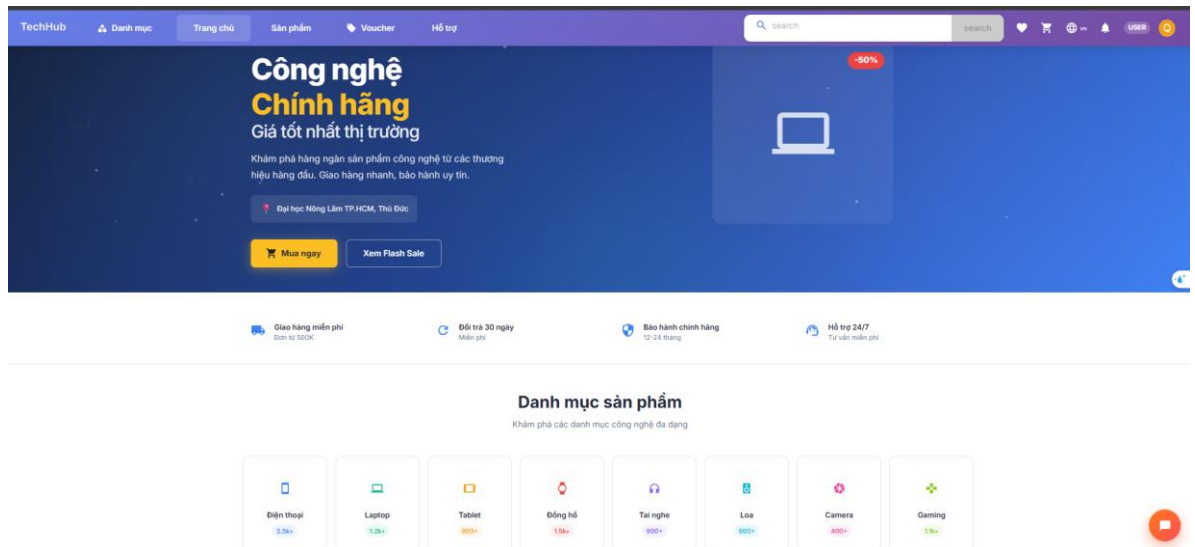


Hình 3.20. Màn hình đăng ký.

- Người dùng sẽ nhập Email và Password để đăng nhập vào hệ thống.

GIẢI PHÁP CHO BÀI TOÁN

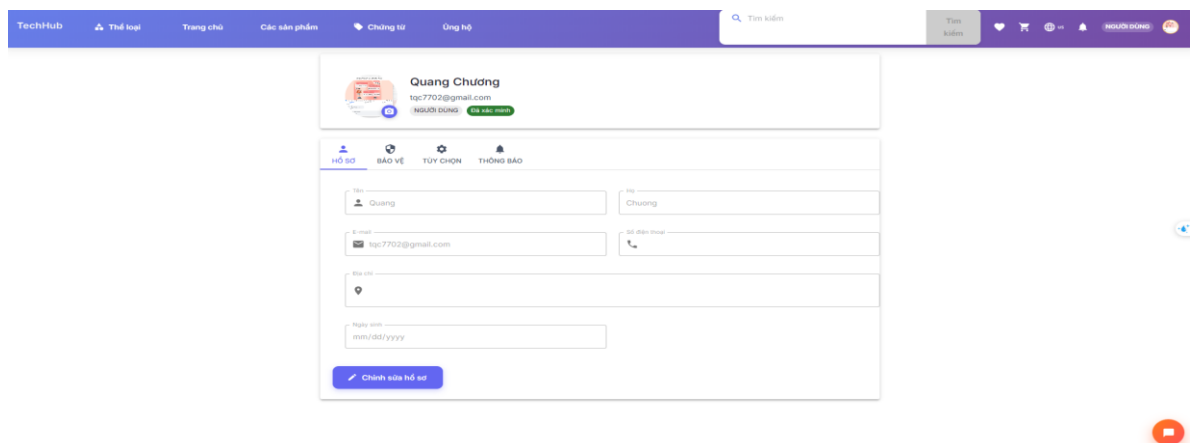
- Nếu đăng nhập thành công hệ thống sẽ điều hướng sang trang chủ.



Hình 3.21. Màn hình trang chủ

Trang chủ gồm có:

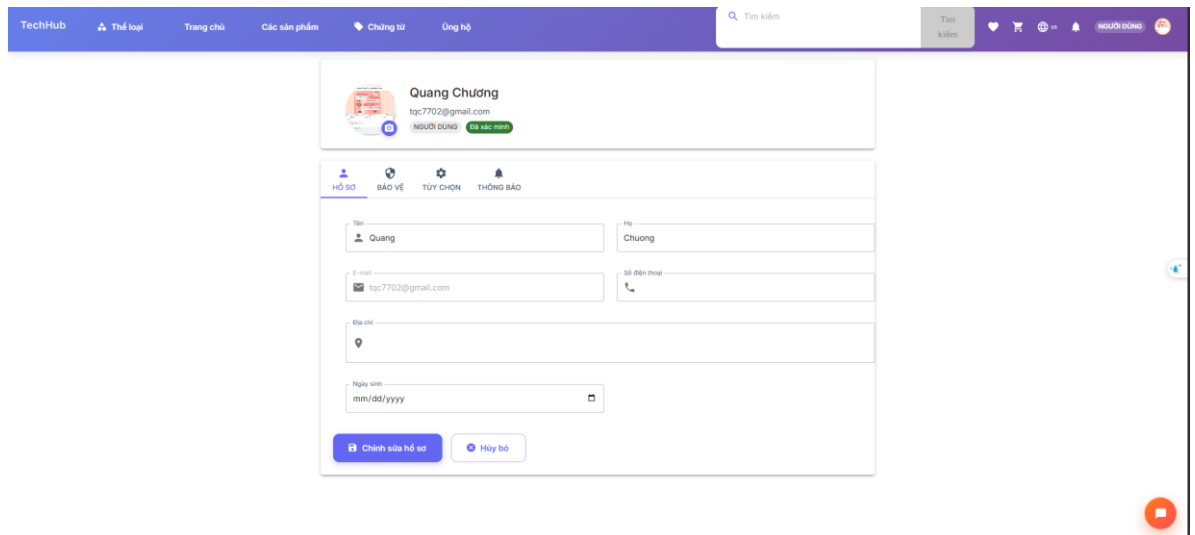
- Thanh tìm kiếm: giúp người dùng nhanh chóng tìm sản phẩm (tên/ID/SKU) hoặc truy cập trực tiếp trang chi tiết khi biết mã sản phẩm.
- Danh sách đề xuất/công khai: hiển thị các sản phẩm nổi bật/khuyến mãi/bán chạy để người dùng truy cập nhanh.
- Header: gồm nút tài khoản (đăng nhập/đăng ký, hồ sơ, đơn hàng) và nút tạo/gửi yêu cầu (ví dụ: yêu cầu tư vấn, báo giá), kèm các nút giỏ hàng và ngôn ngữ nếu cần.



Hình 3.22. Màn hình chức năng đổi thông tin cá nhân.

GIẢI PHÁP CHO BÀI TOÁN

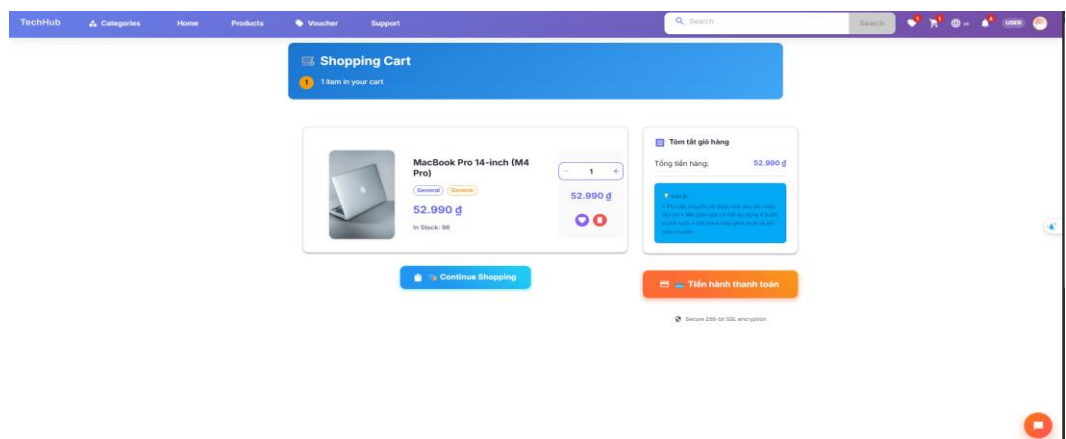
- Nhấn vào Tài khoản để mở ngăn kéo thông tin cá nhân. Tại đây người dùng có thể cập nhật thông tin cá nhân như Email, Họ và tên, xem các cuộc bình chọn, đổi mật khẩu.
- Khi chọn đổi mật khẩu ngăn kéo đổi mật khẩu được mở ra.



Hình 3.23. Màn hình chức năng đổi mật khẩu.

- Người dùng cần nhập mật khẩu cũ và mật khẩu mới để thực hiện đổi mật khẩu.

Màn hình chi tiết giỏ hàng:



Hình 3.24. Màn hình trang chi tiết giỏ hàng.

Cho phép người dùng xem đầy đủ thông tin các sản phẩm trong giỏ hàng, bao gồm ảnh, tên sản phẩm, mô tả ngắn, đơn giá, số lượng, thành tiền và mã sản phẩm/ID. Người dùng có thể thực hiện các chức năng sau:

GIẢI PHÁP CHO BÀI TOÁN

- Điều chỉnh giỏ hàng: tăng/giảm số lượng, xóa sản phẩm khỏi giỏ, chọn/bỏ chọn sản phẩm để thanh toán.
- Xem tổng quan chi phí: hiển thị tổng tiền hàng, phí vận chuyển tạm tính (nếu có), giảm giá (voucher/mã khuyến mãi nếu hệ thống hỗ trợ) và tổng thanh toán dự kiến.

Checkout

1 Shipping Address

2 Payment & Voucher

Địa chỉ giao hàng

Họ *

Quang

Tên *

Chuong

Email *

tqc7702@gmail.com

Số điện thoại *

Địa chỉ chi tiết *

CVCVCV

Tỉnh/Thành phố **

Hòa Bình

Quận/Huyện **

Huyện Mai ...

Xã/Phường **

Xã Đồng Tân

Ghi chú (tùy chọn)

CVC

Lưu địa chỉ này

✓ ☒ Phí vận chuyển: 44.000 đ

← Quay lại

Tiếp tục →

Order Summary

1 item(s)

Subtotal:

52.990 đ

Shipping:

44.000 đ

Tax:

5.299 đ

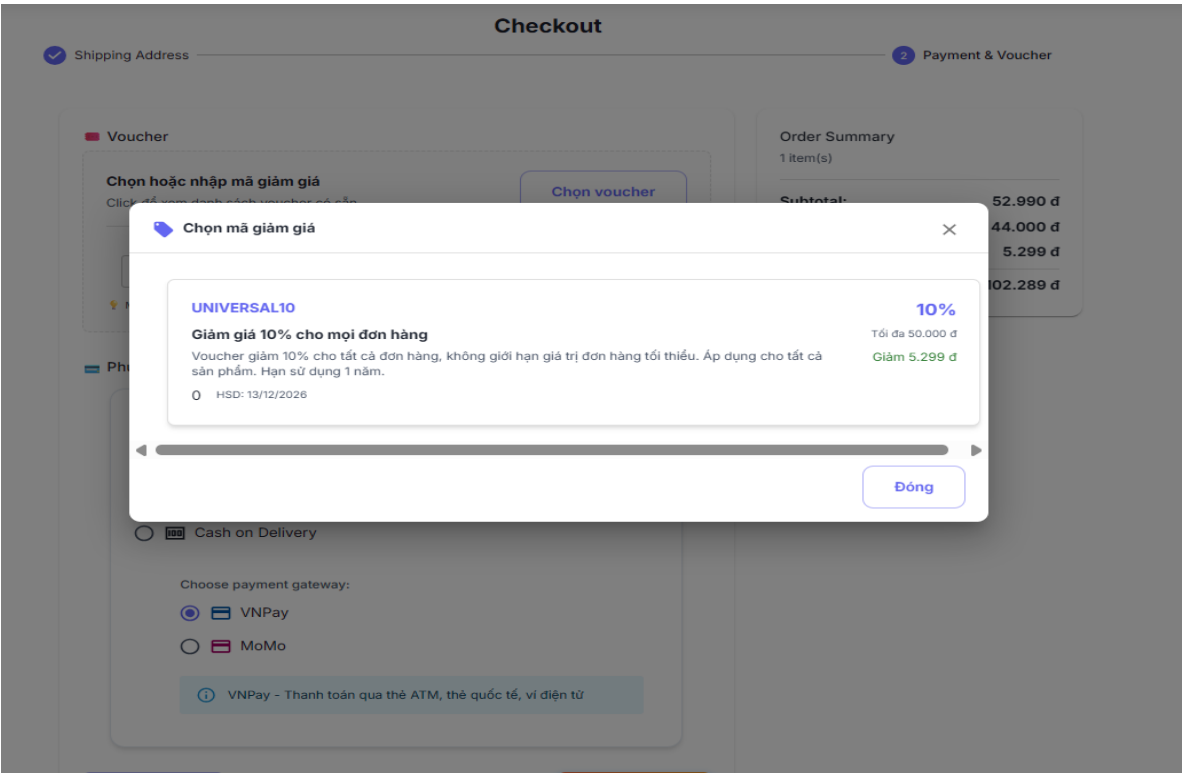
Total:

102.289 đ

Hình 3.25. Giao diện của trang khi điền thông tin để ra giá tiền giao hàng.

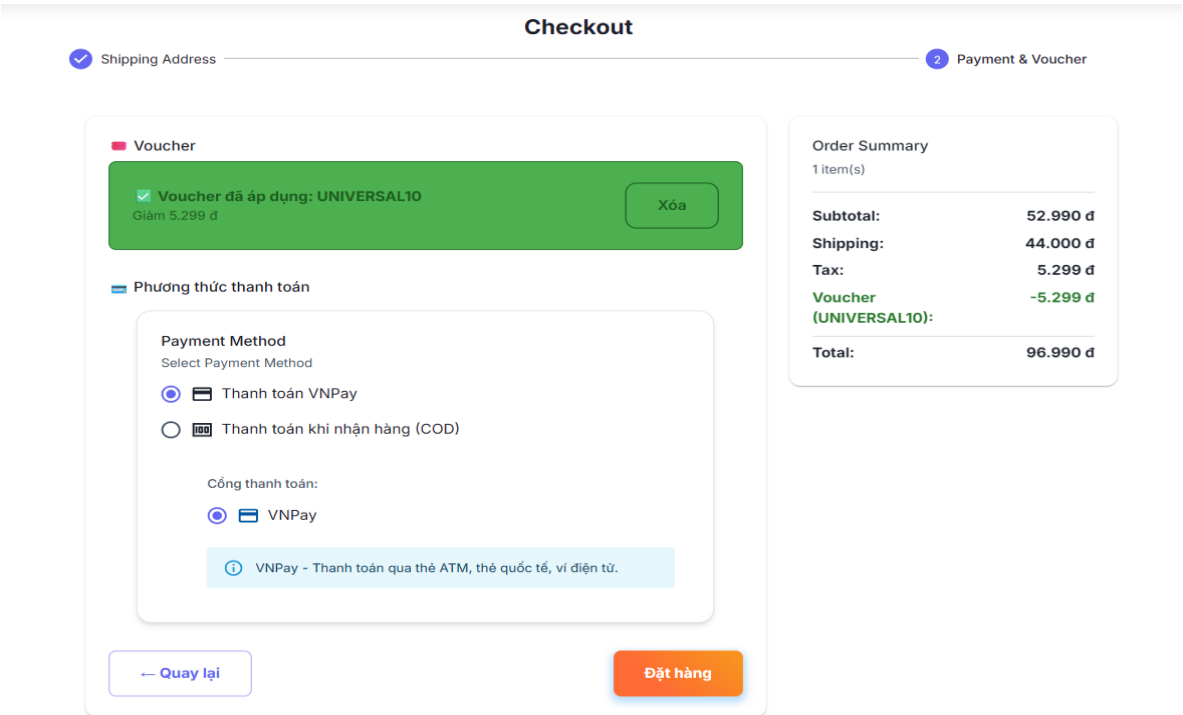
Khi nhấn điền thông tin các tỉnh/huyện để giao hàng thì hệ thống tính toán ra giá tiền giao hàng.

GIẢI PHÁP CHO BÀI TOÁN



Hình 3.26. Chọn voucher phù hợp với đơn hàng.

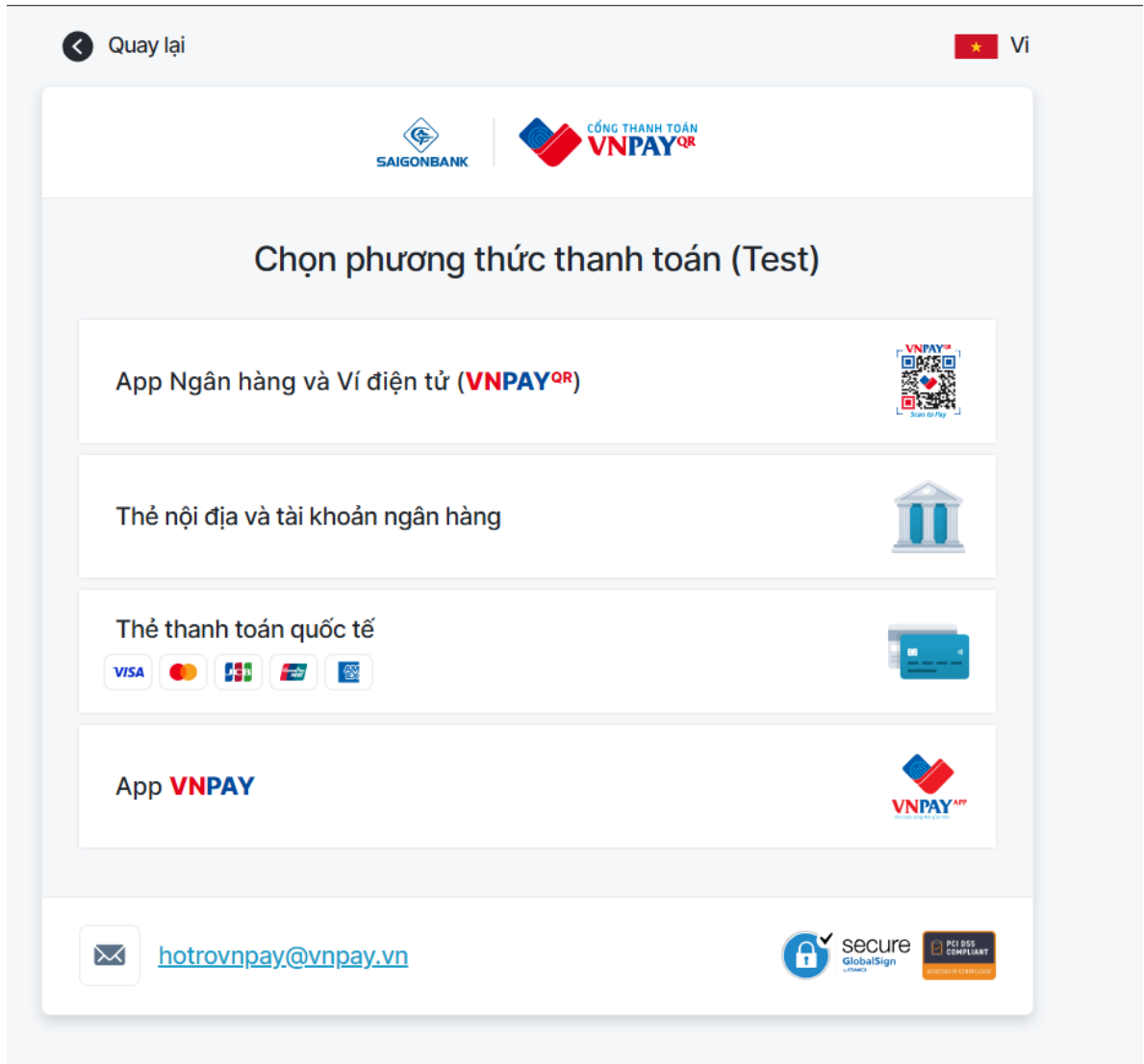
- Người dùng có thể chọn các phương thức thanh toán phù hợp với cá nhân.



Hình 3.27. Hình các phương thức thanh toán.

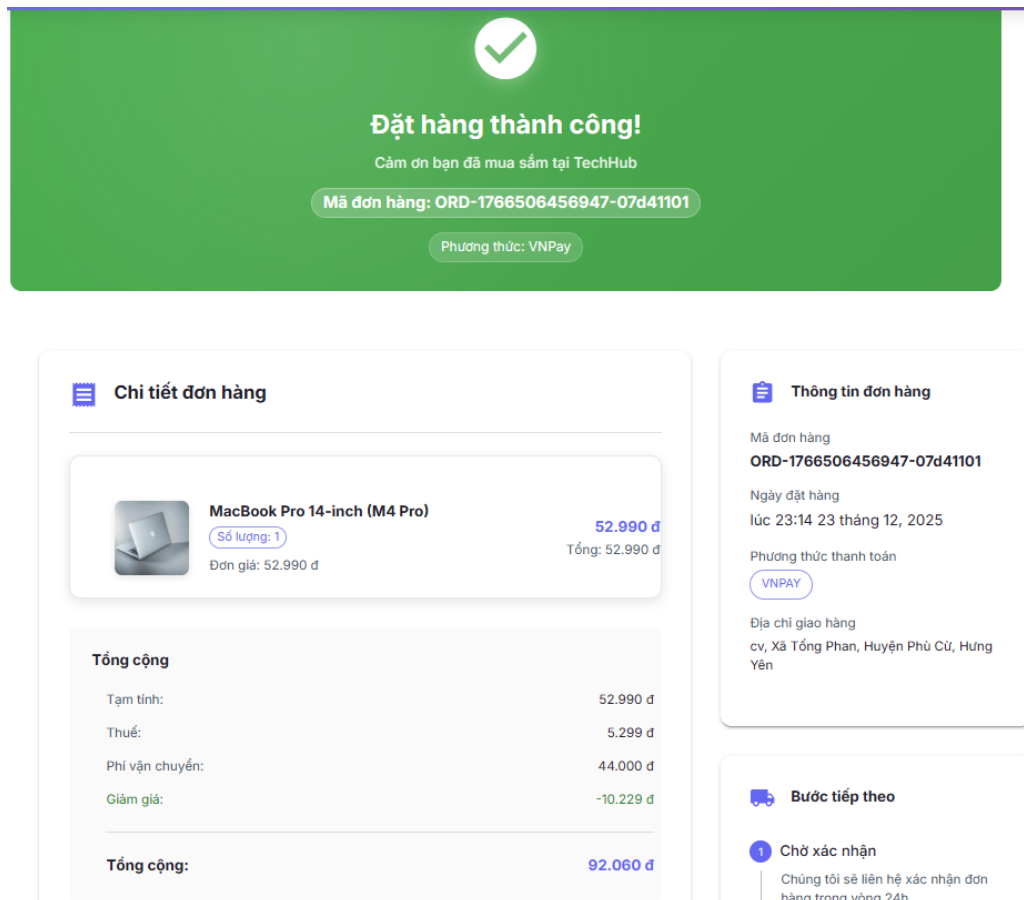
GIẢI PHÁP CHO BÀI TOÁN

- Khi nhấn “đặt hàng”, thì hệ thống sẽ tiến hành bước thanh toán theo phương thức thanh toán mà khách hàng chọn.



Hình 3.28. Khi khách hàng mà chọn vnipay.

- Nếu khách hàng thanh toán hoàn toán bằng tài khoản vnipay và đáp ứng đủ các giá tiền thành công thì hệ thống sẽ thông báo là đặt hàng thành công.



Hình 3.29. Màn hình trang chi tiết đặt hàng thành công.

- Chủ sở hữu nhấn vào “xem chi tiết đơn hàng” để xem lại đúng thông tin đơn hàng mình chọn chưa.

Order Details

Order ID
#332

Status

Processing

Paid

Order Breakdown

Subtotal:	52.990 đ
Tax:	5.299 đ
Shipping:	44.000 đ
Discount:	-10.229 đ
Total:	92.060 đ

Items (1)



MacBook Pro 14-inch (M4 Pro)

Qty: 1 52.990 đ each

Total: 52.990 đ

Ordered: lúc 23:14 23 tháng 12, 2025

Updated: lúc 23:14 23 tháng 12, 2025

✕ Hủy đơn hàng

Hình 3.30.Hình lịch sử đơn hàng.

Màn hình admin duyệt đơn hàng:

GIẢI PHÁP CHO BÀI TOÁN

Order Management
Manage customer orders, track shipments, and update order statuses

415 Total Orders
1 Pending Orders
2 Shipped Orders
532.304 d Total Revenue

Order List

Order ID	Order #	Customer	Status	Total	Created	Actions
#332	ORD-1766506456947-07	Order #ORD-1766506456947-07d41101	PROCESSING	92.060 d	23/12/2025	
#331	ORD-1765963844489-d	Order #ORD-1765963844489-d73194c	PENDING	30.576 d	17/12/2025	
#330	ORD-1765958722685-68	Order #ORD-1765958722685-68b675e7	SHIPPED	32.105 d	17/12/2025	
#328	ORD-1765957776849-41	Order #ORD-1765957776849-414232c2	CANCELLED	30.393 d	17/12/2025	
#327	ORD-1765803332805-e1	Order #ORD-1765803332805-e1b980af	SHIPPED	153.834 d	13/12/2025	
#325	ORD-1765534960864-21	Order #ORD-1765534960864-21a29402	CANCELLED	38.814 d	12/12/2025	
#123	ORD-1765087363180-71	Order #ORD-1765087363180-71d9658f6	CANCELLED	36.512 d	7/12/2025	
#122	ORD-1764848187568-71	Order #ORD-1764848187568-71d0d8c5	CANCELLED	39.882 d	4/12/2025	
#120	ORD-1764843740633-81	Order #ORD-1764843740633-81b76787	CANCELLED	39.064 d	4/12/2025	
#119	ORD-1764843740633-81	Order #ORD-1764843740633-81b76787	CANCELLED	39.064 d	4/12/2025	

Rows per page: 10 1-10 of 415

Hình 3.31. Màn hình admin quản lý các đơn hàng được tạo.

- Cho phép admin duyệt đơn , kiểm soát các đơn hàng.
- Có thể hủy nhận đơn.

Update Order Status

Order Details

Order #ORD-1766506456947-07d41101 -
Current Status: PROCESSING

New Status

Processing

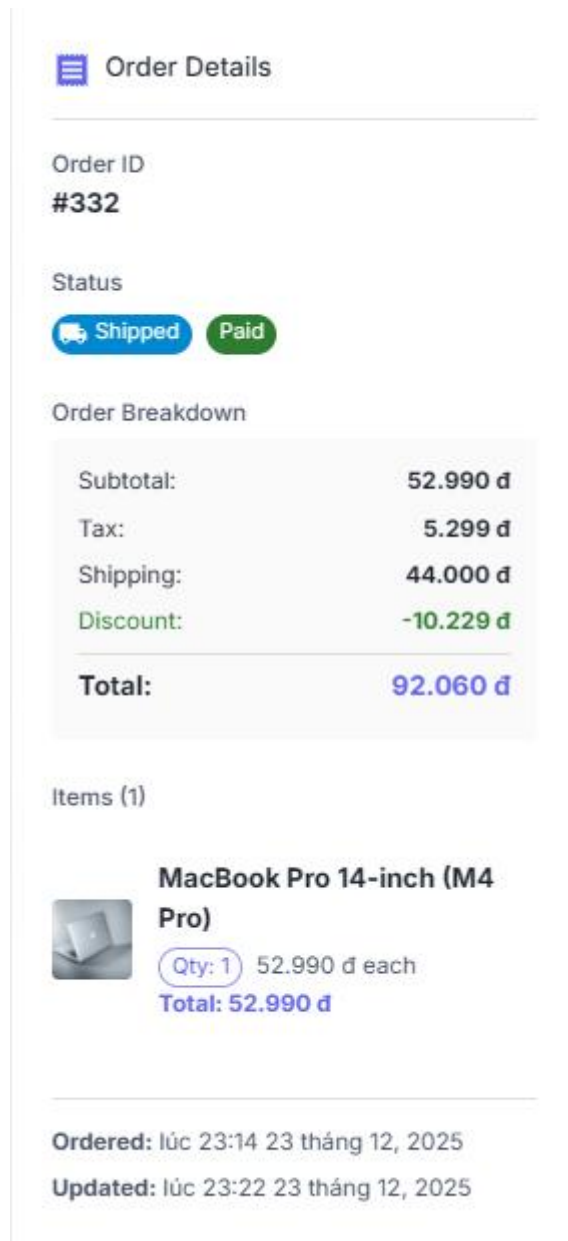
- Pending
- Pending Approval
- Approved
- Confirmed
- Processing
- Shipped
- Delivered
- Completed
- Cancelled
- Refunded

Hình 3.32. Màn hình chỉnh sửa trạng thái đơn.

GIẢI PHÁP CHO BÀI TOÁN

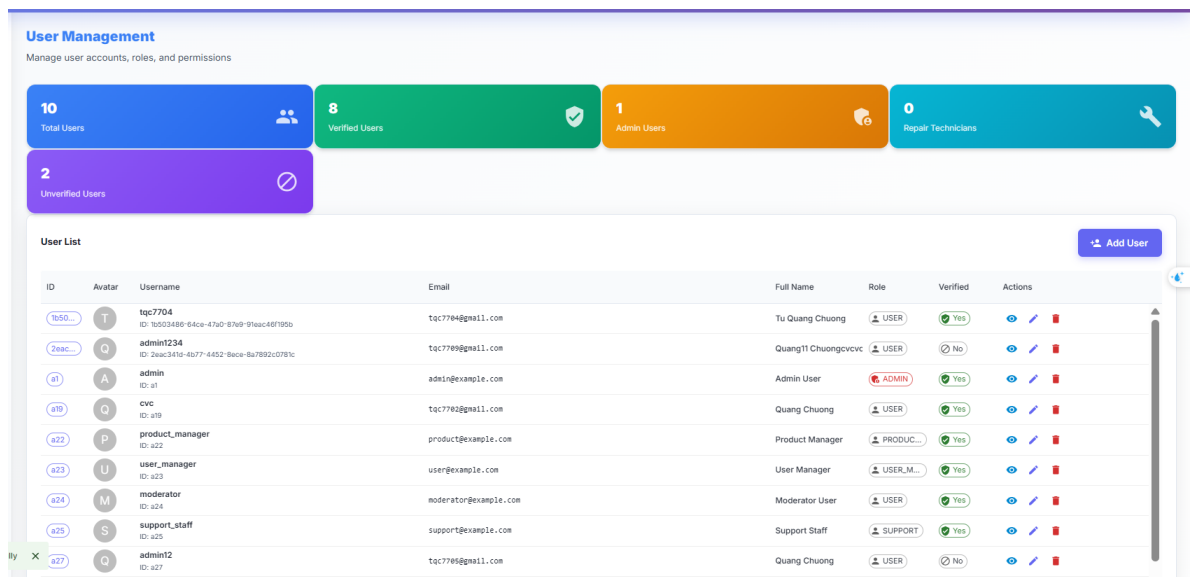
- Trạng thái đơn hàng phía người dùng được cập nhật ngay lập tức theo thời gian thực.
- User biết ngay trạng thái đơn hàng của mình như thế nào để tiện theo dõi đơn hàng.

Phía User:



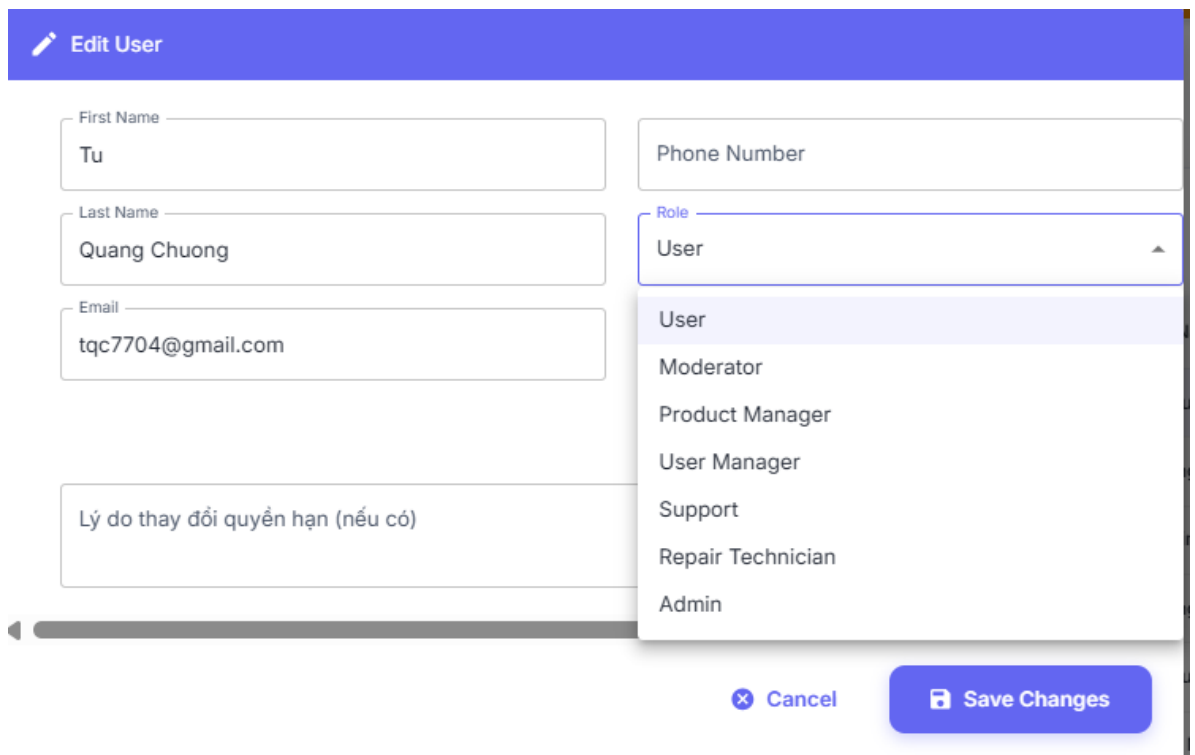
Hình 3.33. Trạng thái đơn hàng đã được update

GIẢI PHÁP CHO BÀI TOÁN



Hình 3.34. Màn hình danh sách người dùng của hệ thống.

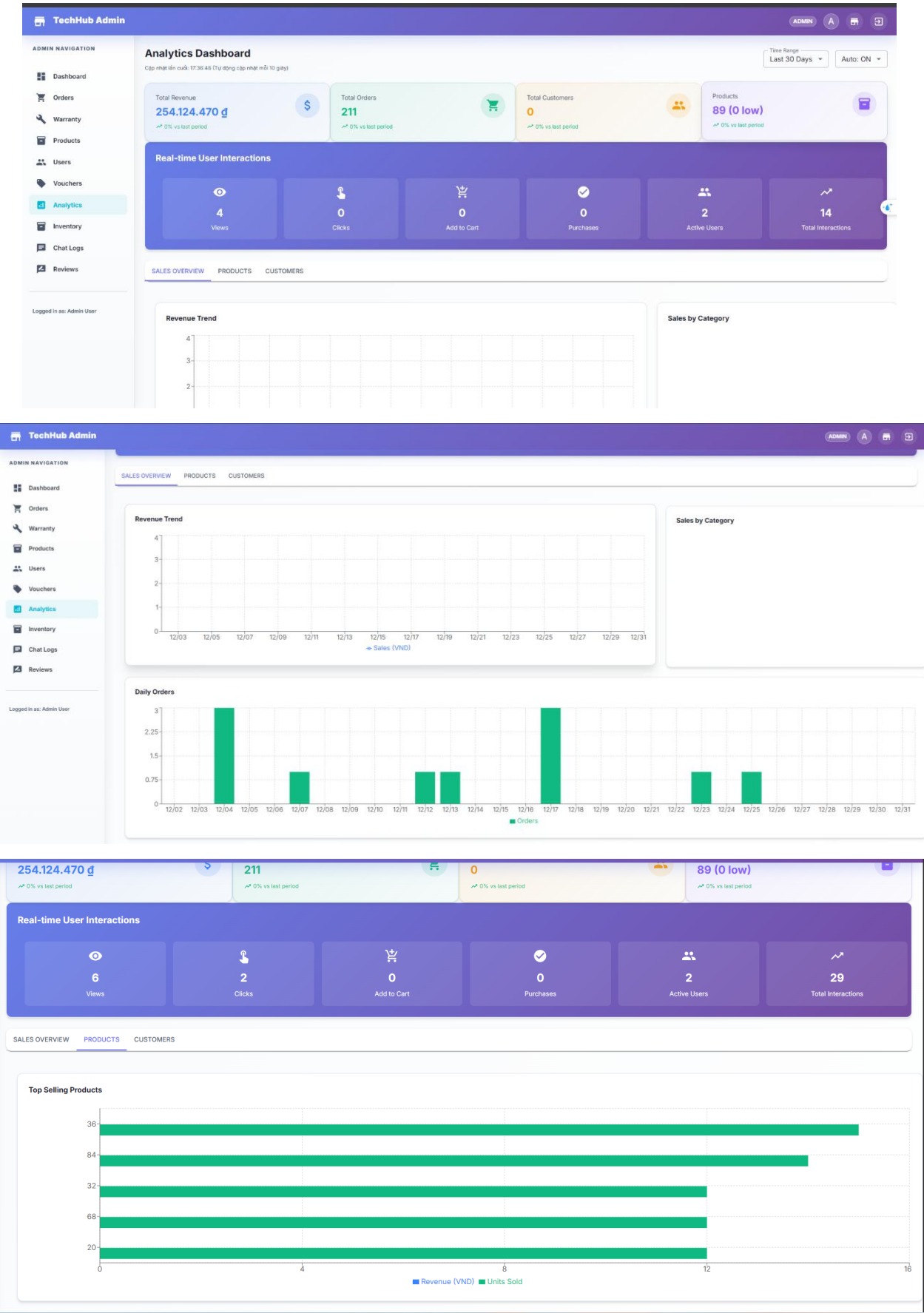
- Tại đây admin có thể hạ quyền hoặc nâng quyền cho các account mà mình muốn.



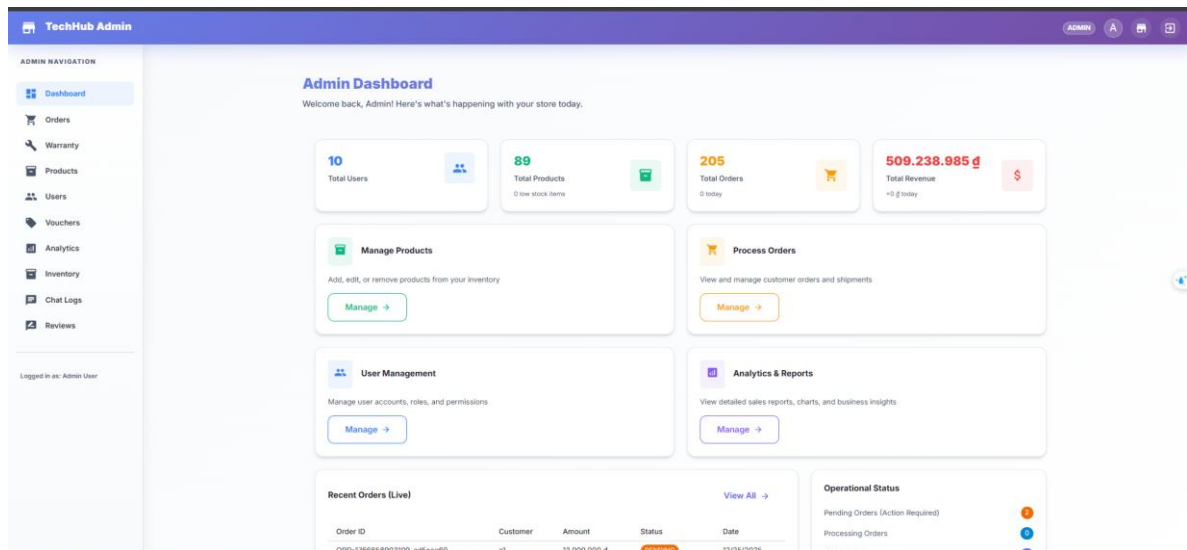
Hình 3.35. Màn hình phân quyền.

- Trang admin quản lý thống kê toàn bộ các chỉ số của web

GIẢI PHÁP CHO BÀI TOÁN

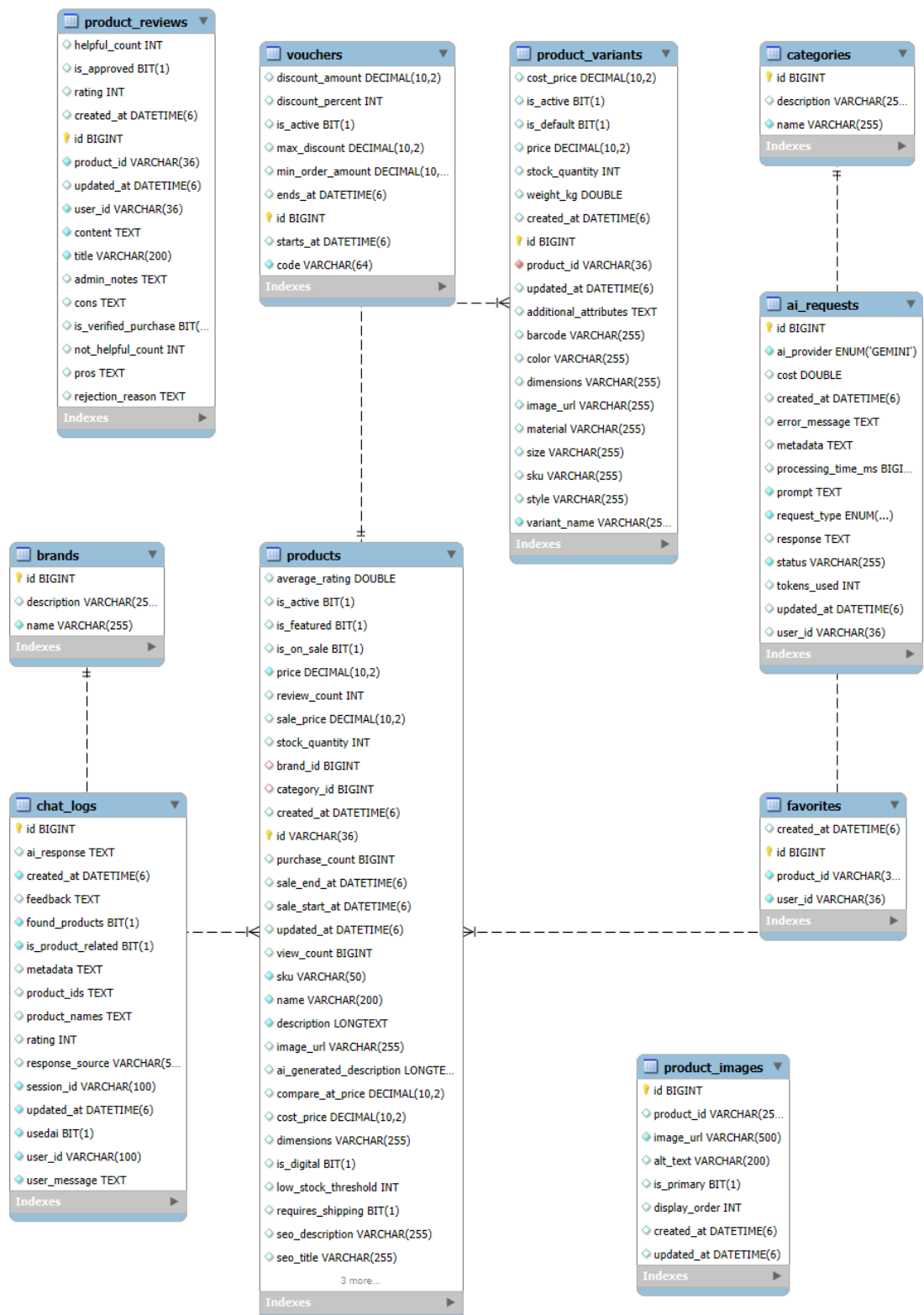


Hình 3.36. Màn hình trang analytics mà admin quản lý



Hình 3.37. Màn hình thống kê của admin

3.3.7. Lược đồ cơ sở dữ liệu



Hình 3.38. Lược đồ cơ sở dữ liệu của hệ thống products.

Phân tích các bảng trong cơ sở dữ liệu:

- *products*: Bảng lưu trữ thông tin sản phẩm chính của hệ thống, gồm tên, mô tả, giá, giá khuyến mãi, tồn kho tổng, cờ đang giảm giá/đang bán, điểm đánh giá trung bình, số lượt xem, SKU, các thông tin SEO và cấu hình khác.
- *product_variants*: Bảng lưu trữ các biến thể của sản phẩm (size, màu, kiểu dáng...), mỗi dòng gắn với một *product_id*, có giá riêng, tồn kho riêng, SKU riêng, thuộc tính bổ sung (màu, kích thước, trọng lượng, chất liệu...).
- *product_images*: Bảng lưu trữ danh sách hình ảnh của sản phẩm, mỗi ảnh gắn với một *product_id*, gồm đường dẫn ảnh, text thay thế, cờ ảnh chính (*is_primary*), thứ tự hiển thị, thời gian tạo/cập nhật.
- *product_reviews*: Bảng lưu trữ các đánh giá của người dùng cho sản phẩm, gồm *user_id*, *product_id*, số sao (*rating*), tiêu đề, nội dung đánh giá, ưu/nhược điểm, cờ đã duyệt, đếm số lượt đánh giá hữu ích, lý do từ chối, v.v.
- *vouchers*: Bảng lưu trữ thông tin mã giảm giá của hệ thống, gồm code, phần trăm/giá trị giảm, giá trị giảm tối đa, điều kiện đơn hàng tối thiểu, thời gian bắt đầu/kết thúc, trạng thái đang hoạt động hay không.
- *categories*: Bảng lưu trữ danh mục sản phẩm, gồm tên và mô tả danh mục; dùng để phân loại sản phẩm và liên kết với bảng *products*.
- *Brands*: Bảng lưu trữ thông tin thương hiệu, gồm tên và mô tả thương hiệu; được tham chiếu bởi bảng *products* để gắn mỗi sản phẩm với một brand cụ thể
- *Favorites*: Bảng lưu trữ danh sách sản phẩm yêu thích (*wishlist*) của người dùng, mỗi dòng chứa *user_id* và *product_id*, cùng thời gian tạo; cho phép người dùng lưu lại sản phẩm quan tâm
- *Chat_logs*: Bảng lưu trữ lịch sử hội thoại của chatbot với người dùng, gồm câu hỏi của user, phản hồi AI, feedback, metadata, cờ hội thoại liên quan sản phẩm, danh sách *product_ids*/tên sản phẩm mà AI đã gợi ý, điểm *rating*, v.v.

- Ai_requests: Bảng lưu trữ log các request gửi tới dịch vụ AI (ví dụ Gemini), gồm nhà cung cấp AI, prompt, loại request, chi phí, thời gian xử lý, số token, metadata, trạng thái (thành công/thất bại) và user_id liên quan.

3.3.8. Xây dựng các endpoint API

Các endpoint API của hệ thống:

+ Xác thực người dùng:

api/auth	/register	Đăng ký tài khoản mới
	/login	Đăng nhập vào hệ thống
	/logout	Đăng xuất khỏi hệ thống
	/changePassword	Đổi mật khẩu người dùng

+ Quản lý các sản phẩm:

api/products	/ {id} /variants, product-variants/	Biến thể của sản phẩm
	/:id	Danh sách, chi tiết từng sản phẩm
	/categories, /brands	Danh mục , thương hiệu
	/ {id} /reviews	Đánh giá sản phẩm

+ Quản lý giỏ hàng:

api/cart	/ {userId}	Lấy giỏ hàng
	/ {userId} /add	Thêm sản phẩm vào giỏ hàng

GIẢI PHÁP CHO BÀI TOÁN

	/guest/add?sessionId	Giỏ hàng guest (chưa login).
	{userId}/count, /empty, /health	Đếm item, kiểm tra trống, health.
	{userId}/remove/{productId}	Xóa item / xóa giỏ.
	/ {userId}/update	Cập nhập số lượng

+ Quản lý đơn hàng:

api/orders	/orders	Danh sách đơn (phân trang, filter theo status).
	{id}, /number/{orderNumber}, /my-orders?userId	Chi tiết / đơn của user
	/user/{userId}/purchased-products	Sản phẩm user đã mua (check review).
	/ {orderId}/confirm	Admin xác nhận đơn hàng
	/confirm-payment	Backend cập nhật đơn sau khi thanh toán thành công.
	/by-number/{orderNumber}/cancel, / {orderId}/cancel	Hủy đơn (kể cả hủy khi thanh toán lỗi).

+ Tương tác với gợi ý sản phẩm:

api/recommendations	/behavior	Ghi lại hành vi người dùng (view, add_to_cart, purchase, favorite...).
	/personalized/{userId}?limit=...:	Gợi ý cá nhân hóa cho user.
	/similar/{productId}?limit=...:	Gợi ý sản phẩm tương tự.

3.3.9. Cơ chế bảo mật JWT

Xác thực người dùng:

- Khi người dùng đăng nhập vào hệ thống, hệ thống sẽ xác thực thông tin đăng nhập (email và mật khẩu).
- Nếu thông tin đăng nhập hợp lệ, máy chủ sẽ tạo ra một JWT (JSON Web Token) bao gồm thông tin nhận dạng của người dùng là id và mã hóa nó bằng một khóa bí mật. Token này sẽ được ghi vào cookies.

Phân quyền và quản lý phiên đăng nhập:

- JWT được gửi từ máy chủ đến người dùng qua HTTP cookie. Mỗi khi người dùng gửi yêu cầu API đến máy chủ, JWT sẽ được gửi kèm theo để xác thực.
- Máy chủ nhận JWT, giải mã nó bằng khóa bí mật, và kiểm tra tính hợp lệ của token.
- Nếu JWT hợp lệ, máy chủ sẽ cho phép người dùng thực hiện các yêu cầu đã được phân quyền dựa trên thông tin trong JWT (ví dụ: quyền truy cập vào các route hay dịch vụ cụ thể).

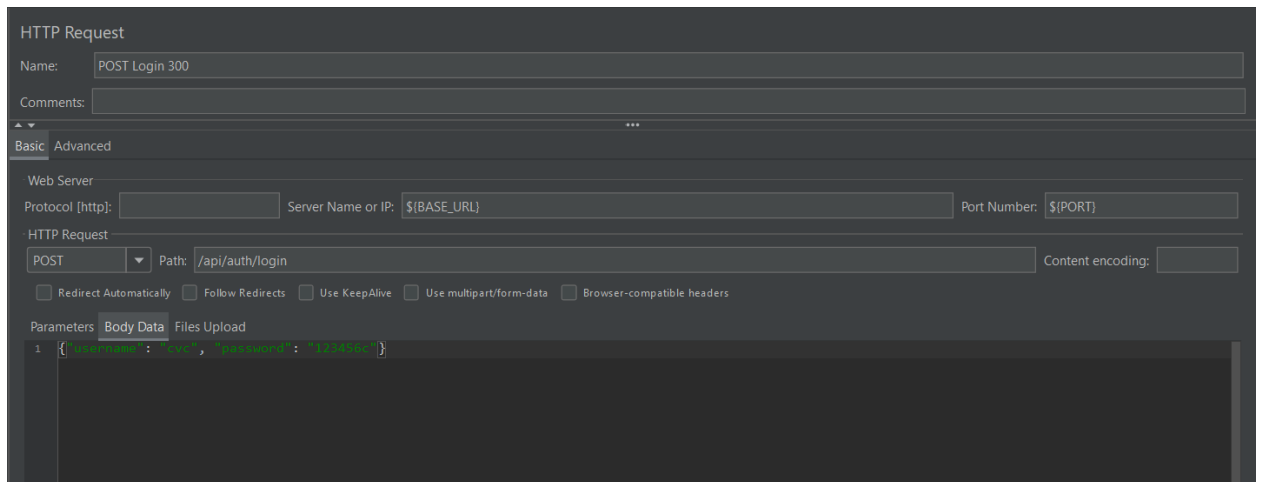
3.3.10. Đánh giá hiệu suất hệ thống

Sau khi hoàn thành việc xây dựng hệ thống, em tiến hành thực hiện kiểm thử tải (Load Test) và kiểm thử chịu tải cao (Stress Test) nhằm đánh giá khả năng đáp ứng và mức độ ổn định của hệ thống trong các tình huống có nhiều người dùng truy cập và gửi yêu cầu

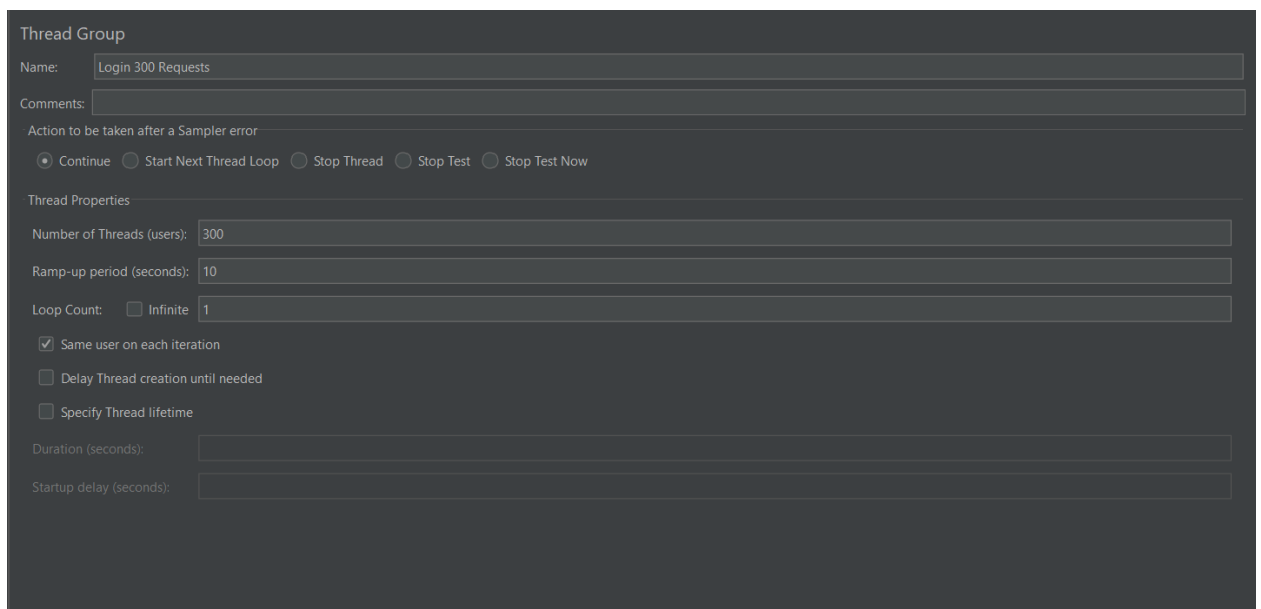
GIẢI PHÁP CHO BÀI TOÁN

đồng thời, cũng như trường hợp một lượng lớn request được gửi liên tục trong thời gian dài.

Để đánh giá khả năng xử lý nhiều yêu cầu đồng thời, em đã sử dụng công cụ Apache JMeter để mô phỏng các kịch bản kiểm thử tại các mức 300, 500 và 1000 request cùng lúc.



Hình 3.39: Giao diện phần dữ liệu request test....



Hình 3.40: Giao diện phần cài đặt số luồng để test liên tục

GIẢI PHÁP CHO BÀI TOÁN

Test Plan

Name:

Add to Cart Performance Test

Comments:

User Defined Variables

Name:	Value
BASE_URL	localhost
PORT	8080
PRODUCT_ID	898b74c1-db41-11f0-b295-bbe859061041

Detail

Add

Add from Clipboard

Delete

Up

Down

☒ Run Thread Groups consecutively (i.e. one at a time)

☐ Run tearDown Thread Groups after shutdown of main threads

☐ Functional Test Mode (i.e. save Response Data and Sampler Data)

Selecting Functional Test Mode may adversely affect performance.

Add directory or jar to classpath

Browse...

Delete

Clear

Library

Hình 4.41 : Giao diện phân cấu hình

Label	Samples	Avg	Min	Max	Std. Dev.	Error	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
POST Login 300	300	4864	642	8820	1547.7	0.00%	22.17787	31.37	4.76	1448.4
POST Login 500	500	6157	552	10087	2093.82	0.00%	29.87572	42.3	6.42	1449.7
POST Login 1000	1000	10839	553	17663	4007.41	0.00%	32.45805	45.92	6.97	1448.7
TOTAL	1800	8543	552	17663	4159.92	0.00%	29.47003	41.7	6.33	1449

Bảng 4.1 : Kiểm thử Stress test – chức năng test login từ request

Label	Samples	Avg	Min	Max	Std. Dev.	Error	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
GET Search 300	300	51	15	584	72.48	0.00%	30.39514	1242.79	4.1	41869

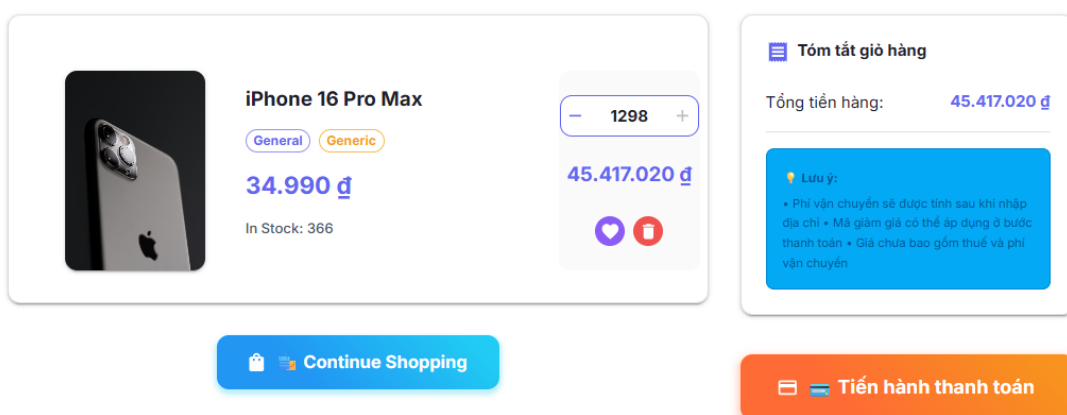
GIẢI PHÁP CHO BÀI TOÁN

GET Search 500	500	27	19	69	4.79	0.00%	49.985	2043.66	6.69	41866.7
GET Search 1000	1000	32	21	160	11.38	0.00%	66.62225	2723.93	9.17	41867.4
TOTAL	1800	34	15	584	31.87	0.00%	51.59663	2109.59	7.02	41867.5

Bảng 4.2: Kiểm thử Load test – chức năng test search products request

Label	Samples	Avg	Min	Max	Std. Dev.	Error	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
POST Add to Cart 300	300	468	50	2934	718.06	0.00%	30.16288	30.41	13.52	1032.3
POST Add to Cart 500	500	62	46	331	35.48	0.00%	49.83058	50.47	22.34	1037.2
POST Add to Cart 1000	1000	55	43	107	8.73	0.00%	66.25588	67.21	29.7	1038.7
TOTAL	1800	126	43	2934	331.11	0.00%	51.04297	51.71	22.87	1037.4

Bảng 4.3 Kiểm thử Load test – chức năng test add cart request



Hình 4.42: Kết quả add cart

Từ kết quả thực nghiệm cho thấy, đối với bài kiểm tra chịu tải (Stress Test) các chức năng chính của hệ thống khi có lượng lớn request, hệ thống vẫn đáp ứng rất tốt với tỷ lệ lỗi bằng 0% tại các mốc 300 và 500 request, và không ghi nhận lỗi ngay cả tại mốc 1000 request đồng thời. Về thời gian phản hồi, đa số các request được xử lý nhanh ở các mốc 300 – 500 request, trong đó các chức năng tìm kiếm và thêm vào giỏ hàng có thời gian phản hồi thấp, trong khi chức năng đăng nhập có xu hướng tăng thời gian phản hồi khi tải cao nhưng vẫn nằm trong ngưỡng chấp nhận được.

Đối với bài kiểm tra tải (Load Test), kết quả cho thấy khi một lượng lớn request được gửi liên tục trong thời gian dài, hệ thống vẫn hoạt động ổn định với tỷ lệ lỗi rất thấp. Điều này cho thấy hệ thống có tính ổn định và khả năng đáp ứng tốt trong điều kiện tải thực tế. Tuy nhiên, do hệ thống còn bị giới hạn về tài nguyên phần cứng nhằm tối ưu chi phí triển khai, khả năng đáp ứng của hệ thống vẫn còn những hạn chế nhất định và có thể được cải thiện thêm trong tương lai.

CHƯƠNG 4. KẾT QUẢ, KẾT LUẬN VÀ KIẾN NGHỊ

4.1. KẾT QUẢ ĐẠT ĐƯỢC

Sau quá trình nghiên cứu và thực hiện đề tài “*Xây dựng hệ thống bán hàng trực tuyến tích hợp AI và công nghệ microservices*”, đề tài đã đạt được các kết quả chính sau:

- **Về kiến thức và kỹ năng:** Đề tài giúp người thực hiện nắm vững kiến trúc microservices, cách xây dựng hệ thống backend với Spring Boot và Spring Cloud, phát triển frontend bằng React, cũng như tích hợp cơ sở dữ liệu và các dịch vụ bên thứ ba. Bên cạnh đó, đề tài bước đầu tiếp cận và ứng dụng AI thông qua chatbot và gợi ý sản phẩm.
- **Về hệ thống xây dựng được:** Hệ thống bán hàng trực tuyến đã được triển khai với các chức năng cơ bản như quản lý người dùng, sản phẩm, giỏ hàng, đặt hàng và thanh toán, vận chuyển, đánh giá sản phẩm, gợi ý sản phẩm và trang quản trị. Các chức năng này đáp ứng yêu cầu cốt lõi của một nền tảng thương mại điện tử hiện đại.
- **Đánh giá chung:** Kiến trúc microservices giúp hệ thống có khả năng mở rộng và bảo trì tốt. Việc tích hợp AI góp phần nâng cao trải nghiệm người dùng. Tuy nhiên, hệ thống vẫn còn hạn chế về độ phức tạp trong triển khai và phụ thuộc vào các dịch vụ bên ngoài.

4.2. KẾT LUẬN

Tiểu luận đã nghiên cứu và xây dựng thành công một hệ thống bán hàng trực tuyến dựa trên kiến trúc microservices, có tích hợp các kỹ thuật AI và các dịch vụ thanh toán, vận chuyển. Kết quả đạt được cho thấy kiến trúc microservices phù hợp với các hệ thống thương mại điện tử, đồng thời việc ứng dụng AI mang lại giá trị trong việc hỗ trợ người dùng và cá nhân hóa trải nghiệm mua sắm.

Nhìn chung, đề tài đã hoàn thành các mục tiêu đề ra và có thể làm nền tảng cho việc nghiên cứu, phát triển và mở rộng trong tương lai.

4.3. KIẾN NGHỊ

Để tiếp tục hoàn thiện hệ thống, đề tài đề xuất một số hướng phát triển trong thời gian tới:

- Hoàn thiện kiến trúc và hiệu năng thông qua tối ưu cơ sở dữ liệu, caching và xử lý bất đồng bộ.
- Mở rộng các chức năng AI, đặc biệt là gợi ý sản phẩm và phân tích hành vi người dùng.
- Tăng cường bảo mật và độ ổn định của hệ thống.
- Nâng cao trải nghiệm người dùng và khả năng triển khai thực tế.

TÀI LIỆU THAM KHẢO

1. Spring Boot Official Documentation. "Building Production-Ready Applications." Spring Framework, 2024. [Online]. Available: <https://spring.io/projects/spring-boot>.
2. Spring Cloud Documentation. "Microservices with Spring Cloud." Spring Framework, 2024. [Online]. Available: <https://spring.io/projects/spring-cloud>
3. Spring Security Documentation. "Authentication and Authorization." Spring Framework, 2024. [Online]. Available: <https://spring.io/projects/spring-security>
4. Google Gemini API Documentation. "Gemini AI Integration Guide." Google AI, 2024. [Online]. Available: <https://ai.google.dev/docs>.
5. React Official Documentation. "Building User Interfaces." Meta, 2024. [Online]. Available: <https://react.dev>.
6. TypeScript Documentation. "JavaScript with Syntax for Types." Microsoft, 2024. [Online]. Available: <https://www.typescriptlang.org/docs>.
7. Material-UI Documentation. "React Component Library." MUI Team, 2024. [Online]. Available: <https://mui.com>.
8. VNPay Integration Documentation. "Payment Gateway Integration Guide." VNPay, 2024. [Online]. Available: <https://sandbox.vnpayment.vn/apis/>.
9. GHN API Documentation. "Shipping Integration Guide." Giao Hàng Nhanh, 2024. [Online]. Available: <https://api.ghn.vn/>.
10. Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook. Springer Science & Business Media.
11. Aggarwal, C. C. (2016). Recommender Systems: The Textbook. Springer International Publishing.
12. Chen, H., Chiang, R. H., & Storey, V. C. (2012). Business intelligence and analytics: From big data to big impact. MIS quarterly, 1165-1188.

-
13. Zhang, S., Yao, L., Sun, A., & Tay, Y. (2019). Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys*, 52(1), 1-38.
 14. Lu, J., Wu, D., Mao, M., Wang, W., & Zhang, G. (2015). Recommender system application developments: a survey. *Decision Support Systems*, 74, 12-32.
 15. Isinkaye, F. O., Folajimi, Y. O., & Ojokoh, B. A. (2015). Recommendation systems: Principles, methods and evaluation. *Egyptian Informatics Journal*, 16(3), 261-273.
 16. FastAPI Documentation. "Modern Python Web Framework." FastAPI, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>.
 17. Scikit-learn Documentation. "Machine Learning in Python." Scikit-learn, 2024. [Online]. Available: <https://scikit-learn.org/stable/>.
 18. MySQL Documentation. "Relational Database Management System." Oracle, 2024. [Online]. Available: <https://dev.mysql.com/doc/>.
 19. Redis Documentation. "In-Memory Data Structure Store." Redis, 2024. [Online]. Available: <https://redis.io/docs/>
 20. React Query Documentation. "Data Fetching for React." TanStack, 2024. [Online]. Available: <https://tanstack.com/query/latest>.
 21. React Hook Form Documentation. "Performant Forms with Easy Validation." React Hook Form, 2024. [Online]. Available: <https://react-hook-form.com/>.
 22. Axios Documentation. "Promise-based HTTP Client." Axios, 2024. [Online]. Available: <https://axios-http.com/docs/intro>.
 23. JWT.io Documentation. "JSON Web Token Introduction." Auth0, 2024. [Online]. Available: <https://jwt.io/introduction>.
 24. OAuth 2.0 Documentation. "Authorization Framework." IETF, 2024. [Online]. Available: <https://oauth.net/2/>.
 25. Chart.js Documentation. "Simple yet flexible JavaScript charting." Chart.js, 2024. [Online]. Available: <https://www.chartjs.org/docs/>.

-
26. Metric.vn. "Báo cáo thị trường Sản TMĐT Quý III/2025." Metric, 2025. [Online]. Available: <https://metric.vn/bao-cau-thi-truong-san-tmdt-quy-3-2025>
27. Asrowardi, I., et al. "Designing microservice architectures for scalability and reliability in e-commerce." *International Journal of Electrical and Computer Engineering (IJECE)*, 2018.
28. G. Liu, B. Huang, Z. Liang, M. Qin, H. Zhou, and Z. Li, "Microservices: architecture, container, and challenges," in *2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, Macau, China, 2020, pp. 629-635. doi: 10.1109/QRS-C51114.2020.00107
29. Mozilla Developer Network (MDN). "HTTP response status codes." Mozilla, 2024. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>.

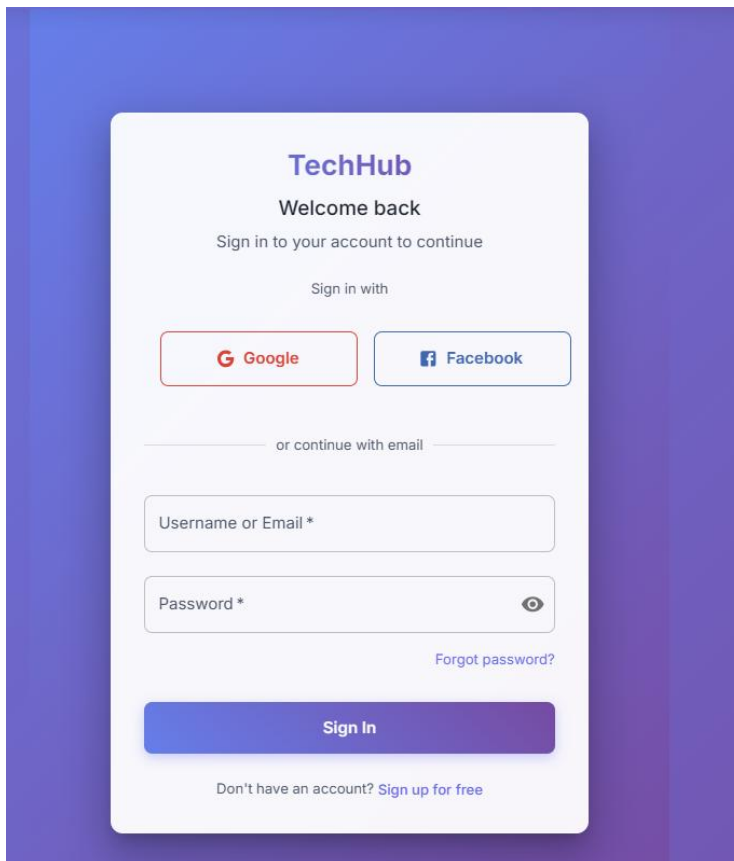
PHỤC LỤC CHO NGƯỜI DÙNG

5.1 DÀNH CHO NGƯỜI DÙNG MUA HÀNG.

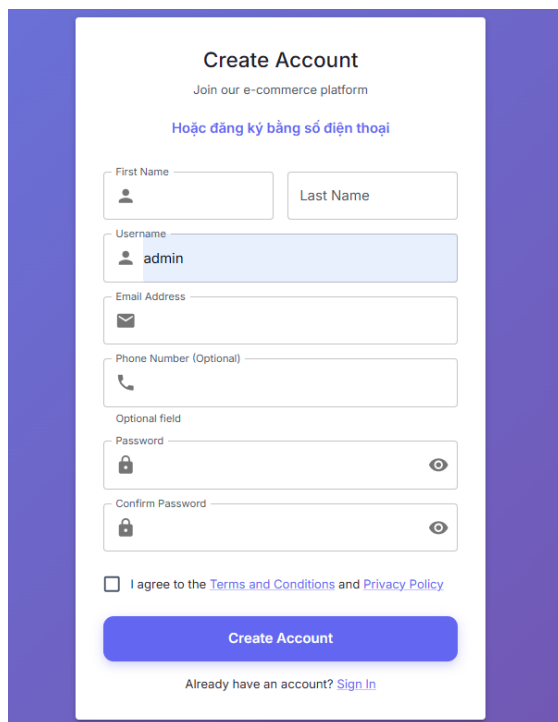
5.1.1 Đăng ký và Đăng nhập

Trước khi bắt đầu mua sắm, bạn cần có tài khoản để quản lý đơn hàng.

- **Đăng ký:** Truy cập trang đăng ký, nhập đầy đủ **Họ tên, Email và Mật khẩu**. Sau khi đăng ký thành công, hệ thống sẽ tự động đăng nhập.
- **Đăng nhập:** Sử dụng Email và Mật khẩu đã đăng ký. Bạn cũng có thể đăng nhập nhanh bằng tài khoản Google hoặc Facebook nếu hệ thống đã kích hoạt tính năng OAuth2.
- **Quên mật khẩu:** Nếu quên mật khẩu, hãy chọn chức năng "Quên mật khẩu" để được hướng dẫn lấy lại quyền truy cập.

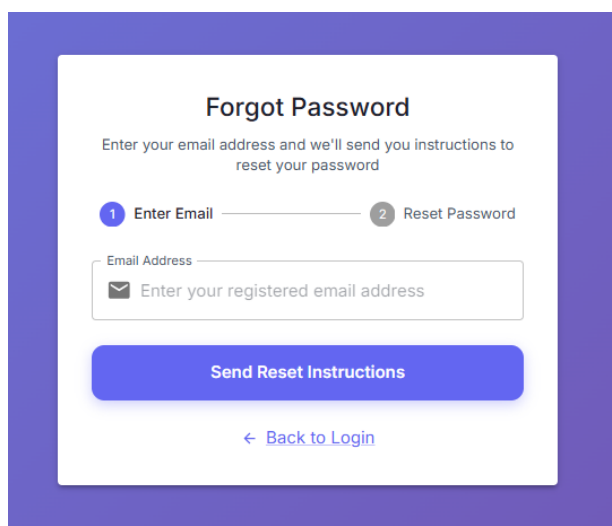


Hình 5.1 Giao diện trang đăng nhập dành cho khách hàng



The image shows a 'Create Account' form for an e-commerce platform. The form is titled 'Create Account' with the subtitle 'Join our e-commerce platform'. Below the title, there is a link 'Hoặc đăng ký bằng số điện thoại' (Or register by phone number). The form contains several input fields: 'First Name' and 'Last Name' (grouped), 'Username' (with the value 'admin' entered), 'Email Address', 'Phone Number (Optional)', 'Password', and 'Confirm Password'. There are also checkboxes for 'Optional field' and 'I agree to the Terms and Conditions and Privacy Policy'. A blue 'Create Account' button is at the bottom, and a link 'Already have an account? Sign In' is below it.

Hình 5.2 Giao diện trang đăng ký cho khách hàng.

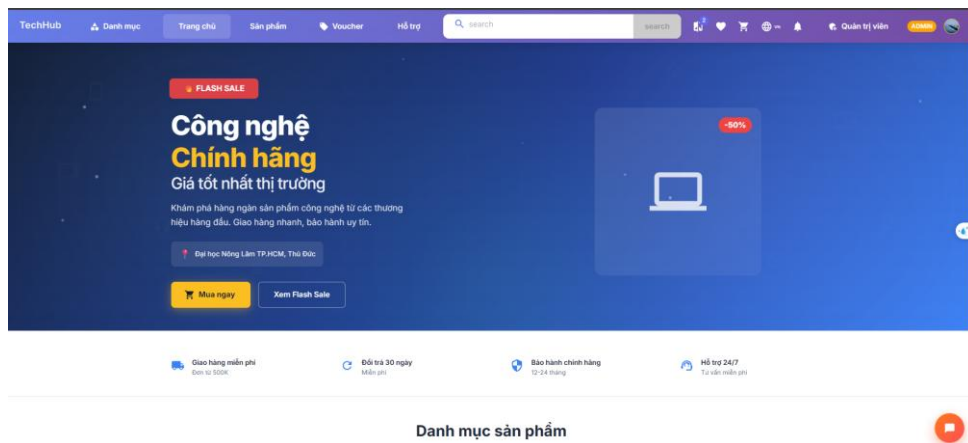


The image shows a 'Forgot Password' form. The title is 'Forgot Password'. Below the title, there is a subtitle 'Enter your email address and we'll send you instructions to reset your password'. The form has two steps: '1 Enter Email' and '2 Reset Password'. The 'Enter Email' step has an input field for 'Email Address' with a placeholder 'Enter your registered email address'. A blue 'Send Reset Instructions' button is below the input field. At the bottom, there is a link '< Back to Login'.

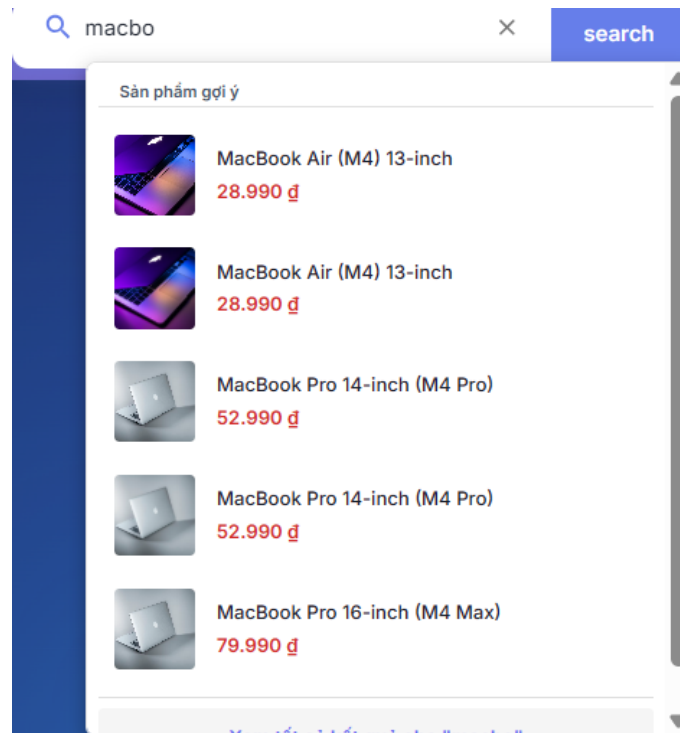
Hình 5.3 Giao diện quên mật khẩu

5.1.2 Tìm kiếm và Khám phá sản phẩm

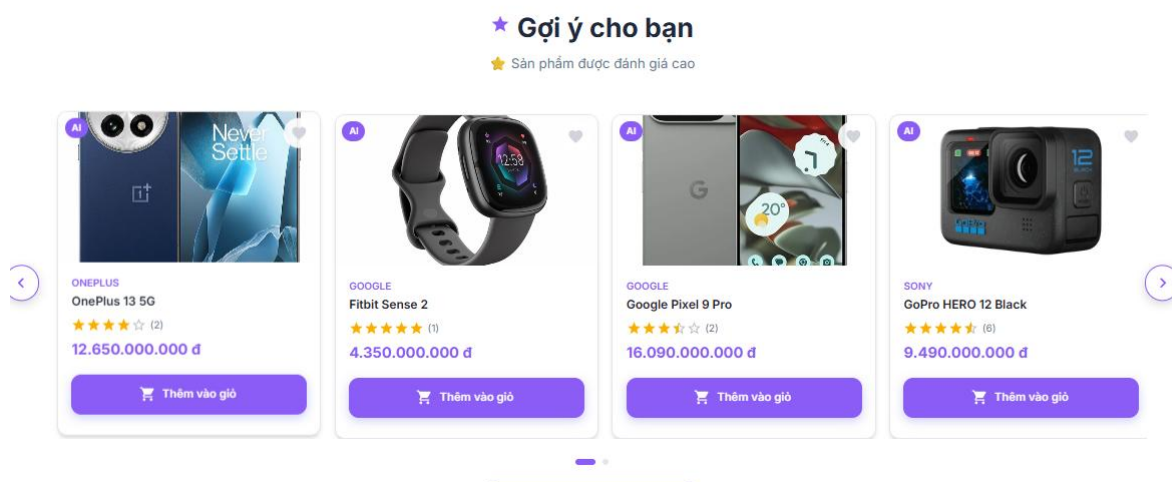
- **Trang chủ:** Tại đây hiển thị các danh sách sản phẩm nổi bật, sản phẩm đang giảm giá (Flash Sale) và các sản phẩm bán chạy.
- **Tìm kiếm thông minh:** Nhập tên sản phẩm, mã SKU hoặc từ khóa vào thanh tìm kiếm ở đầu trang để tìm nhanh món đồ bạn cần.
- **Gợi ý từ AI:** Khi bạn xem một sản phẩm, hệ thống sẽ tự động phân tích hành vi và hiển thị mục "**Có thể bạn sẽ thích**" ở phía dưới. Đây là các sản phẩm được thuật toán AI cá nhân hóa riêng cho bạn.



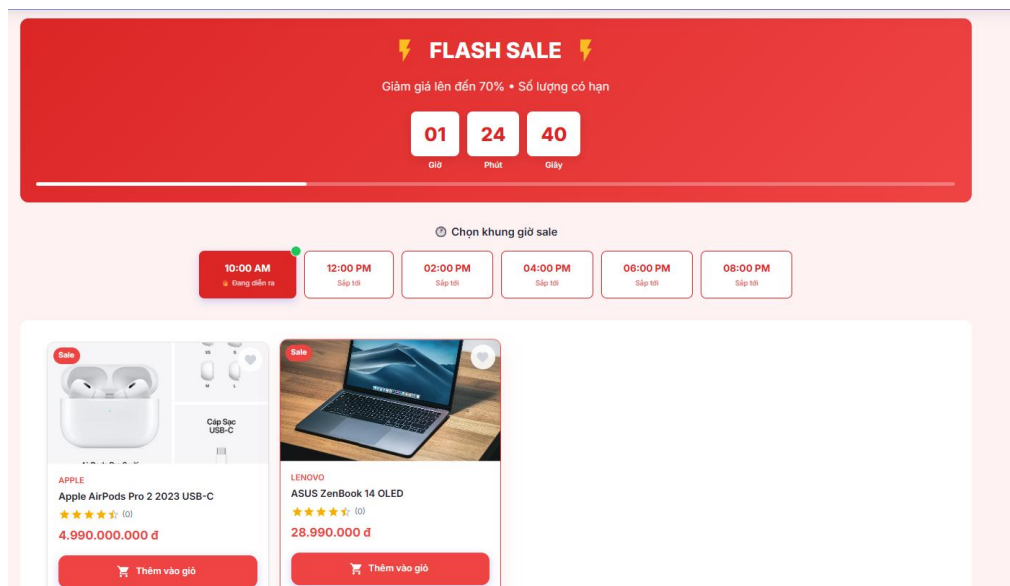
Hình 5.4 Giao diện trang chủ



Hình 5.5 Giao diện tìm kiếm sản phẩm.



Hình 5.6 Giao diện gợi ý sản phẩm.

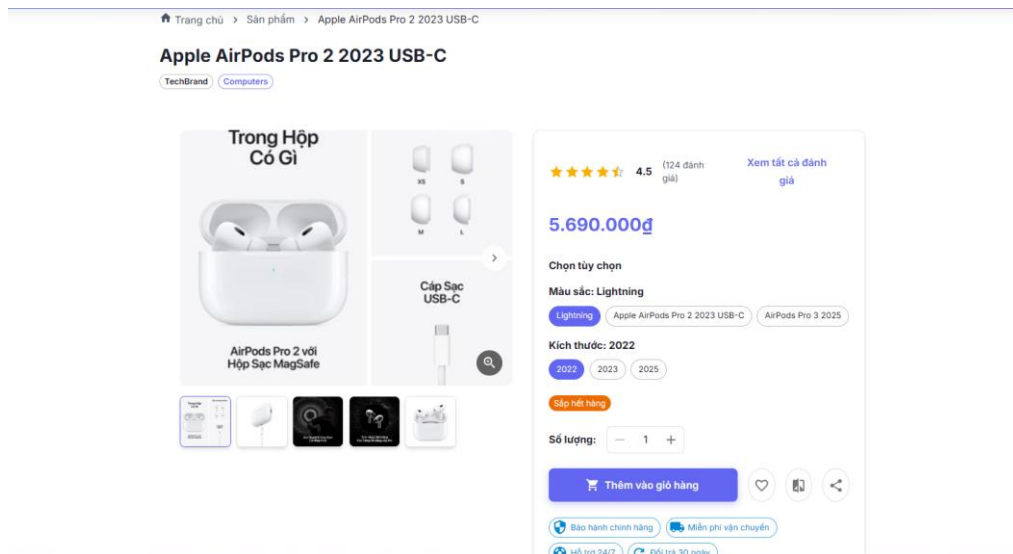


Hình 5.7 Giao diện mục sản phẩm sale

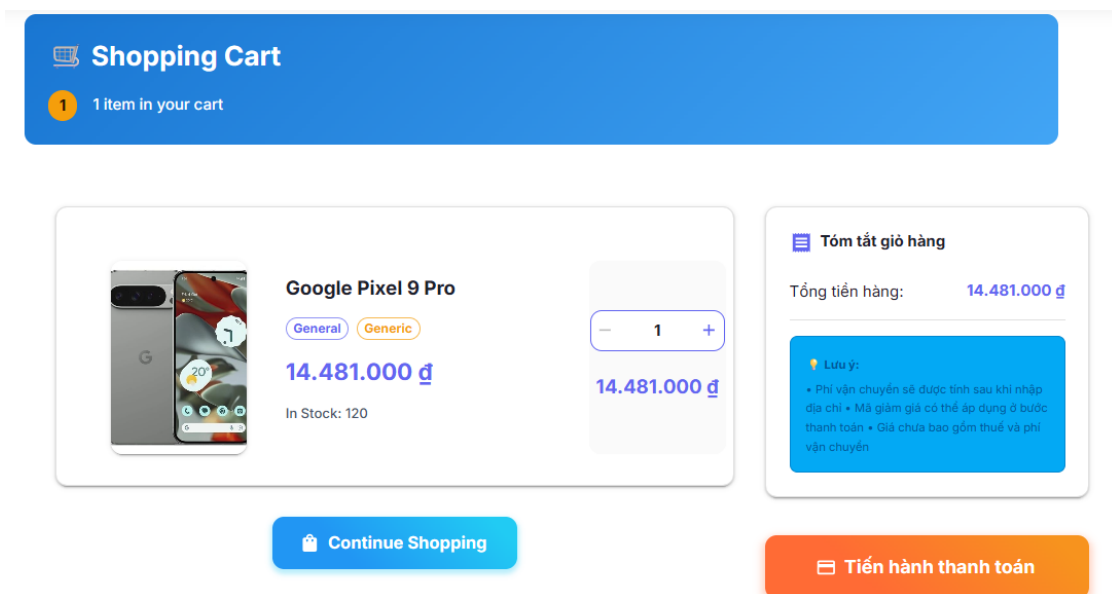
5.1.3 Mua hàng và Thanh toán

Quy trình mua hàng được tối ưu hóa qua 3 bước đơn giản:

1. **Thêm vào giỏ:** Tại trang chi tiết sản phẩm, chọn số lượng, màu sắc/phiên bản rồi nhấn "**Thêm vào giỏ hàng**". Bạn có thể xem lại giỏ hàng để thêm/bớt số lượng hoặc xóa sản phẩm.
2. **Đặt hàng:** Nhấn "Tiến hành đặt hàng". Hệ thống sẽ yêu cầu bạn điền/xác nhận thông tin giao hàng (Tỉnh/Thành phố, Quận/Huyện, Phường/Xã). Phí vận chuyển (kết nối với Giao Hàng Nhanh - GHN) sẽ được tính tự động.
3. **Thanh toán:**
 - Bạn có thể nhập **Mã giảm giá (Voucher)** nếu có.
 - Chọn phương thức thanh toán: **Thanh toán khi nhận hàng (COD)** hoặc **Thanh toán online qua VNPAY** (quét mã QR, thẻ ATM).
 - Sau khi hoàn tất, màn hình sẽ hiển thị thông báo "Đặt hàng thành công" kèm mã đơn hàng.



Hình 5.8 Giao diện trang chi tiết sản phẩm.



Hình 5.9 Giao diện sản phẩm trong giỏ hàng.

Checkout

1 Shipping Address

2 Payment & Voucher

Địa chỉ giao hàng

Họ *

Admin

Tên *

User

Email *

admin@example.com

Số điện thoại *

Địa chỉ chi tiết *

Tỉnh/Thành phố **

Quận/Huyện **

Xã/Phường **

Ghi chú (tùy chọn)

Lưu địa chỉ này

Quay lại

Tiếp tục

Vui lòng điền đầy đủ thông tin địa chỉ giao hàng

Order Summary

1 item(s)

Subtotal:

14.481.000 đ

Shipping:

Chưa xác định

Tax:

1.448.100 đ

Total:

15.929.100 đ

Hình 5.10 Giao diện trang nhập thông tin giao hàng.

Chọn mã giảm giá

UNIVERSAL10

Giảm giá 10% cho mọi đơn hàng

Voucher giảm 10% cho tất cả đơn hàng, không giới hạn giá trị đơn hàng tối thiểu. Áp dụng cho tất cả sản phẩm. Hạn sử dụng 1 năm.

0 HSD: 13/12/2026

10%

Tối đa 50.000 đ

Giảm 50.000 đ

FREESHIP50K

Miễn phí ship

Free Ship 50k

Miễn phí vận chuyển 50,000 VND cho đơn hàng từ 500,000 VND. Áp dụng cho tất cả sản phẩm. Hạn sử dụng 1 năm.

Đơn tối thiểu: 500.000 đ HSD: 05/12/2026

50.000 đ

Giảm 50.000 đ

BVVBCV

vbcbvbcv

Đơn tối thiểu: 5.000 đ HSD: 30/12/2025

10%

Tối đa 500 đ

Giảm 500 đ

CCCC

cccc

1234556666

Đơn tối thiểu: 100.000 đ HSD: 30/12/2025

10.000 đ

Giảm 10.000 đ

Đóng

Hình 5.11 Giao diện áp dụng mã giảm giá.

98

Shipping Address

Payment & Voucher

Voucher

☒ Voucher đã áp dụng: FREESHIP50K

Giảm 50.000 đ

Xóa

Phương thức thanh toán

Payment Method

Select Payment Method

☒ Thanh toán VNPay

☐ Thanh toán khi nhận hàng (COD)

Cổng thanh toán:

☒ VNPay

☐ VNPay - Thanh toán qua thẻ ATM, thẻ quốc tế, ví điện tử.

← Quay lại

Đặt hàng

Order Summary

1 item(s)

Subtotal:

14.481.000 đ

Shipping:

44.000 đ

Tax:

1.448.100 đ

Voucher (FREESHIP50K):

-50.000 đ

Total:

15.923.100 đ

Hình 5.12 Giao diện tổng khi đang thanh toán.

5.1.4 Quản lý Cá nhân & Đơn hàng

- **Theo dõi đơn hàng:** Vào mục "Lịch sử đơn hàng" để xem trạng thái đơn của bạn (ví dụ: *Chờ duyệt*, *Đang giao*, *Đã giao*). Trạng thái này được cập nhật theo thời gian thực (Real-time).
- **Hồ sơ cá nhân:** Bạn có thể cập nhật ảnh đại diện, đổi tên hiển thị hoặc thay đổi mật khẩu trong phần "Tài khoản".
- **Đánh giá:** Sau khi nhận hàng, bạn có thể viết đánh giá và chấm sao cho sản phẩm để giúp những người mua sau.



Người dùng quản trị

admin@example.com

QUẢN TRỊ VIÊN Đã xác minh

HỒ SƠ

BẢO VỆ

TÙY CHỌN

THÔNG BÁO

Tên

Admin

Họ

User

E-mail

admin@example.com

Số điện thoại

Địa chỉ

Ngày sinh

mm/dd/yyyy

Chỉnh sửa hồ sơ

Hình 5.13 Giao diện trang cá nhân người dùng

Chi tiết đơn hàng

Mã đơn hàng

333

Trạng thái

Đang chờ phê duyệt

Chưa thanh toán (COD)

Chi tiết đơn hàng

Tổng phụ:	12.650.000 đ
Thuế:	1.265.000 đ
Vận chuyển:	44.000 đ
Giảm giá:	- 50.000 đ
Tổng cộng:	13.909.000 đ

Các mục (1)



OnePlus 13 5G

Số lượng: 1

Tổng cộng: 12.650.000 đ

Đã ra lệnh: lúc 17:35 25 tháng 12 năm 2025

Đã cập nhật: lúc 17:35 25 tháng 12 năm 2025

đơn hàng

Đơn hàng số 333

Đang chờ phê duyệt

Chưa thanh toán (COD)

1 mặt hàng

13.909.000 đ

Xem chi tiết

Hủy bỏ

Đơn hàng số 330

Đã vận chuyển

Chưa thanh toán (COD)

1 mặt hàng

32.105 đ

Xem chi tiết

Đơn hàng số 328

Đã hủy

Chưa được trả lương

1 mặt hàng

-

Đơn hàng số 325

Đã hủy

Chưa được trả lương

1 mặt hàng

-

Đơn hàng số 123

Đã hủy

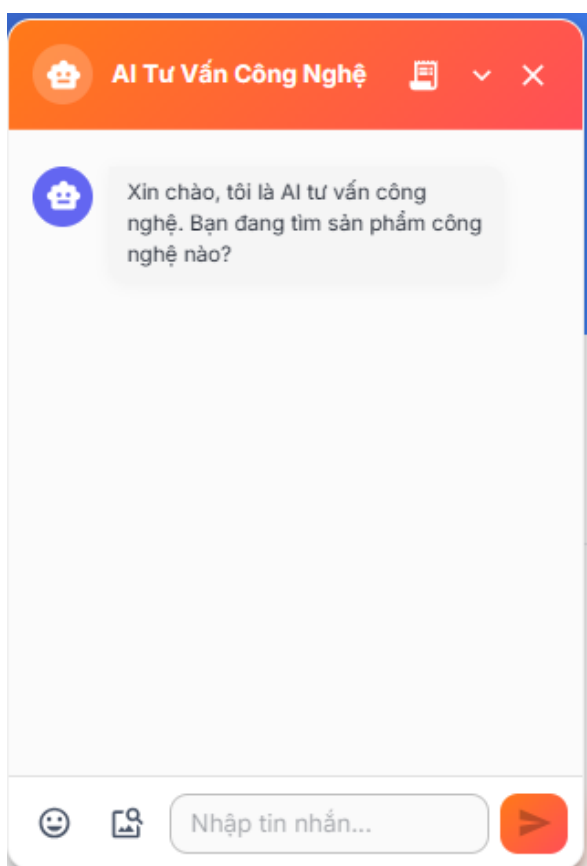
Chưa thanh toán (COD)

Hình 5.14 Giao diện trang lịch sử đơn hàng.

5.1.5 Sử dụng Chatbot AI Tư vấn

Nếu bạn phân vân chưa biết mua gì, hãy hỏi Trợ lý ảo:

- Nhấn vào **biểu tượng Chatbot (hình tin nhắn)** trôi nổi ở góc màn hình.
- Bạn có thể hỏi bằng ngôn ngữ tự nhiên, ví dụ: "Tìm điện thoại dưới 5 triệu chụp ảnh đẹp" hoặc "So sánh iPhone 13 và 14".
- Chatbot sử dụng công nghệ **Google Gemini AI** sẽ trả lời chi tiết, tư vấn cấu hình và đưa ra sản phẩm phù hợp ngay trong khung chat.



Hình 5.15 Giao diện chatbot

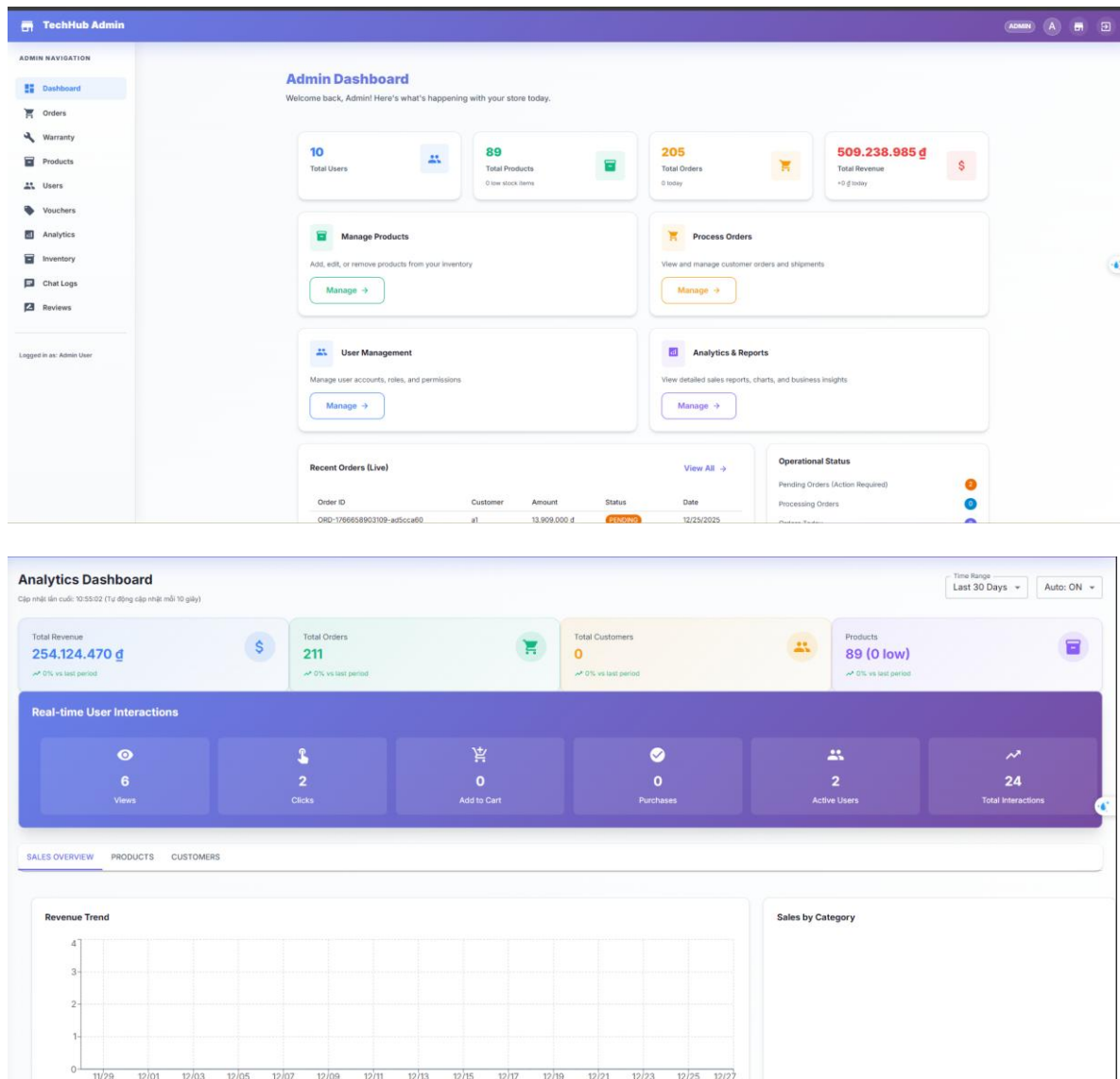
5.2 DÀNH CHO QUẢN TRỊ VIÊN (ADMIN)

5.2.1 Tổng quan (Dashboard)

Ngay khi đăng nhập, màn hình Thống kê (Analytics) sẽ hiển thị các biểu đồ trực quan về:

- Tổng doanh thu theo ngày/tháng.

- Số lượng đơn hàng mới.
- Số lượng người dùng mới đăng ký.

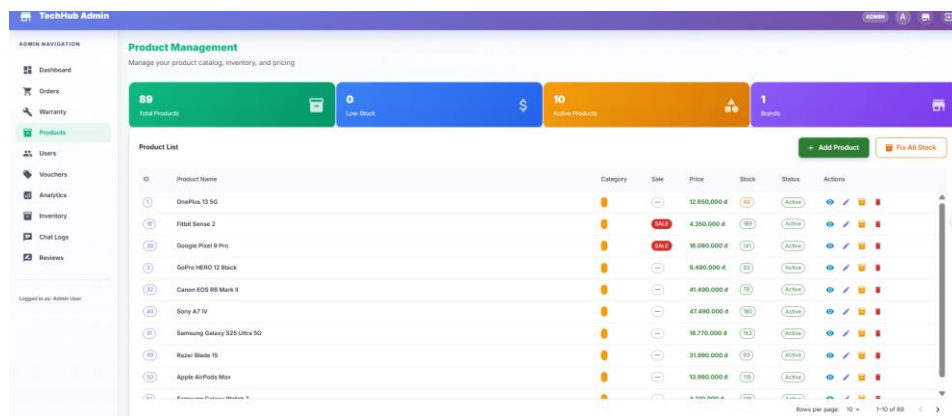


Hình 5.16 Giao diện trang admin Dashboard và Analytics.

5.2.2 Quản lý Sản phẩm & Người dùng

- **Sản phẩm:** Thêm mới, sửa thông tin, cập nhật giá/tồn kho, hoặc xóa sản phẩm khỏi hệ thống, cập nhập sản phẩm nào được sale.
- **Người dùng:** Xem danh sách khách hàng, thực hiện phân quyền (nâng quyền quản trị hoặc khóa tài khoản vi phạm).

- **Voucher:** Tạo các mã giảm giá mới, thiết lập mức giảm (% , số tiền) và thời gian áp dụng.



Hình 5.17 Giao diện trang quản lý sản phẩm.

The screenshot shows the 'Create New User' form. It has a green header bar with a plus icon and the text 'Create New User'. The form contains several input fields: 'First Name', 'Last Name', 'Email', 'Username', 'Password', and 'Phone Number'. A 'Role' dropdown menu is open, displaying a list of roles: 'User', 'Moderator', 'Product Manager', 'User Manager', 'Support', 'Repair Technician', and 'Admin'. Below the input fields is a text area labeled 'Lý do thay đổi quyền hạn (nếu có)'. At the bottom right, there are two buttons: 'Cancel' and 'Create User'.

Hình 5.18 Giao diện thêm user và phân quyền.

Quản lý Voucher

Tạo và quản lý mã giảm giá cho hệ thống

+ Tạo Voucher

5

Tổng voucher

5

Đang hoạt động

0

Lượt sử dụng

5

Công khai

Mã Voucher	Loại	Giá trị	ĐK tối thiểu	Giới hạn	Đã dùng	Trạng thái	Thao tác
ABC AB1	Phần trăm	10%	100.000đ	100	0	Hoạt động Công khai	
CCCC cccc	Số tiền cố định	10.000đ	100.000đ	11	0	Hoạt động Công khai	
BVVBCV vcbcvbvc	Phần trăm	10%	5.000đ	100	0	Hoạt động Công khai	
FREESHIP50K Free Ship 50k	Số tiền cố định	50.000đ	500.000đ	100	0	Hoạt động Công khai	
UNIVERSAL10 Giảm giá 10% cho mọi đơn hàng	Phần trăm	10%	Không có	10000	0	Hoạt động Công khai	

Hình 5.19 Giao diện trang quản lý voucher.

5.2.3 Quản lý Đơn hàng

- Admin có quyền xem danh sách tất cả đơn hàng.
- Duyệt đơn:** Chuyển trạng thái đơn hàng từ "Chờ duyệt" sang "Đang giao" hoặc "Hủy đơn" nếu cần thiết. Mọi thay đổi sẽ được cập nhật ngay lập tức bên phía khách hàng.

Order Management

Manage customer orders, track shipments, and update order statuses

416

Total Orders

2

Pending Orders

3

Shipped Orders

14.402.240 đ

Total Revenue

Order List

Filter by Status

Order ID	Order #	Customer	Status	Total	Created	Actions
#333	ORD-1766658903309-3k	Order #ORD-1766658903309-3k	PENDING	13.900.000 đ	25/12/2025	
#332	ORD-1766506456947-0i	Order #ORD-1766506456947-0i	SHIPPED	92.080 đ	23/12/2025	
#331	ORD-1765963844689-e	Order #ORD-1765963844689-e	PENDING	30.576 đ	17/12/2025	
#330	ORD-1765958722685-6l	Order #ORD-1765958722685-6l	SHIPPED	32.105 đ	17/12/2025	
#328	ORD-1765957776849-4i	Order #ORD-1765957776849-4i	CANCELLED	30.393 đ	17/12/2025	
#327	ORD-1765683332805-en	Order #ORD-1765683332805-en	SHIPPED	153.834 đ	13/12/2025	
#325	ORD-1765534960864-2i	Order #ORD-1765534960864-2i	CANCELLED	38.814 đ	12/12/2025	
#323	ORD-1765087363380-7n	Order #ORD-1765087363380-7n	CANCELLED	36.512 đ	7/12/2025	
#322	ORD-1764848387568-7i	Order #ORD-1764848387568-7i	CANCELLED	39.882 đ	4/12/2025	

Rows per page: 10

1-10 of 416

Hình 5.20 Giao diện trang quản lý đơn hàng.

5.2.4 Quản lý chatLogs và review sản phẩm

- Admin có quyền xem lại các phản hồi từ các câu hỏi của khách hàng và các lượt đánh giá sản phẩm.
- **Giám sát ChatLogs:** Hệ thống lưu trữ và hiển thị toàn bộ lịch sử hội thoại giữa người dùng và Chatbot. Admin có thể truy xuất các đoạn hội thoại này để:
 - Đánh giá độ chính xác trong câu trả lời của AI (Google Gemini).
 - Nắm bắt nhu cầu tìm kiếm thực tế của khách hàng để tối ưu hóa chiến lược nhập hàng.
- **Quản lý Đánh giá (Reviews):** Admin có quyền kiểm soát các nội dung đánh giá sản phẩm:
 - Xem chi tiết số sao (Rating) và nội dung bình luận (Comment) của từng sản phẩm.
 - Phát hiện và ẩn/xóa các bình luận spam, thiếu văn hóa hoặc vi phạm tiêu chuẩn cộng đồng để đảm bảo môi trường mua sắm lành mạnh.

Chat Logs Management						
Xem và phân tích các câu hỏi từ chatbot để cải thiện sản phẩm						
Tổng số logs 36	Tìm thấy sản phẩm 22	Không tìm thấy 0	Câu hỏi AI 12			
Tìm kiếm...		User ID	Loại câu hỏi Tất cả	Làm mới		
Thời gian	User ID	Câu hỏi	Nguồn	Trạng thái	Đánh giá	Thao tác
22:35 23/12/2025	af9	1+1 = ?	FALLBACK		Chưa đánh giá	
22:35 23/12/2025	af9	alo	RULE-BASED		Chưa đánh giá	
12:37 05/12/2025	af	shop có bán ship hàng không	AI	AI	Chưa đánh giá	
23:35 04/12/2025	af0d00b-1d14-4ca3-9d7b-7a19a72ae610	1+3 = bao nhiêu á	AI	AI	Chưa đánh giá	
23:34 04/12/2025	af0d00b-1d14-4ca3-9d7b-7a19a72ae610	shop hướng dẫn cách dùng thanh toán vnp...	AI	AI	Chưa đánh giá	
23:33 04/12/2025	af0d00b-1d14-4ca3-9d7b-7a19a72ae610	shop có bán sản phẩm samsung nào không	DATABASE	Sản phẩm Có kết quả	Chưa đánh giá	
20:23					Chưa đánh...	

Hình 5.21 Giao diện trang ghi nhận lại nội dung từ chatbot.

Quản lý Đánh giá Sản phẩm										
608 Tổng đánh giá		232 Đã duyệt		376 Chờ duyệt		4.1 Điểm trung bình				
Trạng thái: Tất cả		Mã sản phẩm		Đánh giá		Xóa bộ lọc				
ID	Người dùng	Mã sản phẩm	Đánh giá	Tiêu đề	Nội dung	Trạng thái	Xác minh	Hữu ích	Ngày tạo	Thao tác
603	User	p109	★★★★★ 5/5	Rất hài lòng	Chất lượng tuyệt vời, thiết kế đẹp, hiệu n...	Đã duyệt	Đã mua	33/49	28/8/2025	
604	User	48	★★★★★ 5/5	Đáng đồng tiền	Chất lượng tuyệt vời, thiết kế đẹp, hiệu n...	Chờ duyệt	Đã mua	33/45	12/7/2025	
605	User	95	★★★★★ 5/5	Hoàn hảo!	Sản phẩm rất tốt, chất lượng vượt trội, gi...	Đã duyệt	Chưa mua	34/35	28/5/2025	
606	User	27	★★★★ 4/5	Khả ản	Sản phẩm khá tốt, tuy có một số điểm n...	Đã duyệt	Đã mua	44/47	14/9/2025	
607	User	100	★★★★ 3/5	Bình thường	Thiết kế bình thường, hiệu năng vừa đủ. ...	Đã duyệt	Đã mua	45/48	13/8/2025	
608	QuangT1 Chuongcoi	82	★★★★★ 5/5	Đáng đồng tiền	Sản phẩm rất tốt, chất lượng vượt trội, gi...	Đã duyệt	Đã mua	3/19	23/10/2025	
609	User	45	★★★★ 3/5	Cũng được	Sản phẩm bình thường, không có gì đặc ...	Đã duyệt	Chưa mua	45/84	23/8/2025	
610	User	61	★★★★ 3/5	Không quá tệ	Thiết kế bình thường, hiệu năng vừa đủ. ...	Đã duyệt	Đã mua	12/28	31/8/2025	
611	User	6	★★★★★ 5/5	Hoàn hảo!	Đã mua nhiều sản phẩm công nghệ như...	Đã duyệt	Chưa mua	14/29	10/7/2025	

Hình 5.22 Giao diện trang đánh giá.

HƯỚNG DẪN CÀI ĐẶT HỆ THỐNG

6.1 TỔNG QUAN HỆ THỐNG.

Dự án xây dựng một hệ thống thương mại điện tử đa dịch vụ (Microservices), có khả năng mở rộng và tích hợp trí tuệ nhân tạo (AI) nhằm nâng cao trải nghiệm người dùng.

Kiến trúc tổng thể:

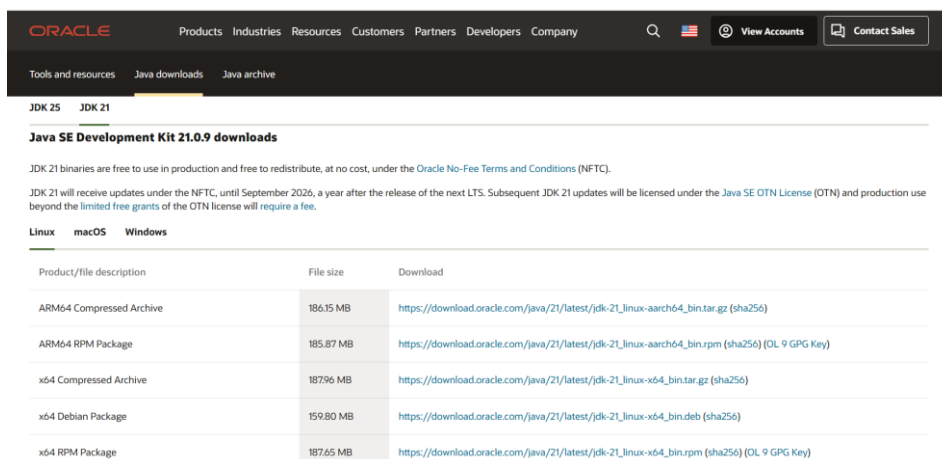
- **Backend:** Spring Boot & Spring Cloud (Java), triển khai theo mô hình Microservices.
- **Frontend:** ReactJS sử dụng TypeScript và Material UI.
- **AI Services:** Google Gemini Pro và dịch vụ Machine Learning viết bằng Python.
- **Hạ tầng & Trung gian:** MySQL, Redis.

Hệ thống hỗ trợ các chức năng chính như xác thực người dùng, quản lý sản phẩm, xử lý đơn hàng, gợi ý sản phẩm thông minh và chatbot tư vấn.

6.2 YÊU CẦU MÔI TRƯỜNG.

6.2.1 Phần mềm cơ bản

Java Development Kit (JDK): Phiên bản 17 trở lên.

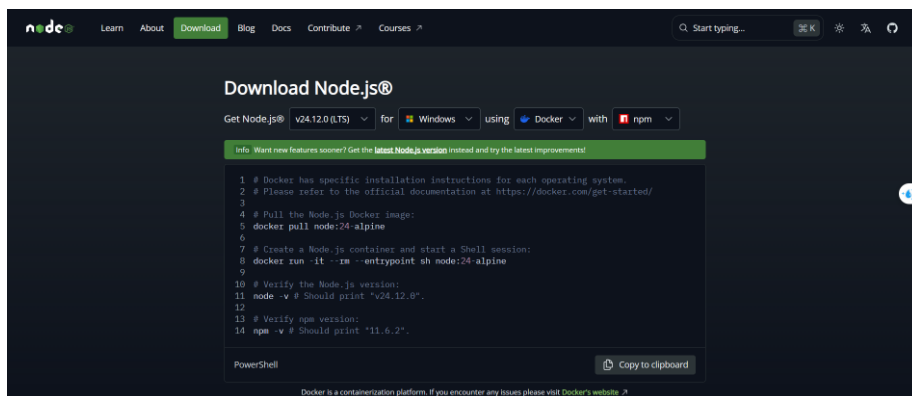


The screenshot shows the Oracle Java Downloads page for JDK 21.0.9. It includes navigation links for Products, Industries, Resources, Customers, Partners, Developers, and Company. Below the navigation bar, there are tabs for Tools and resources, Java downloads (selected), and Java archive. The main content area is titled 'Java SE Development Kit 21.0.9 downloads'. It contains a table with columns for Product/file description, File size, and Download. The table lists five download options for Linux, macOS, and Windows, including ARM64 and x64 Compressed Archives, RPM Packages, and Debian Packages. Each row provides a link to the download page.

Product/file description	File size	Download
ARM64 Compressed Archive	186.15 MB	https://download.oracle.com/java/21/latest/jdk-21_linux-aarch64_bin.tar.gz (sha256)
ARM64 RPM Package	185.87 MB	https://download.oracle.com/java/21/latest/jdk-21_linux-aarch64_bin.rpm (sha256) (OL 9 GPG Key)
x64 Compressed Archive	187.96 MB	https://download.oracle.com/java/21/latest/jdk-21_linux-x64_bin.tar.gz (sha256)
x64 Debian Package	159.80 MB	https://download.oracle.com/java/21/latest/jdk-21_linux-x64_bin.deb (sha256)
x64 RPM Package	187.65 MB	https://download.oracle.com/java/21/latest/jdk-21_linux-x64_bin.rpm (sha256) (OL 9 GPG Key)

Hình 6.1 Trang tải JDK (nguồn [Java Downloads | Oracle](https://www.oracle.com/in/java/technologies/javase-downloads.html))

Node.js: Phiên bản 24.x (kèm npm hoặc yarn).



Hình 6.2 Trang tải node.Js (nguồn <https://nodejs.org/en/download>)

Python: Phiên bản 3.11 trở lên.

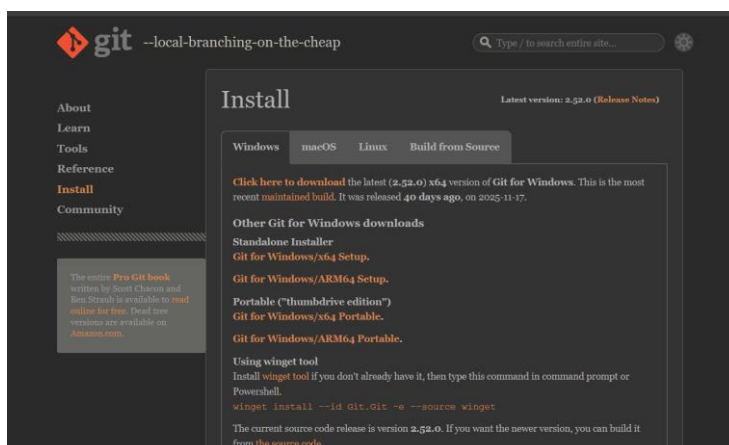
Active Python releases

For more information visit the Python Developer's Guide.

Python version	Maintenance status		First released	End of support	Release schedule
3.15	pre-release	Download	2026-10-07 (planned)	2031-10	PEP 790
3.14	bugfix	Download	2025-10-07	2030-10	PEP 745
3.13	bugfix	Download	2024-10-07	2029-10	PEP 719
3.12	security	Download	2023-10-02	2028-10	PEP 693
3.11	security	Download	2022-10-24	2027-10	PEP 664
3.10	security	Download	2021-10-04	2026-10	PEP 619
3.9	end of life, last release was 3.9.25	Download	2020-10-05	2025-10-31	PEP 596

Hình 6.3 Trang tải python (nguồn <https://www.python.org/downloads/>)

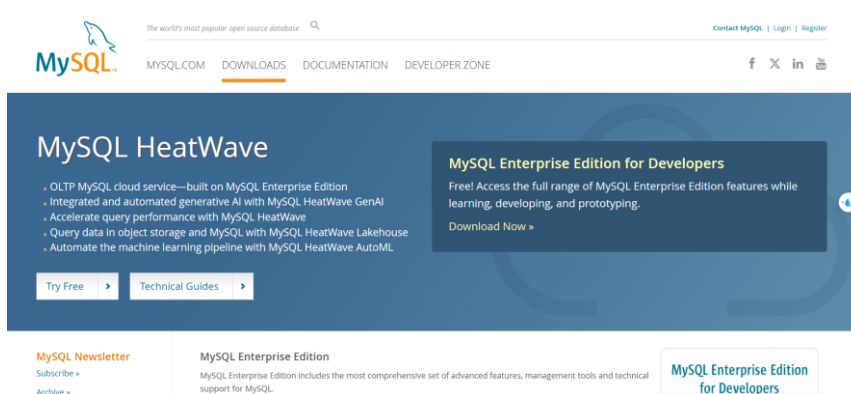
Công cụ build & quản lý mã nguồn: Apache Maven 3.8+ và Git.



Hình 6.4 Trang tải git (nguồn <https://git-scm.com/install/windows>)

6.2.2. Cơ sở dữ liệu & Middleware

MySQL: Phiên bản 8.0 trở lên.



Hình 6.5 Trang tải MySQL (nguồn <https://www.mysql.com/downloads/>)

Redis: Phiên bản 6.0+ (dùng cho Session và Caching).

6.3. CẤU HÌNH CƠ SỞ DỮ LIỆU

Đăng nhập MySQL và tạo các Schema cho từng dịch vụ:

- ecommerce_db: Lưu trữ dữ liệu dùng chung.
- auth_db: Quản lý tài khoản và xác thực.
- product_db: Quản lý danh mục và sản phẩm.
- order_db: Quản lý đơn hàng và giao dịch.
- admin_db: Dữ liệu quản trị và báo cáo.

Lưu ý: Các bảng dữ liệu sẽ được tự động tạo khi khởi chạy lần đầu thông qua cấu hình hibernate.ddl-auto: update.

```
url: jdbc:mysql://localhost:3306/product_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true
username: ${DB_USERNAME:root}
password: ${DB_PASSWORD:123456}
driver-class-name: com.mysql.cj.jdbc.Driver
jpa:
  hibernate:
    ddl-auto: update
  properties:
    hibernate:
      dialect: org.hibernate.dialect.MySQLDialect
show-sql: true
```

Hình 6.6 Giao diện phần cấu hình sql

6.4. TRIỂN KHAI BACKEND (MICROSERVICES)

Các dịch vụ Backend cần được khởi động theo đúng trình tự để đảm bảo quá trình đăng ký dịch vụ diễn ra chính xác.

6.4.1. Khởi động các dịch vụ lõi (Core Services)

1. **Config Server** (Port: 8888) – Quản lý cấu hình tập trung. Lệnh chạy: `mvn spring-boot:run` tại thư mục `/services/config-server`.
2. **Discovery Server (Eureka)** (Port: 8761) – Quản lý định danh dịch vụ. Lệnh chạy: `mvn spring-boot:run` tại thư mục `/services/discovery-server`.
3. **API Gateway** (Port: 8080) – Điểm tiếp nhận request tập trung. Lệnh chạy: `mvn spring-boot:run` tại thư mục `/services/gateway`.

6.4.2. Khởi động các dịch vụ nghiệp vụ

Sau khi Core Services hoạt động ổn định, tiến hành chạy các dịch vụ sau (thứ tự không bắt buộc):

- **auth-service** (Port: 8081)
- **user-service** (Port: 8082)
- **product-service** (Port: 8083)
- **order-service** (Port: 8084)
- **ai-service** (Port: 8088)
- **recommendation-service** (Port: 8094)

```
D:\Build\24_10\Build\39_7\Build\43\services\ai-service>mvn spring-boot:run
[INFO] Scanning for projects...
[INFO]
----- com.example.ai-service -----
[INFO] Building ai-service 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO]
----- [ jar ]-----
[INFO]
>>> spring-boot:3.3.0:run (default-cli) > test-compile @ ai-service >>>
[INFO]
--- resources:3.3.1:resources (default-resources) @ ai-service ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
--- compiler:3.13.0:compile (default-compile) @ ai-service ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
--- resources:3.3.1:testResources (default-testResources) @ ai-service ---
[INFO] skip non existing resourceDirectory D:\Build\24_10\Build\39_7\Build\43\services\ai-service\src\test\resources
[INFO]
--- compiler:3.13.0:testCompile (default-testCompile) @ ai-service ---
[INFO] No sources to compile
[INFO]
<<< spring-boot:3.3.0:run (default-cli) < test-compile @ ai-service <<<
[INFO]
--- spring-boot:3.3.0:run (default-cli) @ ai-service ---
[INFO] Attaching agents: []
```


=====

```
:: Spring Boot ::                (v3.3.4)

2025-12-27T23:08:48.377+08:00 INFO 24394 --- [ai-service] [           main] com.example.ai.AiServiceApplication : Starting AiServiceApplication using
Java 21.0.2 with PID 24394 (D:\Build\24_10\Build\39_7\Build\43\services\ai-service\target\classes started by TU QUANG CHUNG in D:\Build\24_10\Build\39_7\)
```

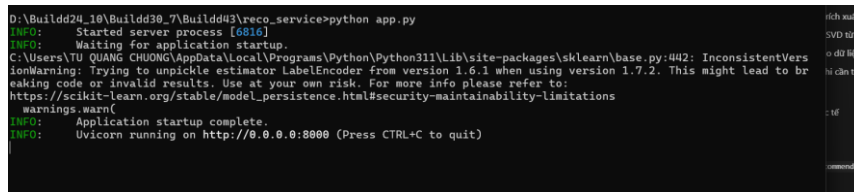
Hình 6.7 Cách run lệnh `mvn spring-boot:run`

6.5. TRIỂN KHAI AI & MACHINE LEARNING SERVICE

Dịch vụ Machine Learning cung cấp thuật toán gợi ý sản phẩm chuyên sâu. Các bước triển khai:

1. Truy cập thư mục: `services/python-ml-service`.

2. Tạo môi trường ảo: `python -m venv venv`.
3. Kích hoạt môi trường:
 - Windows: `venv\Scripts\activate`
 - Linux/macOS: `source venv/bin/activate`
4. Cài đặt thư viện: `pip install -r requirements.txt`.
5. Khởi chạy dịch vụ (Port: 8000): `python main.py`.



```
D:\Build24_10\Build30_7\Build43\reco_service>python app.py
INFO: Started server process [6816]
INFO: Waiting for application startup.
C:\Users\TU QUANG CHUONG\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:442: InconsistentVersionWarning: Trying to unpickle estimator LabelEncoder from version 1.6.1 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

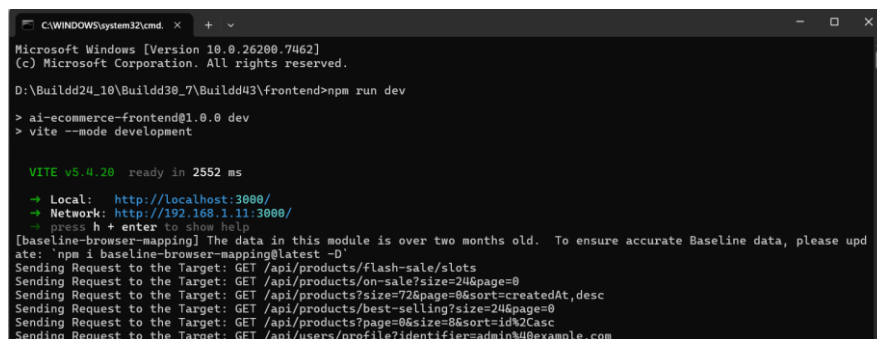
Hình 6.8 Giao diện run thành công

6.6 TRIỂN KHAI FRONTEND

1. Truy cập thư mục frontend: `cd frontend`.
2. Cài đặt thư viện: `npm install`.
3. Tạo file cấu hình môi trường `.env` với nội dung:

`VITE_API_BASE_URL=http://localhost:8080`

4. Khởi chạy ứng dụng: `npm run dev`.
5. Truy cập hệ thống tại: <http://localhost:3000>.



```
C:\WINDOWS\system32\cmd. X +
Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

D:\Build24_10\Build30_7\Build43\frontend>npm run dev

> ai-e-commerce-frontend@1.0.0 dev
> vite --mode development

VITE v5.4.20 ready in 2552 ms
  ➔ Local:   http://localhost:3000/
  ➔ Network: http://192.168.1.11:3000/
    press h + enter to show help
[baseline-browser-mapping] The data in this module is over two months old. To ensure accurate Baseline data, please update: 'npm i baseline-browser-mapping@latest -D'
Sending Request to the Target: GET /api/products/flash-sale/slots
Sending Request to the Target: GET /api/products/on-sale?size=24&page=0
Sending Request to the Target: GET /api/products?size=72&page=0&sort=createdAt,desc
Sending Request to the Target: GET /api/products/best-selling?size=24&page=0
Sending Request to the Target: GET /api/products?page=0&size=8&sort=id%2Casc
Sending Request to the Target: GET /api/users/profile?identifier=admin%40example.com
```

Hình 6.9 Giao diện run frontend

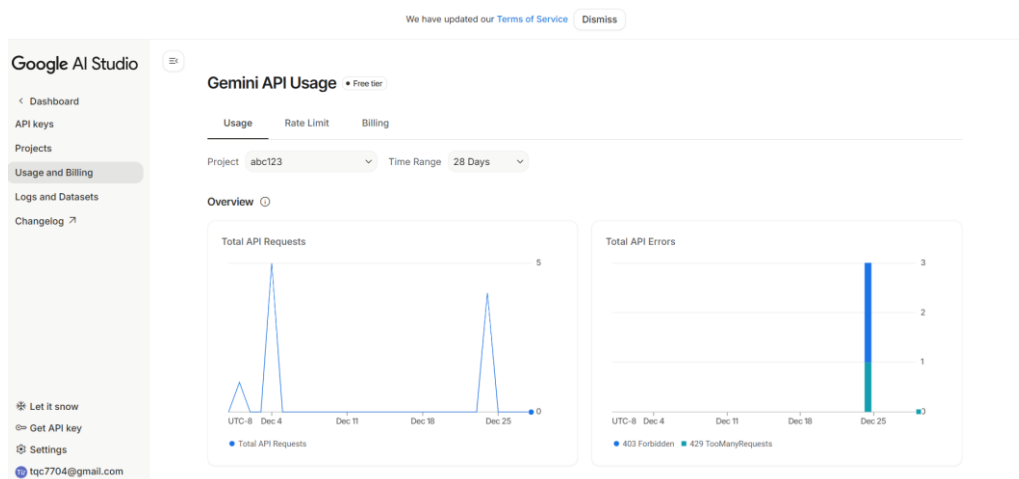
6.7 CẤU HÌNH TÍCH HỢP AI (GOOGLE GEMINI)

Để kích hoạt chatbot tư vấn thông minh:

1. Mở file cấu hình trung tâm: services\.env
2. Cập nhật khóa API
3. Khởi động lại **ai-service** để áp dụng cấu hình mới.

```
GEMINI_API_KEY=AIZAaSyBdUm0PTtZ3JZ0M5f6o58vABWpbnBUU22c
GEMINI_RATE_LIMIT_RPS=1
GEMINI_RETRY_MAX_ATTEMPTS=5
GEMINI_RETRY_BASE_BACKOFF_MS=2000
```

Hình 6.10 Giao diện cấu file api key gemini



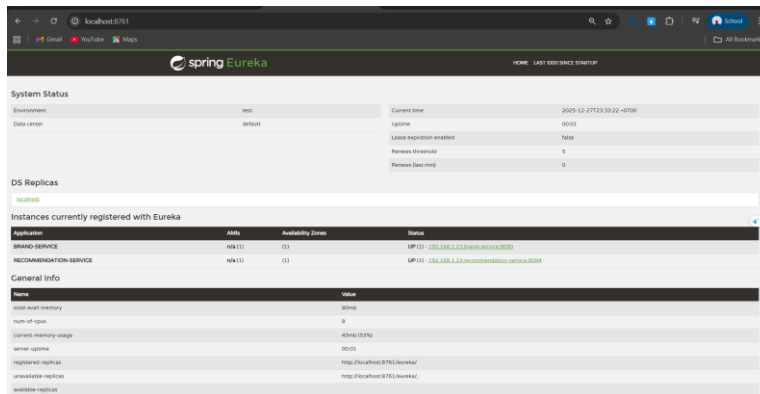
Hình 6.11 Giao diện trang lấy api key gemini link [Google AI Studio](#)

6.8 KIỂM TRA TRẠNG THÁI TRIỂN KHAI

Sau khi hoàn tất triển khai, tiến hành kiểm tra:

- **Trạng thái Microservices:** Truy cập <http://localhost:8761>, đảm bảo tất cả dịch vụ ở trạng thái UP.

- **Kiểm tra Gateway:** Truy cập <http://localhost:8080/actuator/health> để xác nhận hệ thống hoạt động bình thường.
- **Kiểm tra giao diện người dùng:** Đăng nhập, duyệt sản phẩm, đặt hàng và trò chuyện với AI Assistant thành công.



Hình 6.12 Giao diện trang Spring Eureka